Finite State Machines
Moore and Mealy (FSM)

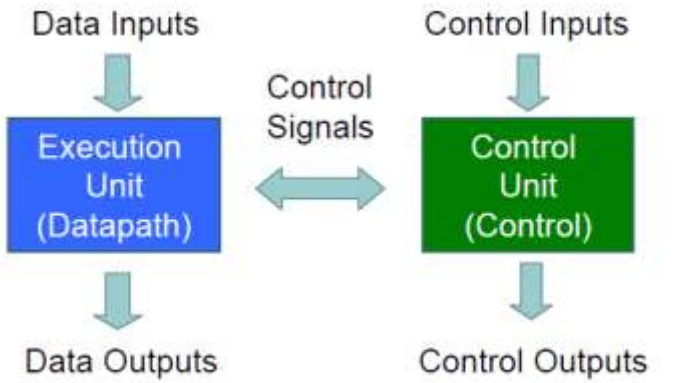
9/1/2026

Adapted from: CSE 140: *CPRE/COM S 583* – Prof. Joseph Zambreno, Department of Electrical and Computer Engineering, Iowa State University
Prof. Diba Mirza, Department of Computer Science and Engineering, University of California, San Diego.

Digital Systems (Microprocessors)

Execution Unit (Datapath)
Provides the necessary **resources and interconnections** to perform a specified task. **Examples of resources:**

- Adders
- Multipliers
- Registers
- Memories
- ALUs
- Multiplexers



Structure of a Typical Digital System

Control Unit (Control)

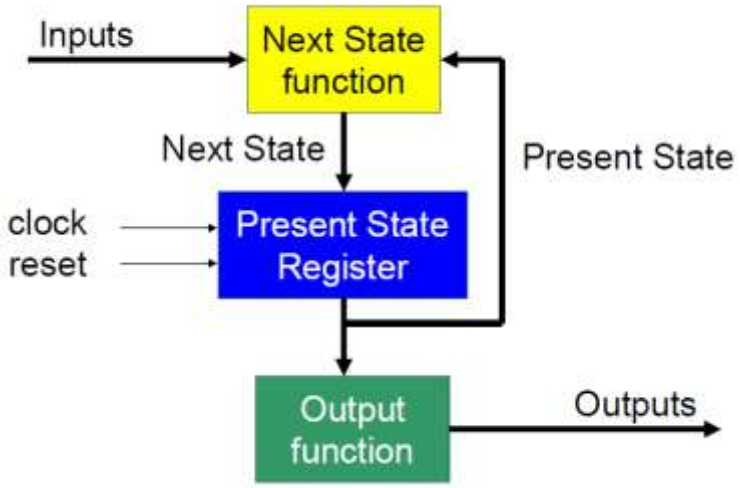
- Controls **data movement and operations** in the datapath.
- Switches **multiplexers** and enables or disables resources as needed.
- Follows a specific **program, sequence, or schedule** of operations.
- Often implemented as a **Finite State Machine (FSM)** or a collection of FSMs.

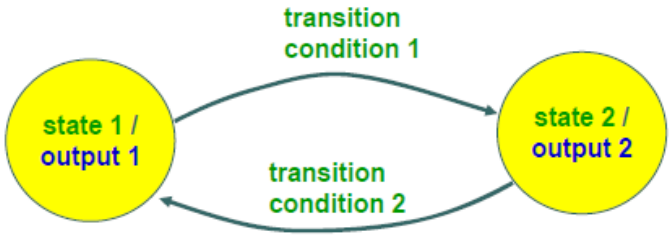
Finite State Machines (FSMs)

- Any digital circuit with **memory** can be viewed as a **Finite State Machine (FSM)**.
- Even a **computer** can be viewed as a very large FSM.
- Designing an FSM involves:
  - **Defining the states**
  - **Defining transitions** between states
  - **Optimizing or minimizing** the number of states
- This approach is most practical for **small FSMs**.

Moore FSM – Definition

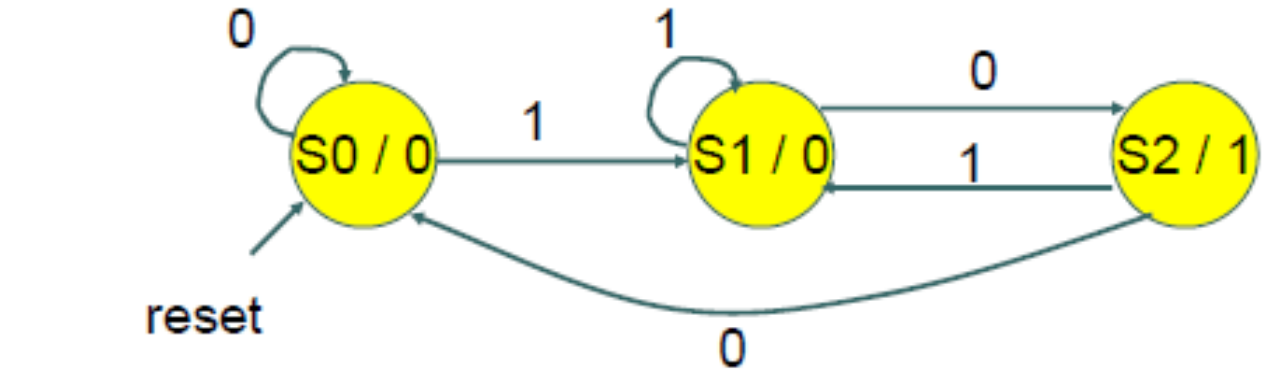
In the Moore FSM the Output is a function of present state only





Moore FSM – Example

Moore FSM that recognizes sequence “10”

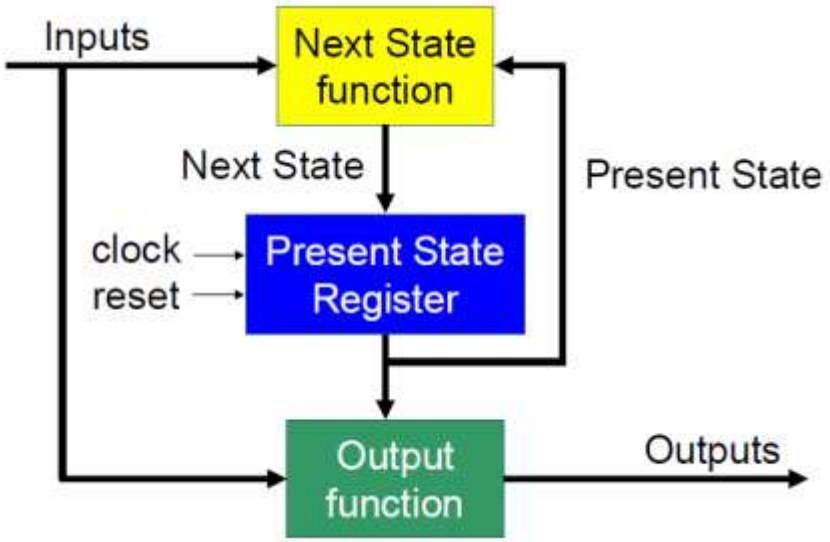


Meaning of states:

- S0: No elements of the sequence observed
- S1: “1” observed
- S2: “10” observed

Mealy FSM – Definition

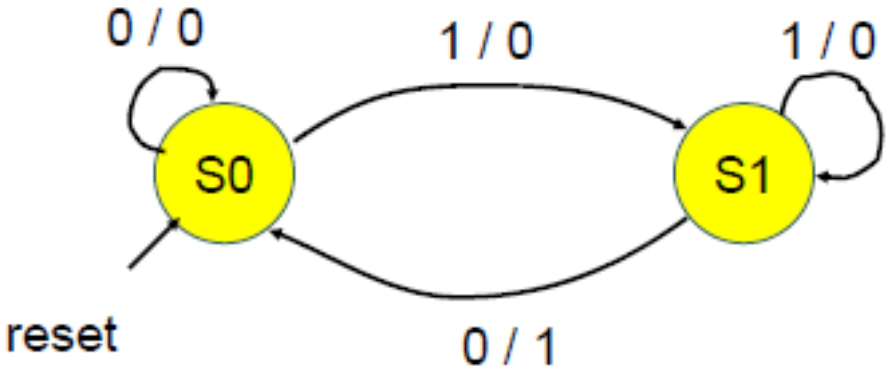
In the Mealy FSM, the Output is a function of a present state and inputs





Mealy FSM - Example

Mealy FSM that recognizes sequence “10”



Meaning of states:

S0: No elements of the sequence observed

S1: “1” observed

Moore vs. Mealy FSM

- **Moore and Mealy FSMs can be functionally equivalent.**
- An equivalent **Mealy FSM can be derived from a Moore FSM**, and vice versa.
- **Mealy FSMs** typically require fewer states and provide a more compact description.
- Fewer states can result in **smaller circuit area**.
- A **Mealy FSM** computes its outputs based on the **current state and current inputs**.
- A Mealy FSM can respond to an input change **within the same clock cycle**, potentially responding one clock cycle earlier than an equivalent Moore FSM.
- A **Moore FSM** computes outputs based **only on the current state**.
- A Moore FSM has **no combinational path directly from inputs to outputs**.
- Moore FSMs are therefore often easier to synchronize and may have a **shorter critical path**.

Practical Pattern Recognizer: Life on Mars?

Pattern Recognizer‘001’ Example!

Assume that a sequential machine receives a **binary input x** from a remote robot on Mars. When the machine receives the input sequence:

x(t−2, t) = 001 the robot has detected **life on Mars**, so: **y(t) = 1**

For all other input sequences:

y(t) = 0 (no life on Mars).

Draw the Moore FSM for this pattern recognizer.

- Define the required states.
- Show all state transitions for inputs 0 and 1.
- Assign the appropriate output (y = 0 or y = 1) to each state.
- Identify the state representing **“Life Detected: 001.”**
- **Use one hot controller to show the hardware of the FSM**

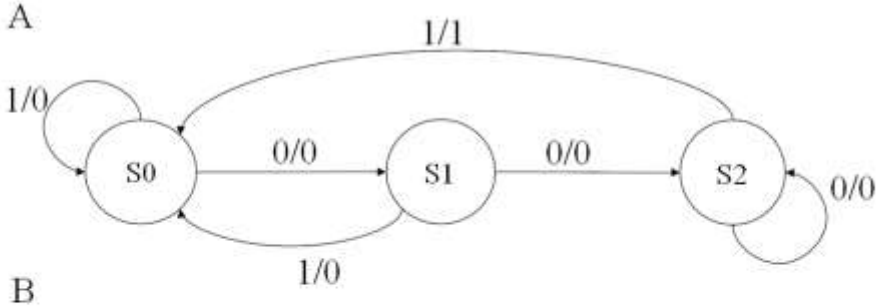
Adapted from: CSE 140: Components and Design Techniques for Digital Systems, Diba Mirza, Department of Computer Science and Engineering, University of California, San Diego.

Pattern Recognizer: Life on Mars?

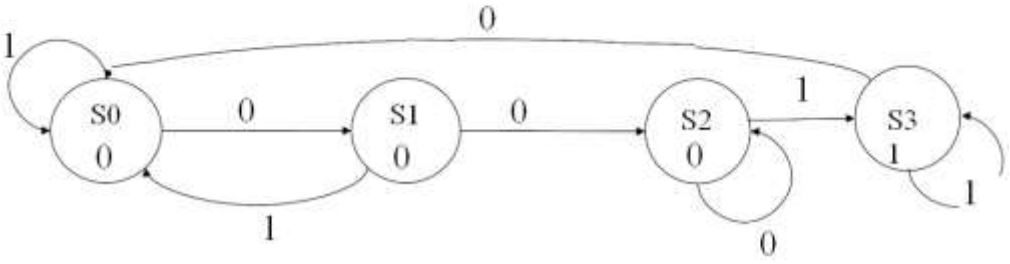
Which of the following diagrams you think is a correct Moore FSM solution

for the "001" pattern recognizer?

A



B

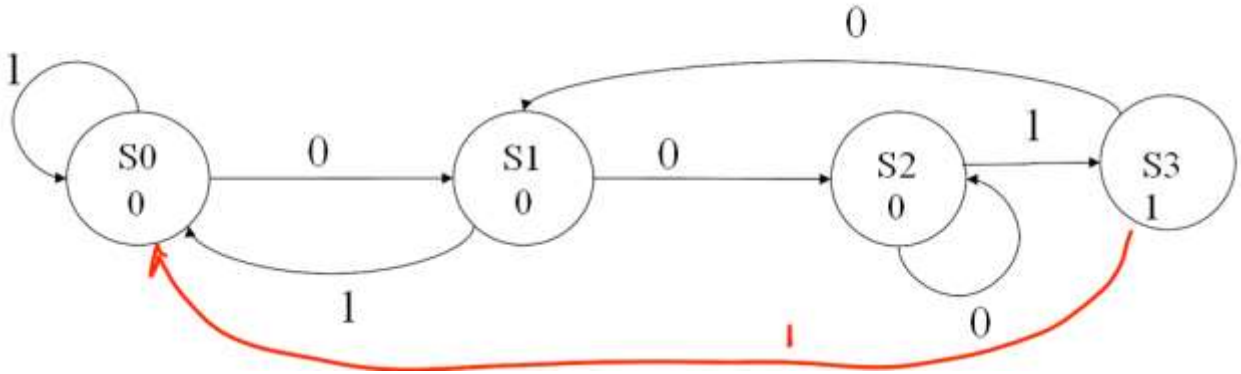


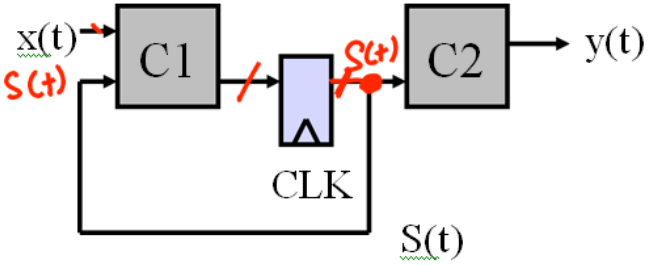
C- Both A and B are correct

D- None of the above

Pattern Recognizer: Life on Mars?

Pattern Recognizer‘001’



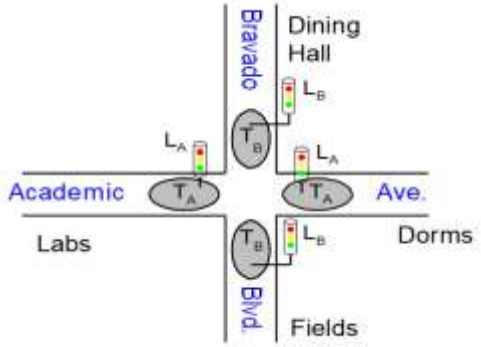


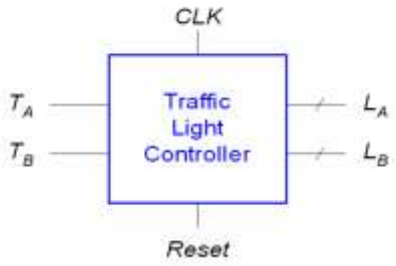
Moore Machine

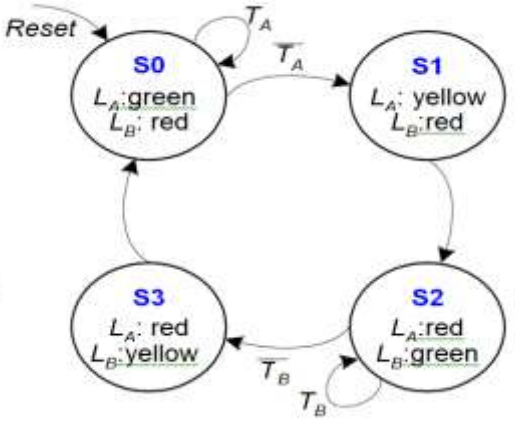
In a Moore FSM, what determines the output (y(t))?

A. The current input (x(t)) only
B. The current state (S(t)) only
C. The current input (x(t)) and current state (S(t))
D. The current state (S(t)) and next state (S(t+1))
E. The next state (S(t+1)) only

Finite State Machine Example







Traffic Sensors:
(TA): TRUE when there is traffic on Road A
(TB): TRUE when there is traffic on Road B

Traffic Lights:
(LA): Traffic light for Road A
(LB): Traffic light for Road B

Inputs:
CLK — Clock signal
Reset — Resets the controller to its initial state
TA — Traffic sensor for Road A
TB — Traffic sensor for Road B
Outputs:
LA — Traffic light control for Road A
LB — Traffic light control for Road B

Moore FSM:

- Outputs are **labeled in each state**.
- The output depends **only on the current state**.
- A state is written as: **State / Output**

Which of the following is true about the controller?

A. The traffic light on Academic Ave. (LA) remains green as long as there is traffic on that street.
B. The traffic lights on both avenues are green for exactly one clock cycle in every four clock cycles.

Course: Fall 2026 Embedded Systems (CEG-7360-01)
Topic: Programming FPGA — Combinational and Sequential Logic

By

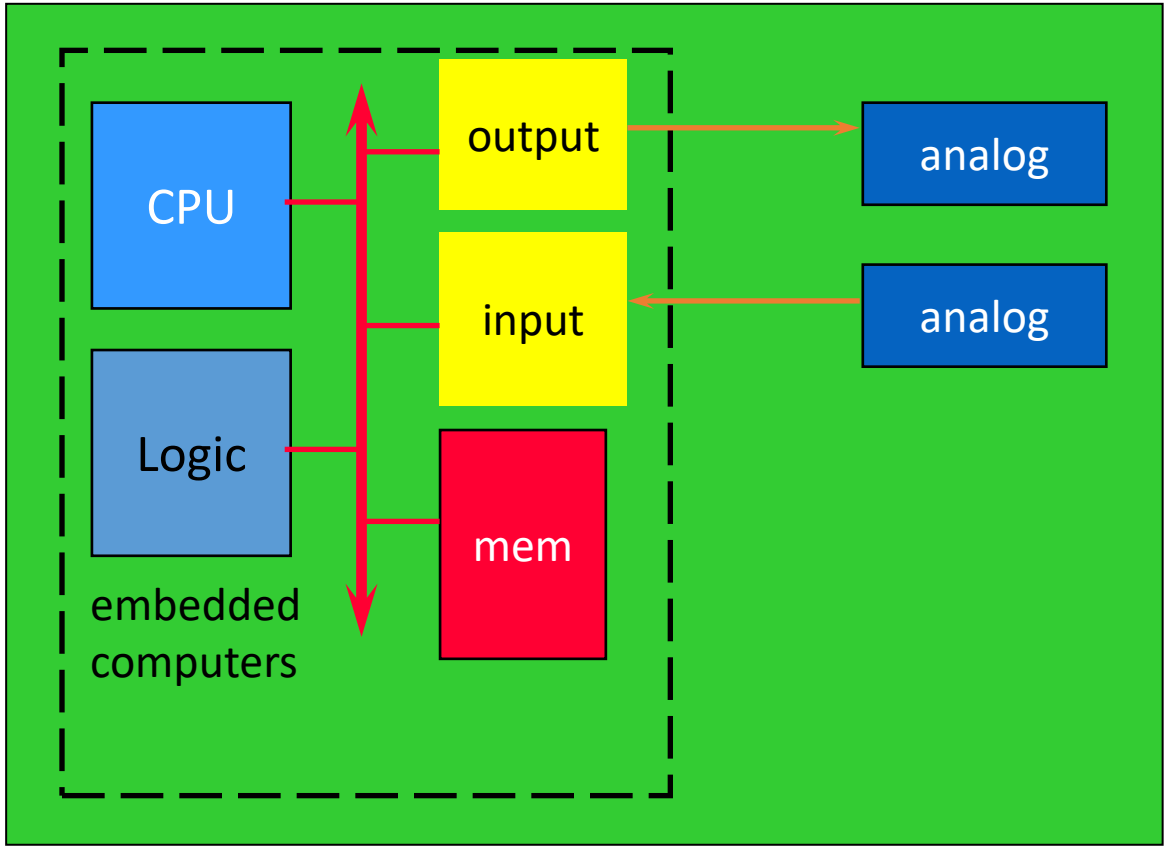
Fathi Amsaad

9/8/2026

Some slides adapted from **Dr. Tahrun, UF–EE 505: Digital Systems — VHDL, Verilog, and FPGA Programming.**

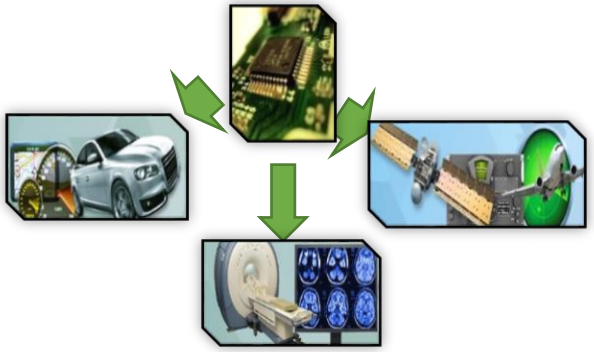
Embedded Computing System?- Definition

- Not a general-purpose computer
  - ❖Optimized according to the specific needs of an application
  - ❖No need for all the general-purpose characteristics (registers, architecture)



Embedded Computing System- Emerging Application

- Automotive;
- Aerospace;
- Medical;
- Telecommunication;
- Networking;
- Consumers;
- Portable and Wireless;
- DoD;
- And US Air force.























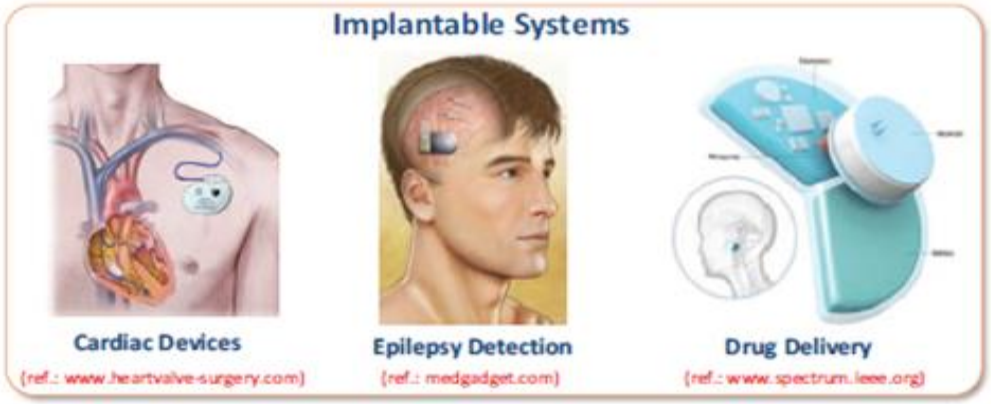


Embedded Computing System- Emerging Application

- Computing systems expected to be deeply embedded within physical objects and people
- This enable a wide spectrum of applications such as:
  - Internet of Things or IoT sensors and Infrastructures
  - Smart buddings and Smart meters
  - Smart and autonomies Vehicles
  - Implantable and wearable medical devices
    - Vitally essential for patient and perform life critical functions







Embedded Computing System- Smart Vehicle Example

ECUs (Engine Control Units), any idea?

1- Hardware: PCB (printed circuit board) ?

Real-time sensors (i.e. engine, emergency brake, seat control, etc.)

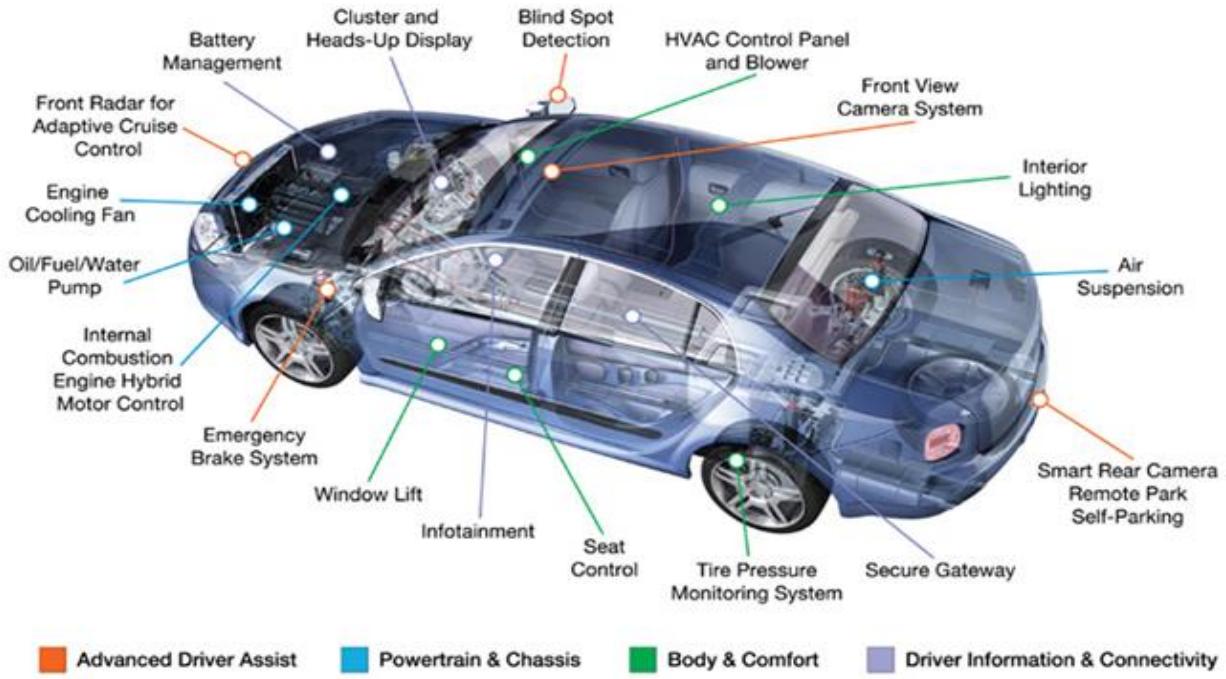
- Real-time processing (Microcontroller, FPGAs, etc.)
- Storage EPROM (Re-programmed)
- To upload and update codes

2- Software ?

- Firmware: sort of the OS (operating system): Perform control, monitoring and data manipulation functions.

The total number of ECUs a modern automobile contains, any guess! ?

From 50 to100 and the number is in increase !

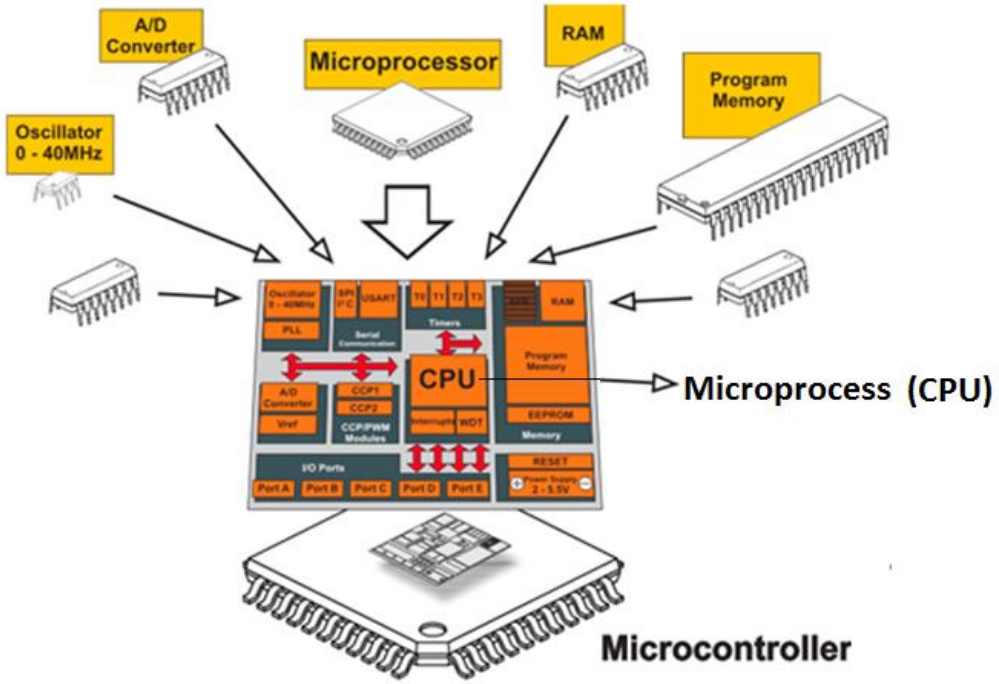


Embedded Computing System- Integrated Circuits (ICs)

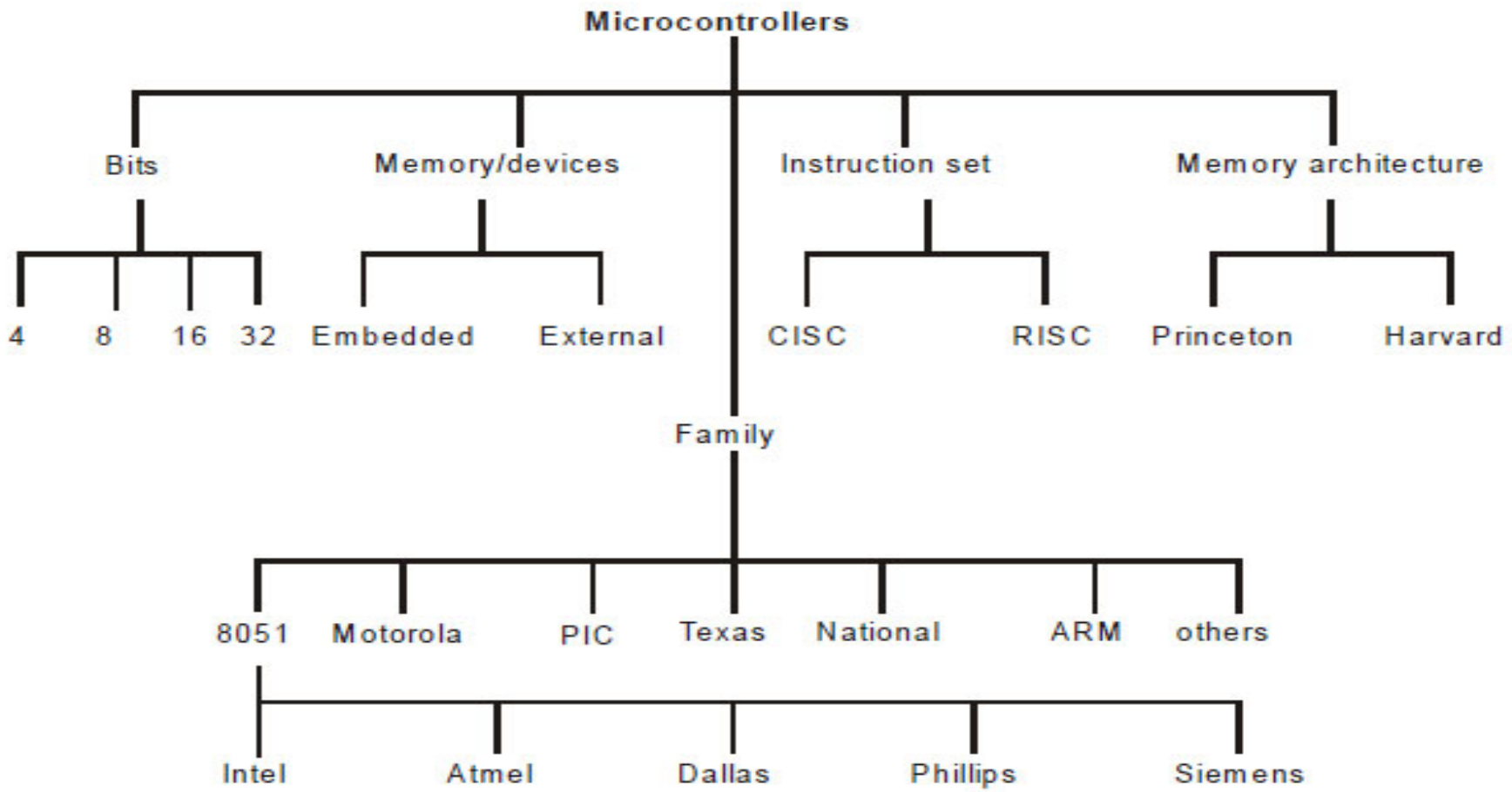
- Application Specific Integrated Circuits (ASIC) design
  - Fixed Fabrication or design of a specific IC;
  - Uses Cadence and Mentor-graphics simulation tools;
  - Higher performance for greater logic density custom design;
  - Microcontroller, GPU, Microprocessor.
- Programmable Logic Devices (PLDs)
  - Re-programmed and Flexible Design;
  - Large size (millions of logic gate) and high performance characteristics;
  - Have HDLs CAD tools (Xilinx and Altera etc.) enable simulation and automate device reprogramming;
  - Variety of types PLAs, PALs, EPROM, and FPGAs etc.

Embedded Computing System- ASICs Microprocessor vs Microcontroller

- Microcontroller chip
  - A small computer embedded on a silicon chip containing:
    - Microprocessor (CPU);
    - Memory unite(MU);
    - Input and output (I/O) devices;
    - and other peripherals;
- Microprocessor
  - Incorporates the functions of a computer's central processing unit (CPU) on an integrated circuit (IC)
    - Contain Data Path Units
    - Controller Units



Microcontroller Categories



CISC vs. RISC Architectures

- CISC – Complex Instruction Set Computers
- RISC – Reduced Instruction Set Computers
- Below is an example of the different architectures for doing the same operations
- Suppose you wanted to multiply two numbers in memory locations *mem0* & *mem1* and store the results back in *mem0:*
  - *For CISC you use one complex instruction Mull mem0, mem1*
  - Same result but the complexity of operations and the number of steps used in the two cases differ

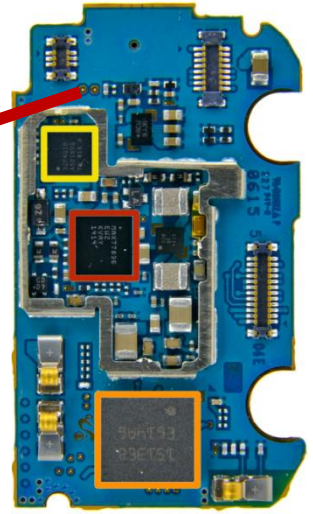
CISC	RISC
Example – Intel x86 chips	Examples – ARM, PowerPC
Large number of complex instructions more than 500	Few small complexity instructions, typically less than 100
Variable-length instructions, instructions can range from 1-15 bytes	Fixed-length instructions, all instructions have the same number of bytes
Some instructions can have very long execution times	No instruction with a long execution times
Poor pipelining	Designed for pipelining

CISC Approach	RISC Approach

`mull mem0, mem1`	`ldr r0, mem0`
	`ldr r1, mem1`
	`mul r0, r0, r1`
	`str mem0, r0`

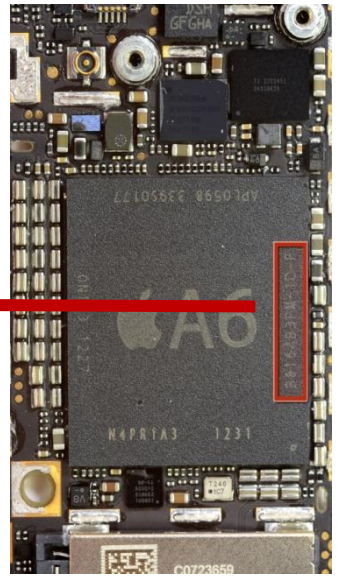
Embedded Computing System- Microcontroller Applications



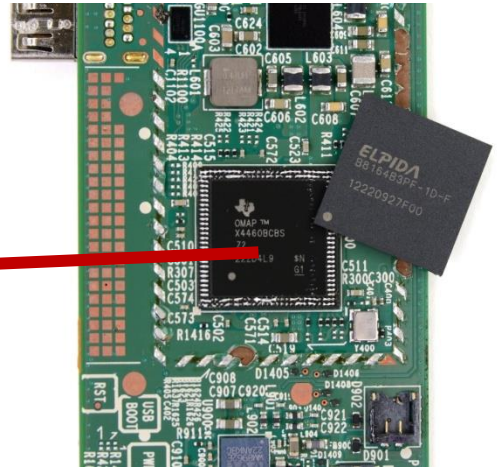


Apple watch S1
1 core 32-bit ARMv7-A
with 28 nm CMOS Technology

I Phone 6 System-on-Chip (SoC) design based off the **ARMv7** Microcontroller.



A tablet computer developed by Texas Instruments **Dual-core** Microcontroller.

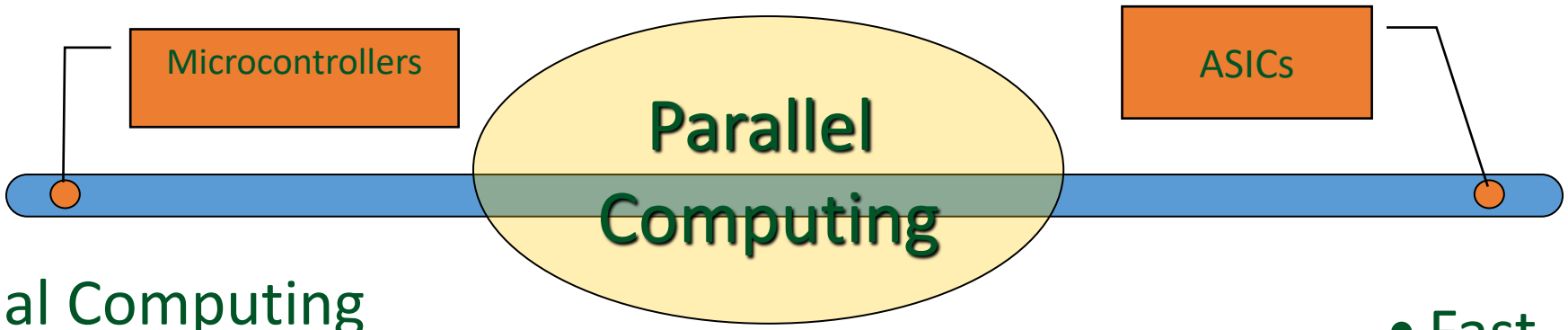


http://www.ifixit.com

Embedded Computing System- Reconfigurable Computing -Field Programmable Gate Arrays (FPGAs)

Semiconductor Integrated Circuits (ICs) are the core elements of these systems

- Slow
- Flexible
- Sequential Computing



- Fast
- Inflexible

?

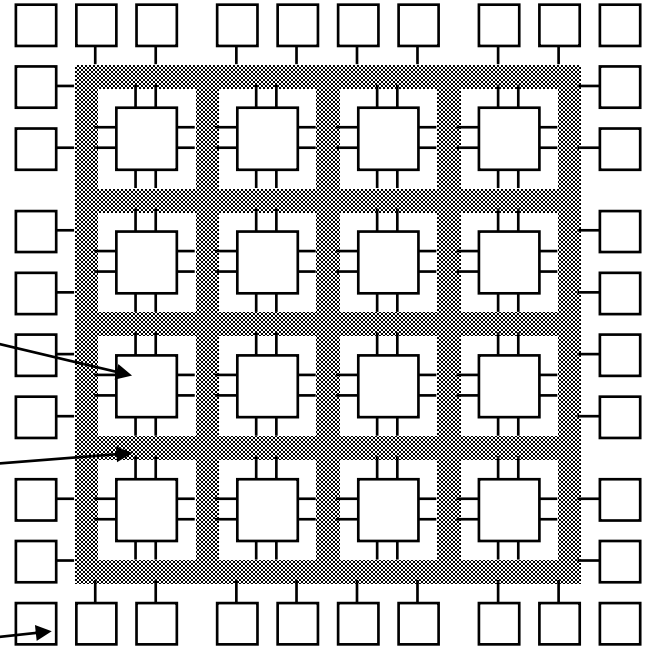
Reconfigurable Computing (RC) Platforms

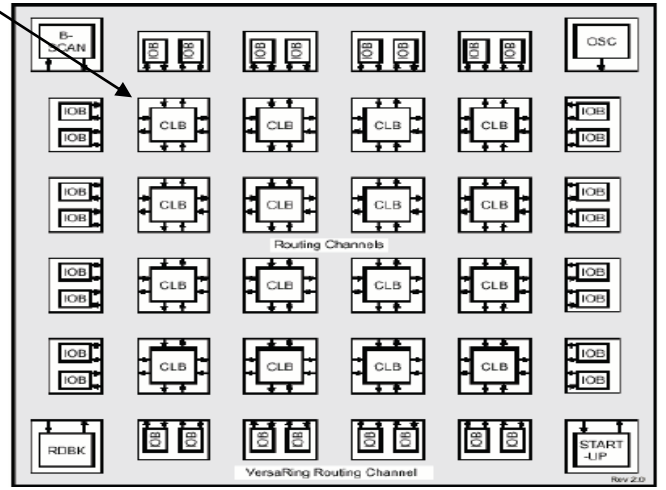
E.g.: PALs, FPGAs, etc.

(Field Programmable Gate Arrays)

Field-Programmable Gate Arrays

- Configurable Logic blocks (CLBs)
  - To implement combinational and sequential logic using Look up Tables (LUTs), Muxs, F.F, etc.
- Interconnect
  - Wires to connect inputs and outputs to logic blocks
- I/O blocks
  - Special logic blocks at periphery for external connections
- Key questions:
  - What makes logic blocks programmable?
  - how to connect the wires?





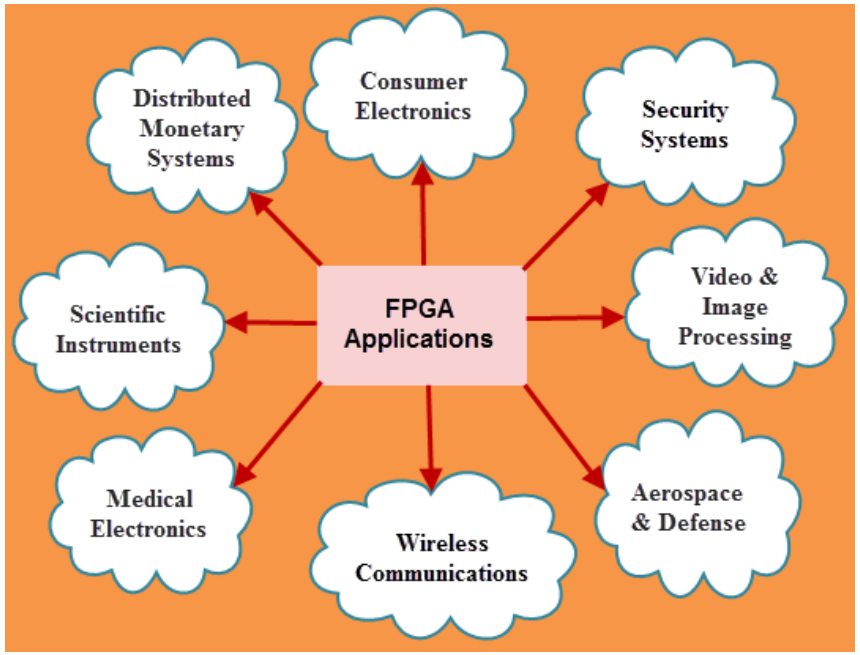
Why FPGA - Advantages

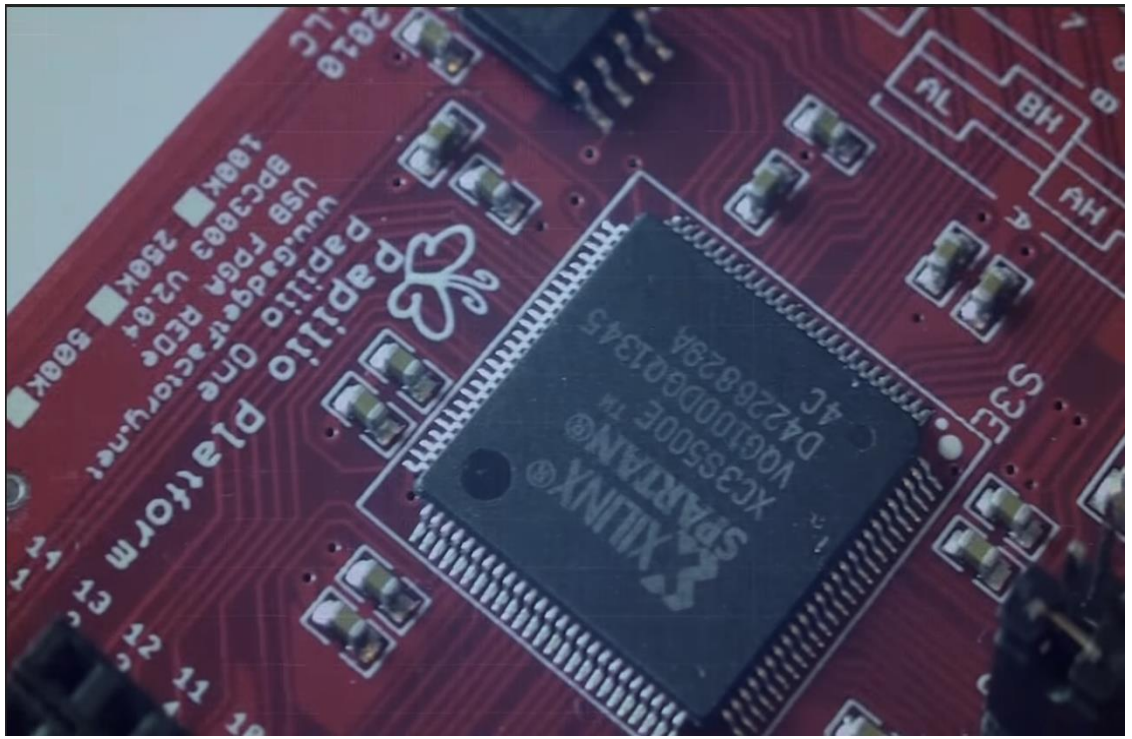
- **Very fast custom logic**
  - Massively parallel operation
- **Faster than microcontrollers and microprocessors**
  - Much faster than CPU engines: An FPGA is faster than a CPU in parallel computing because it processes data using custom hardware circuits built for a specific task rather than running sequential software instructions.
- **More affordable and less risky than ASICs**
- **Reprogrammable at any time**
  - in design, in manufacturing, after installation
- **Offer the advantages of high integration**
  - High complexity, density, reliability
  - Low cost, power consumption, small physical size
- **Avoid the problems of ASICs**
  - Long delay in design and testing
  - Increasingly demanding on supply chain electrical issues

Why FPGA - Advantages

- **Large dedicated Logic capacity at reasonable cost**
  - Several million gates
  - Offer On-chip fast RAM and DSP and CPU cores
- **Clock speed**
  - Generally operates in a lower clock, not like CPUs, (typically 50 MHz – 800 MHz), global clocks, clock management
- **Design effort and time**
  - synthesis, fast compile times, tested and proven cores
- **Power consumption**
  - must stay within reasonable limits

Field Programmable Gate Arrays (FPGAs)- Architecture and Applications

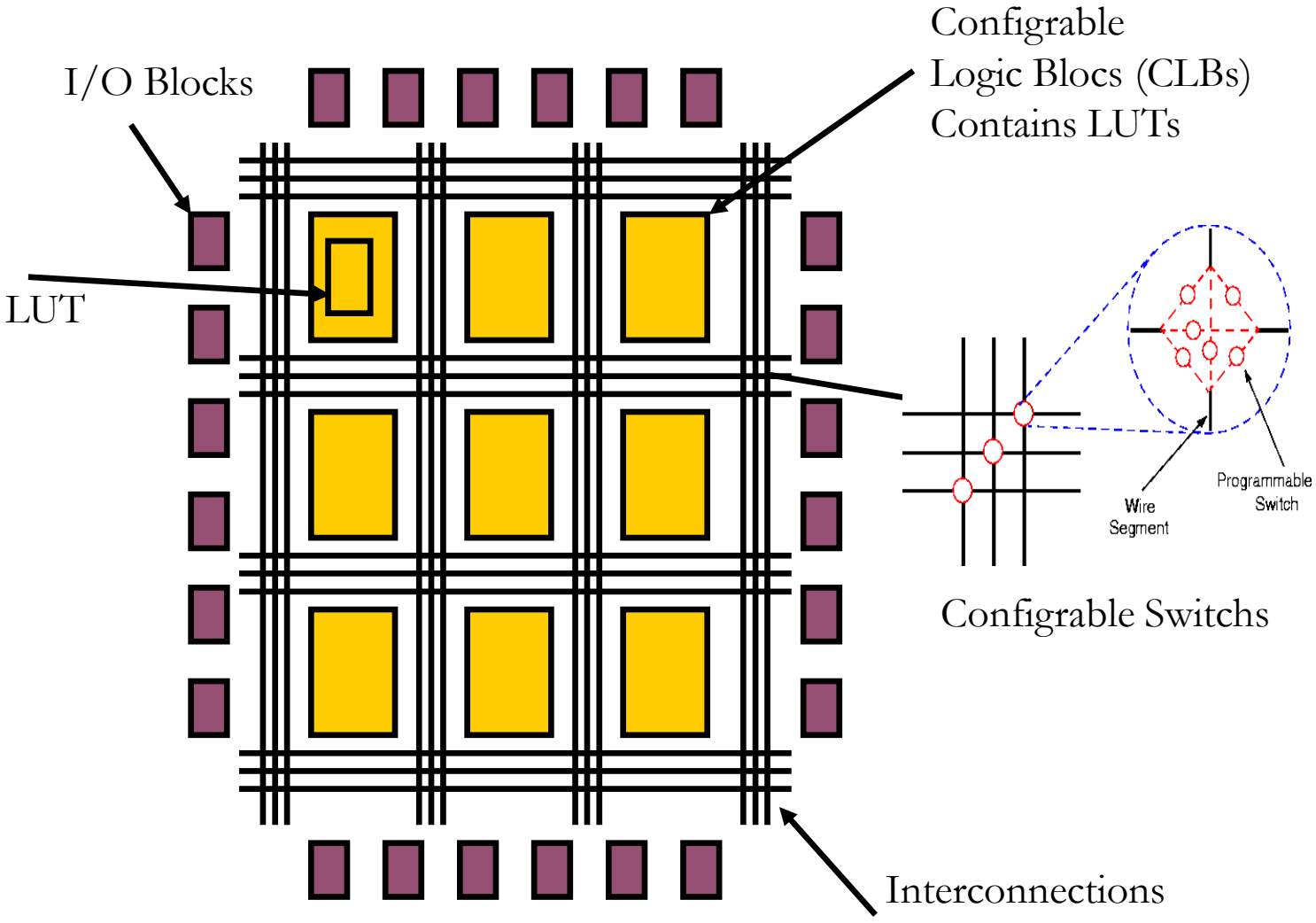




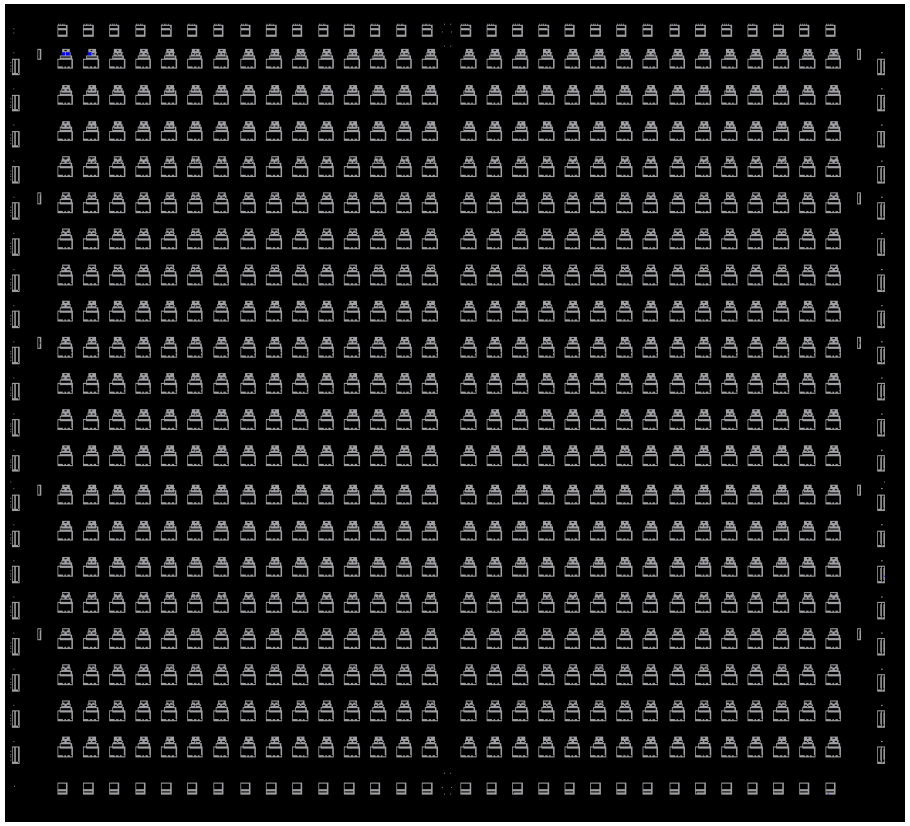
https://www.youtube.com/watch?v=5qNAJEIqk4Q

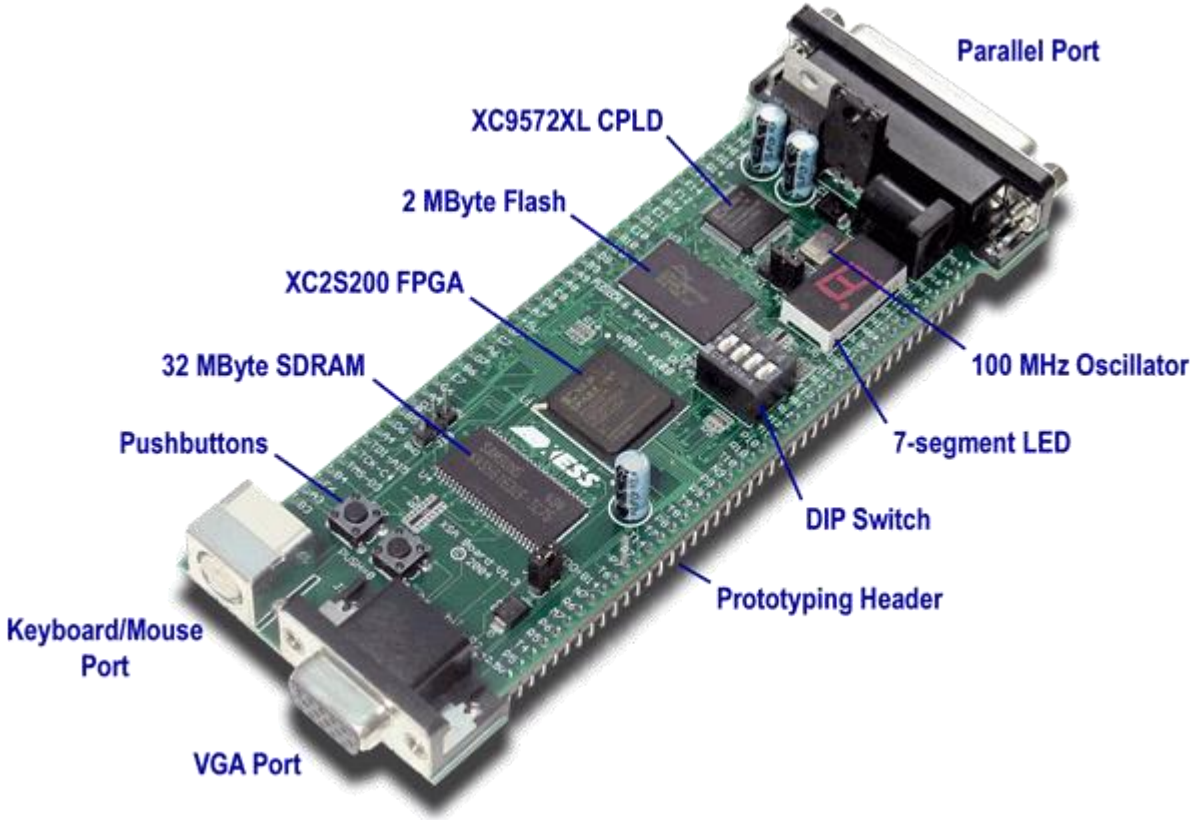
Embedded Computing System- Field Programmable Gate Arrays (FPGAs) Architecture

Identiocal Array of N columons and M rows:
1-Configrable Logic Blocs (CLBs)
2- Interconnets
3- I/O Blocks

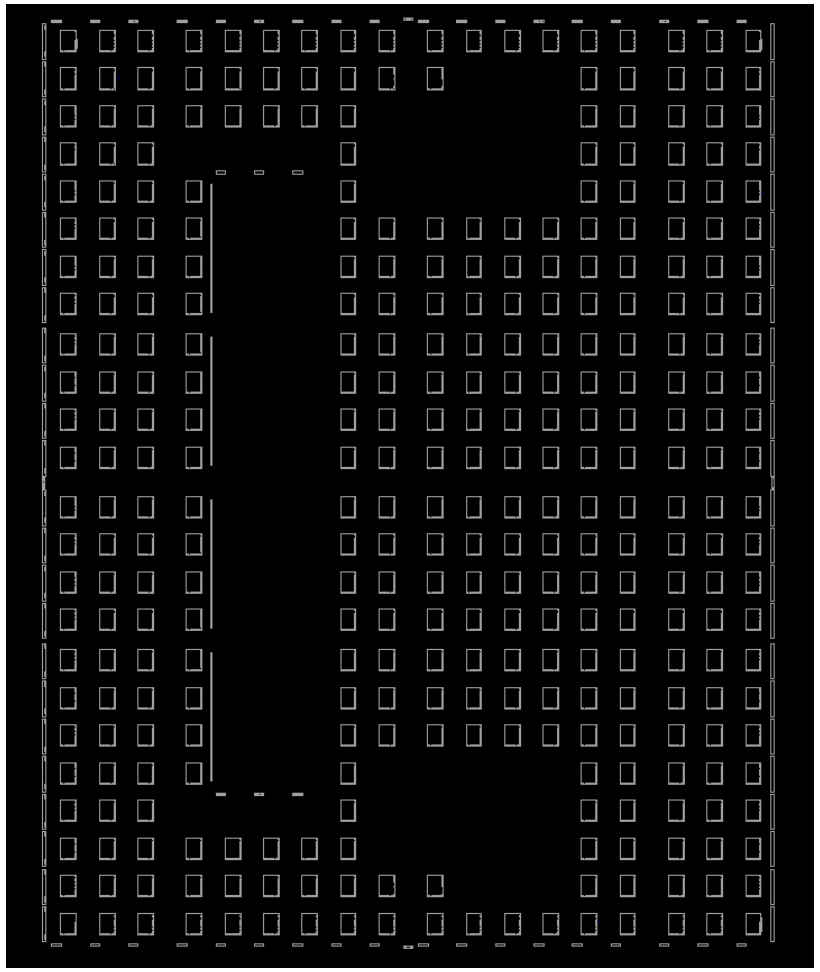


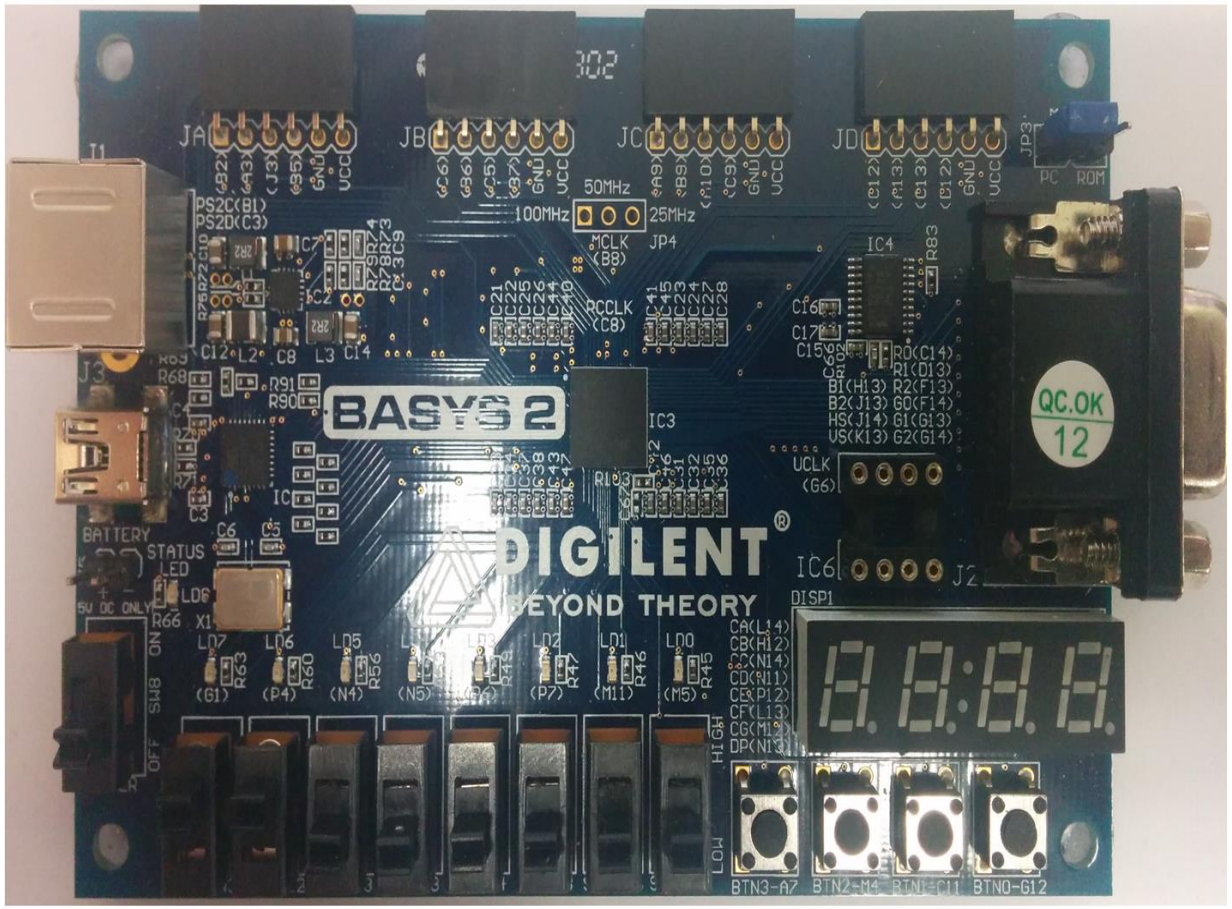
Embedded Computing System- Field Programmable Gate Arrays (FPGAs) Architecture Spartan 2 layout (128 nm technology)



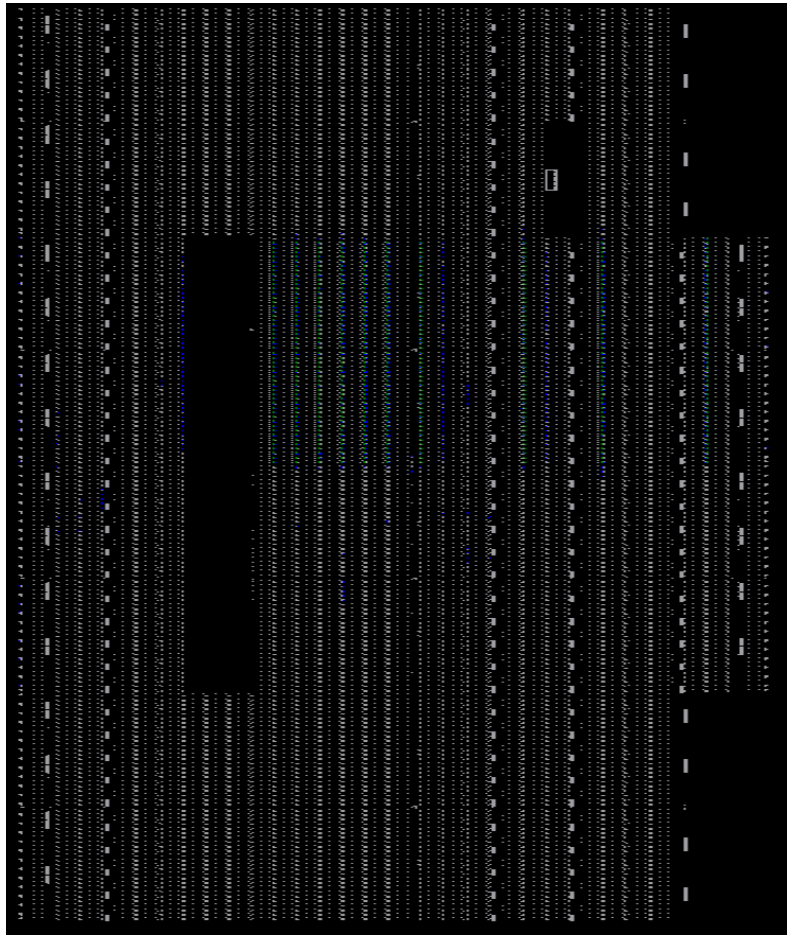


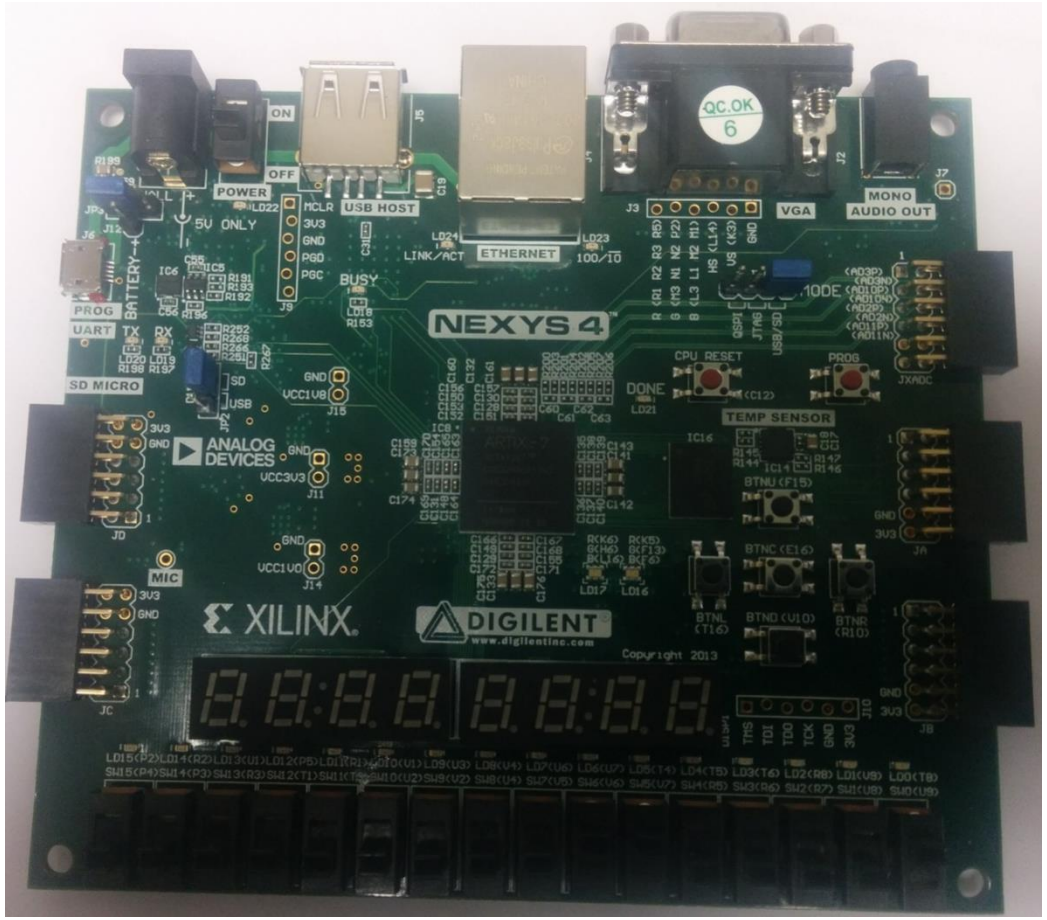
Embedded Computing System- Field Programmable Gate Arrays (FPGAs) Architecture Spartan 3E layout (90 nm technology)



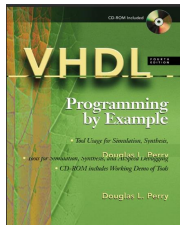


Embedded Computing System- Field Programmable Gate Arrays (FPGAs) Architecture Artix-7 layout (28 nm technology)



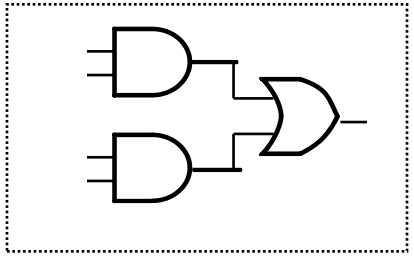


Steps to Implement Digital Logic on FPGA

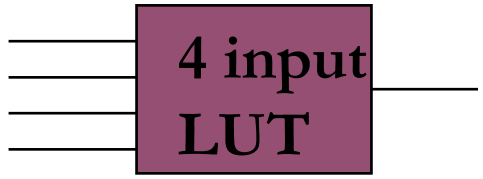


1-Program in C/VHDL/Verilog (Xilinx ISE, Altera quartos , Vivado, etc.)

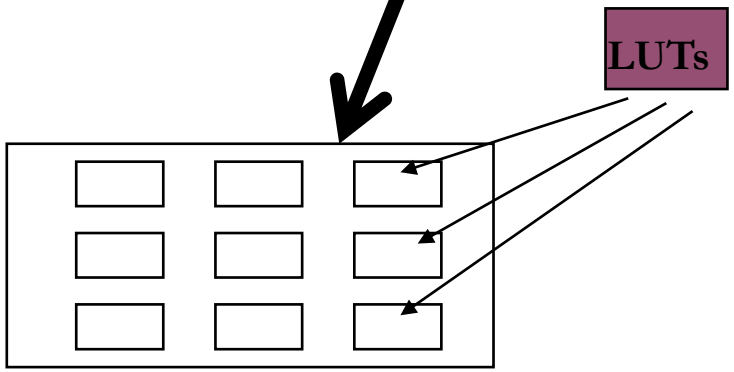
2- Circuit Mapping



Lookup Tables (LUTs)



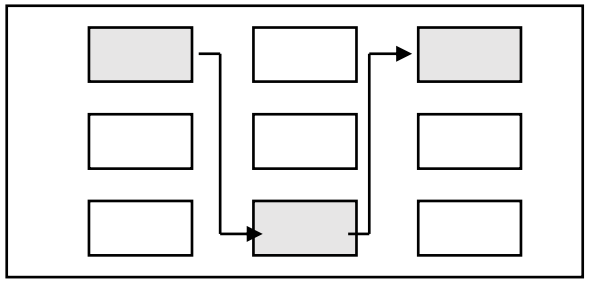
3- Placement



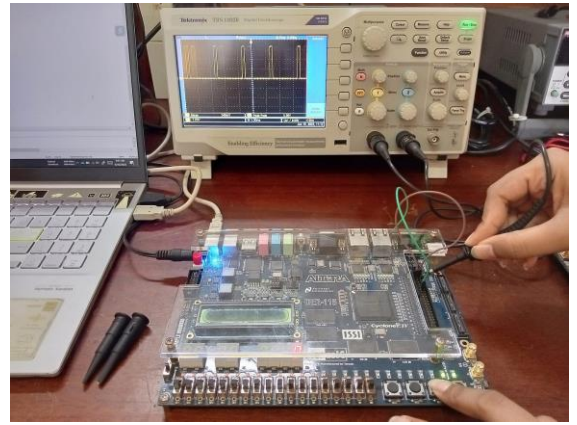
Assign a logical function to a physical location (LUT).

4- Routing

Select wire segments and switches for Interconnection.



5- Convert all implementation "Design Speciation" to bit stream files(configuration bits): LUT SRAM bits, SRAM switches bits, etc. Use FPGA to implement your logic multiple times. ROM and load into FPGA as needed



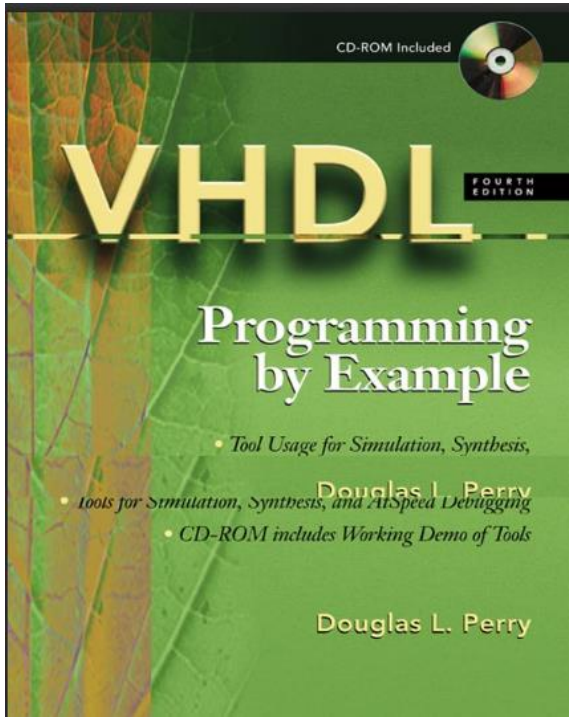
FPGAs Programming

Configuration Files (Bit stream)
110011101 ...

FPGA

Can you Build LUT1? This is Part of your homework today

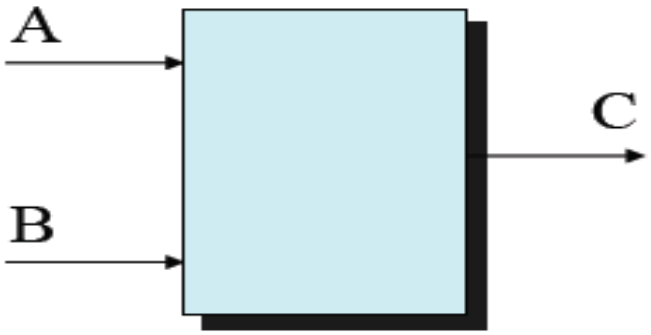




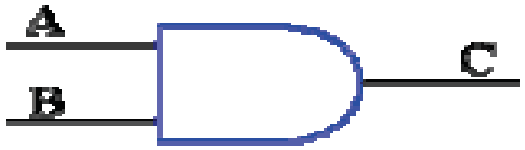
https://www.ics.uci.edu/~jmoorkan/vhdlref/vhdl_golden_reference_guide.pdf
http://www.srmuniv.ac.in/ramapuram/sites/ramapuram/files/EC308.pdf
http://www.ics.uci.edu/~jmoorkan/vhdlref/Synario%20VHDL%20Manual.pdf

FPGA Structures - Basic Lookup Table (LUT) Idea

A	B	C
0	0	0
0	1	0
1	0	0
1	1	1

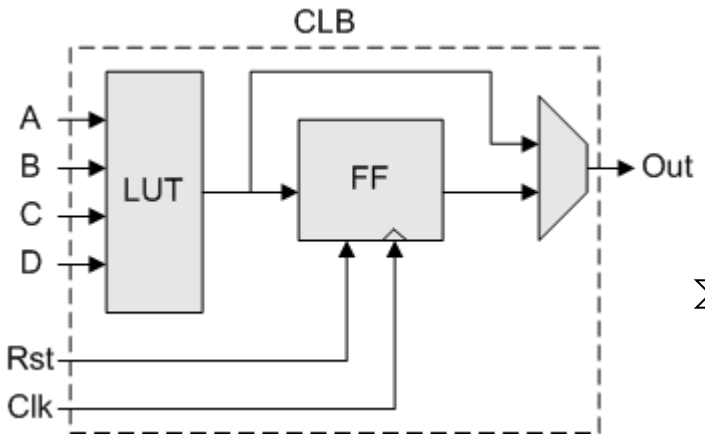


=

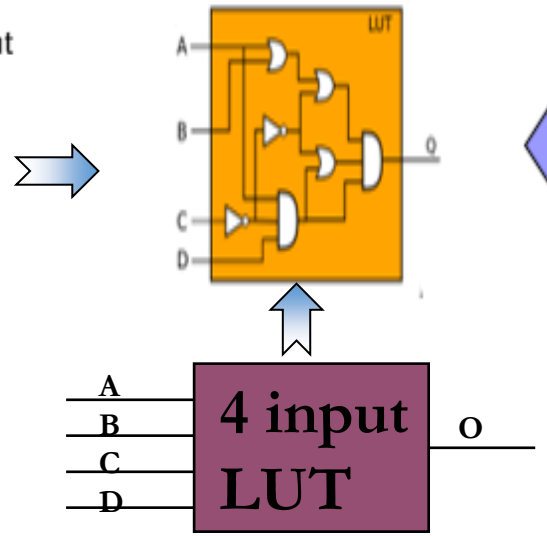


Field Programmable Gate Arrays (FPGAs)- Look UP Table (LUT)?

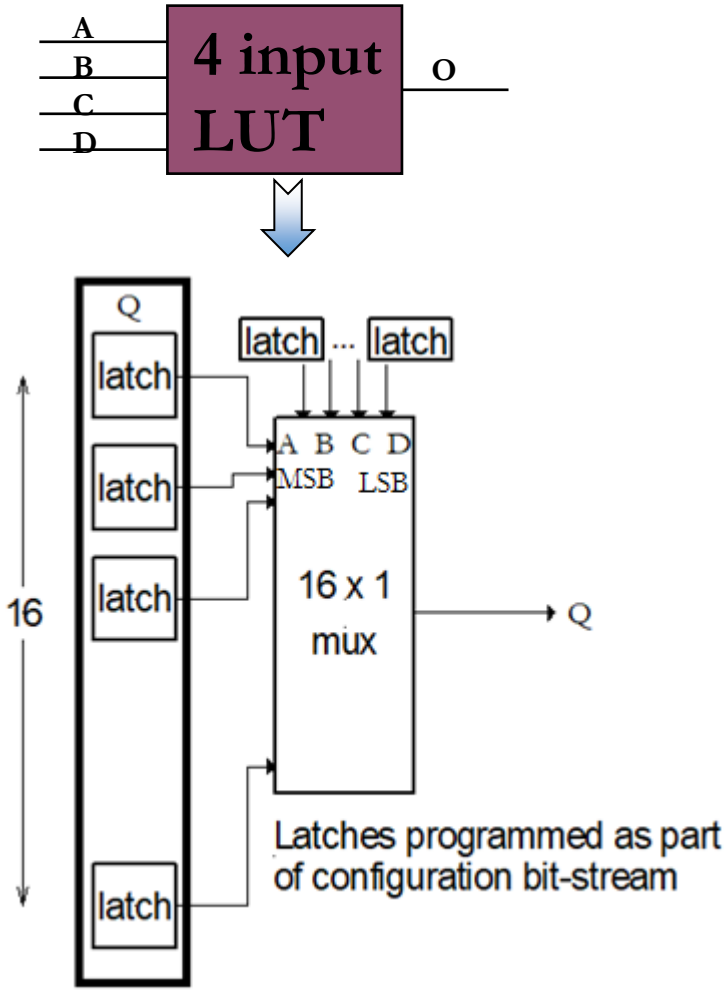
- Identical CLBs are divided into Slices (2, 4, 6)
- Each Slice contain N LUTs (4-inputs, 6 inputs LUTs)
- An LUT is an SRAM direct implementation of a truth table
- LUTs implement any n-input combinational logic depends on the type of the FPGA



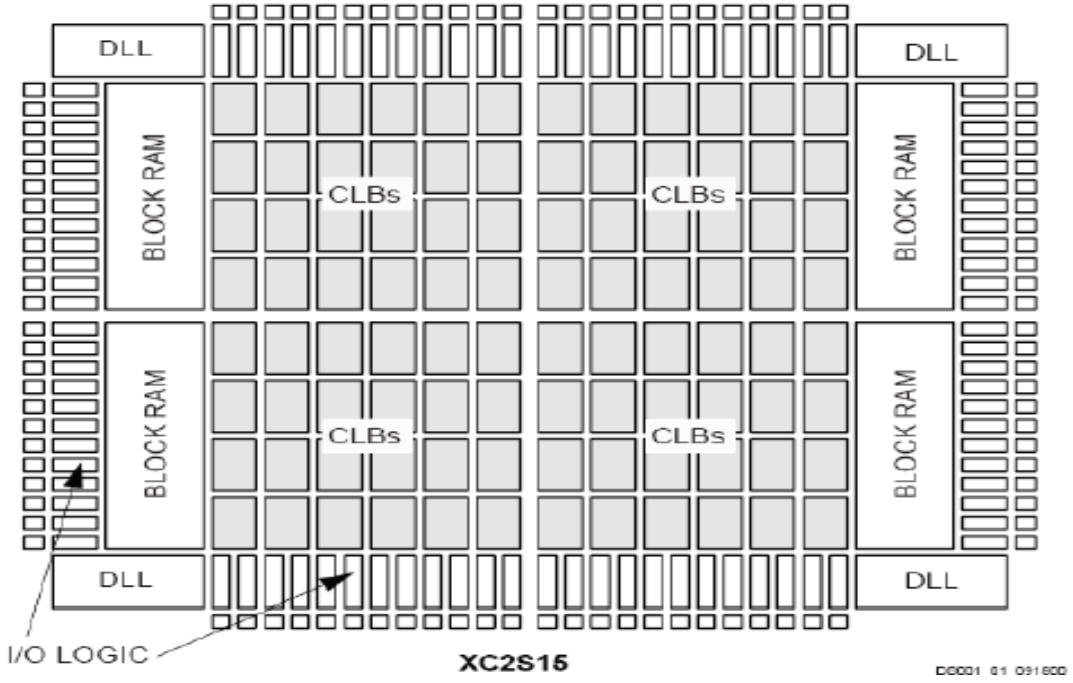
Configurable Logic Block (CLB)

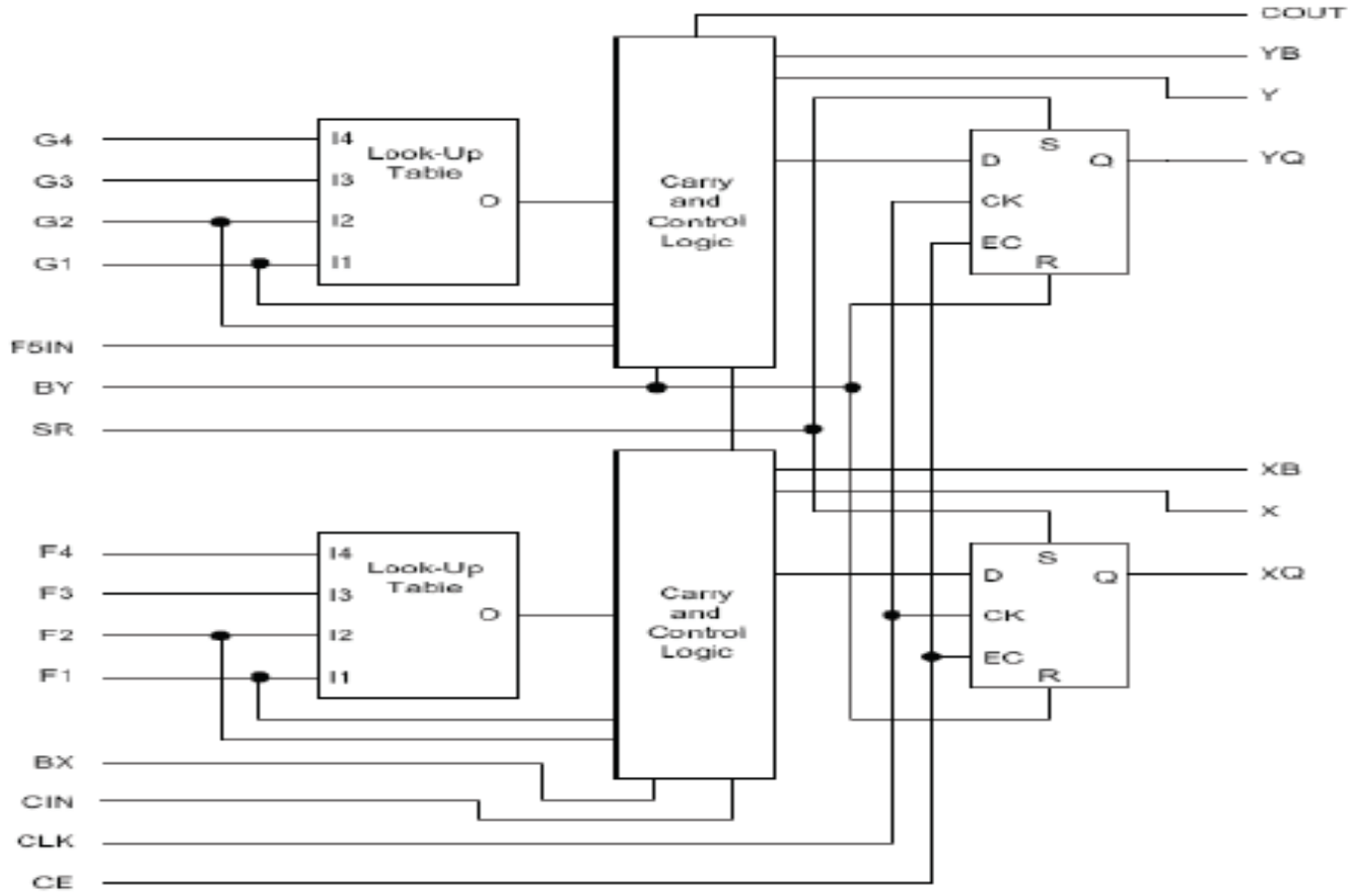


A	B	C	D	Q
0	0	0	0	0
0	0	0	1	0
...				
1	1	0	1	1
1	1	1	0	0
1	1	1	1	0



Spartan-II FPGA Family – CLBs and Look up Table (LUT)

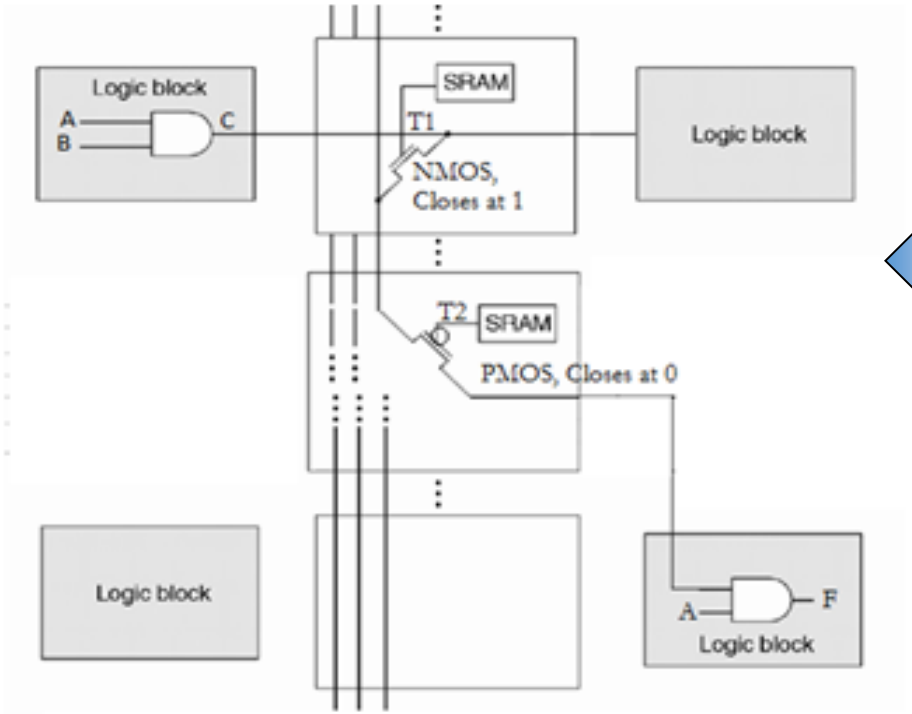


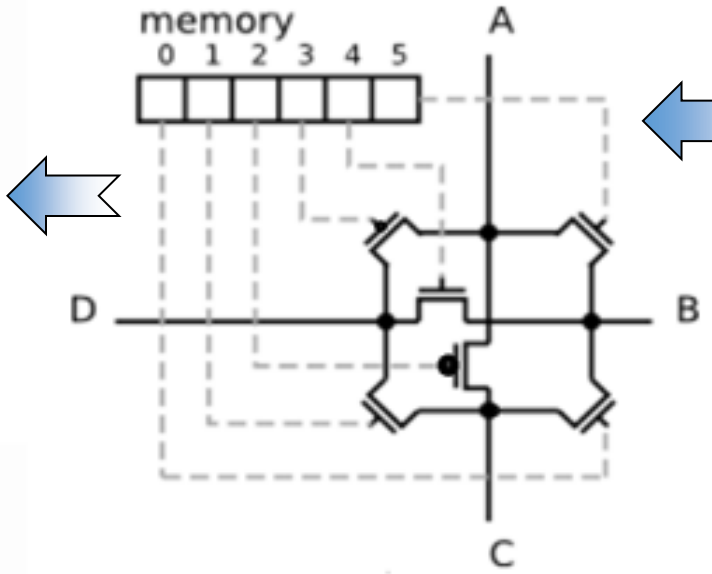


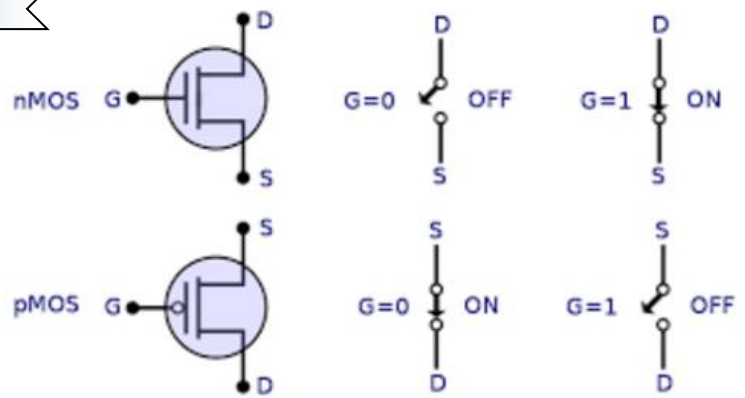
Field Programmable Gate Arrays (FPGAs)- Programmable Switches?

1- What should be the valoues of T1 and T2?

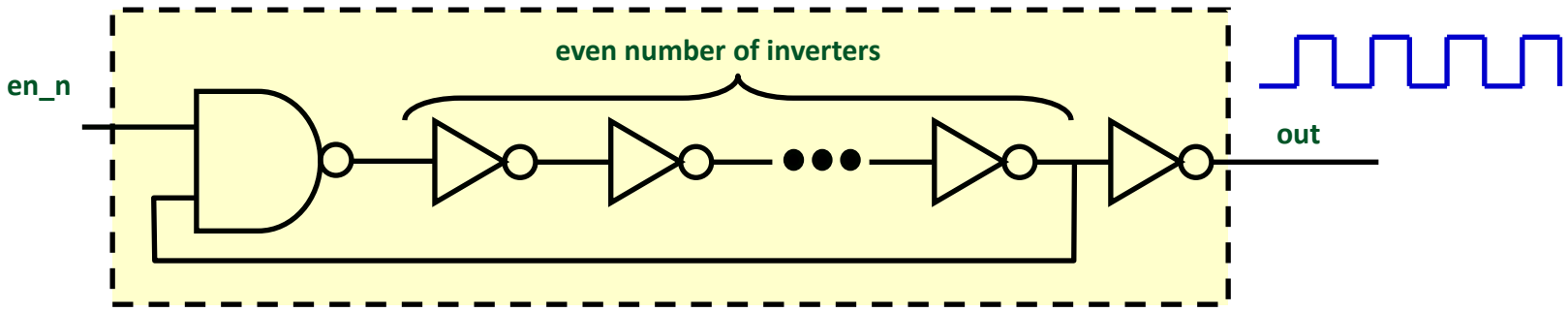
2-What is F?







Ring Oscillator

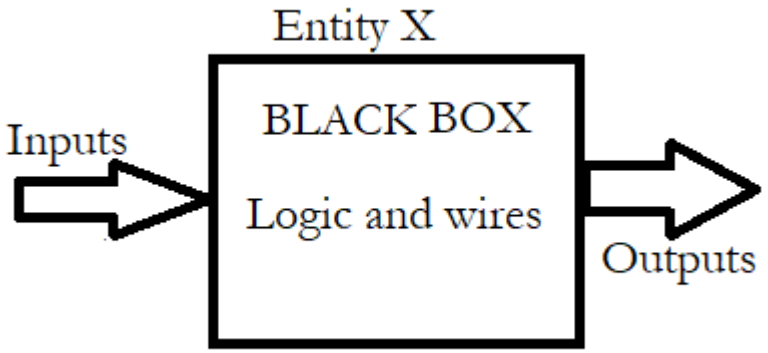


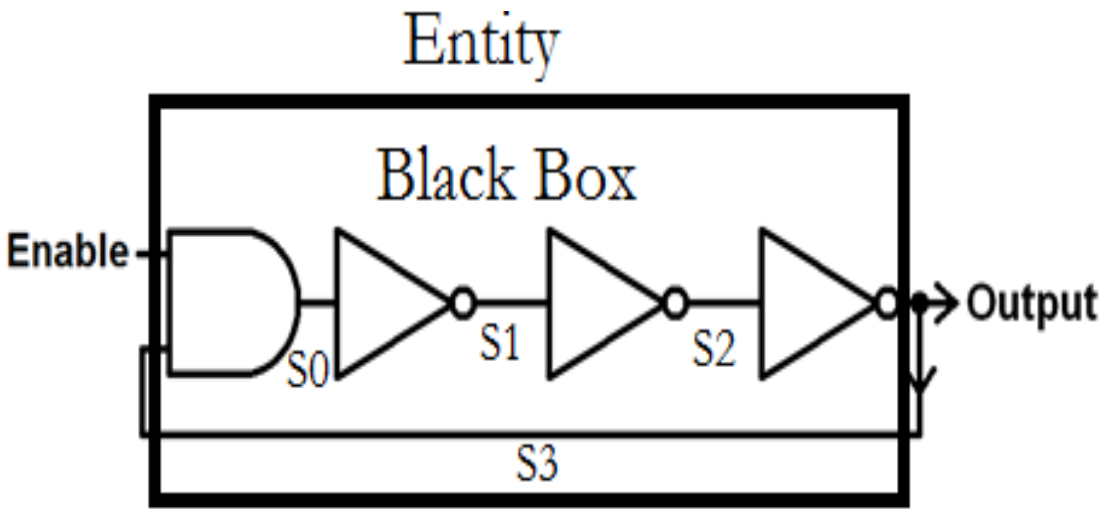
Ring Oscillator Module

- Ring oscillators are widely used in ICs to generate clocks or characterize performance
- Each ring oscillator has a unique frequency even if many oscillators are fabricated from the same mask

VHDL Ring Oscillators (RO) Implementation

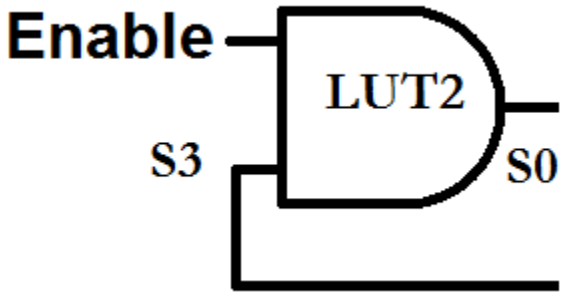
```
Entity Three_stages_RO is
port (Enable: in std_logic;  Output: out std_logic);
Three_stages_RO;
Architecture Behavioral of Three_stages_RO is
Signal s0, s1, s2, s3: std_logic;
begin
 RO-Body
END
```

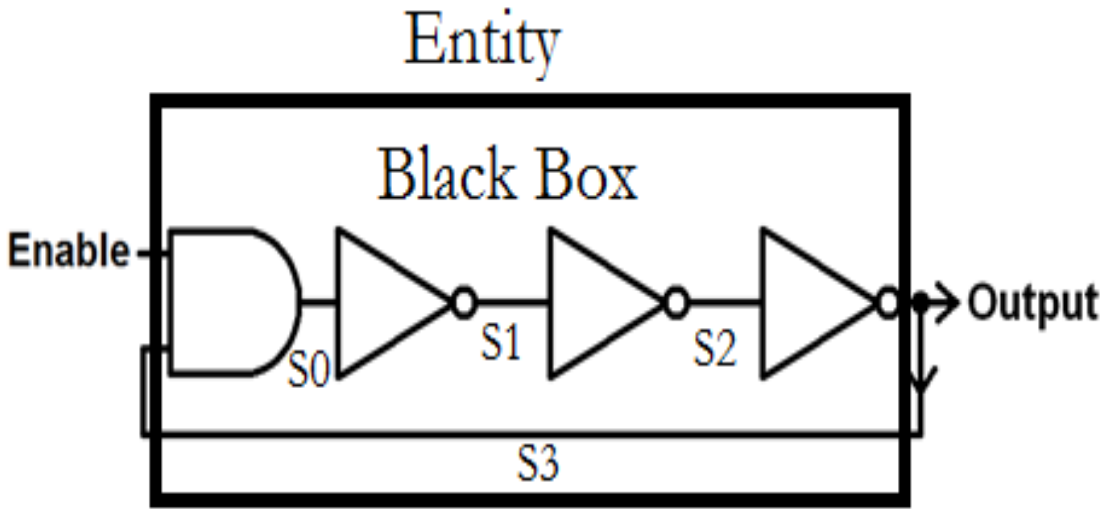




VHDL RO Implementation

```
Entity Three_stages_RO is
port (Enable: in std_logic;  Output: out std_logic);
Three_stages_RO;
Architecture Behavioral of Three_stages_RO is
signal s0, s1, s2, s3: std_logic;
begin
LUT1_inst : LUT2
 generic map (INIT => X"7")
 port map ( O => S0, (LUT output)
I0 => Enable, (LUT input1)
I1 => s3 (LUT input2);
End
```



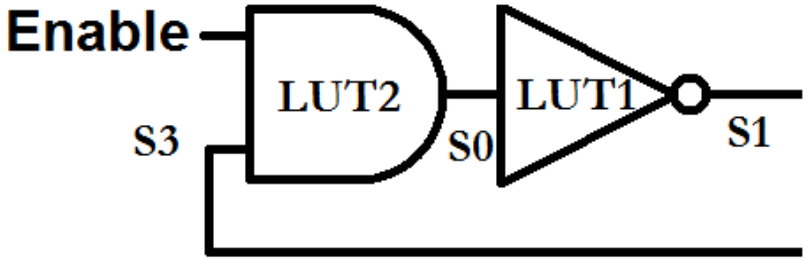


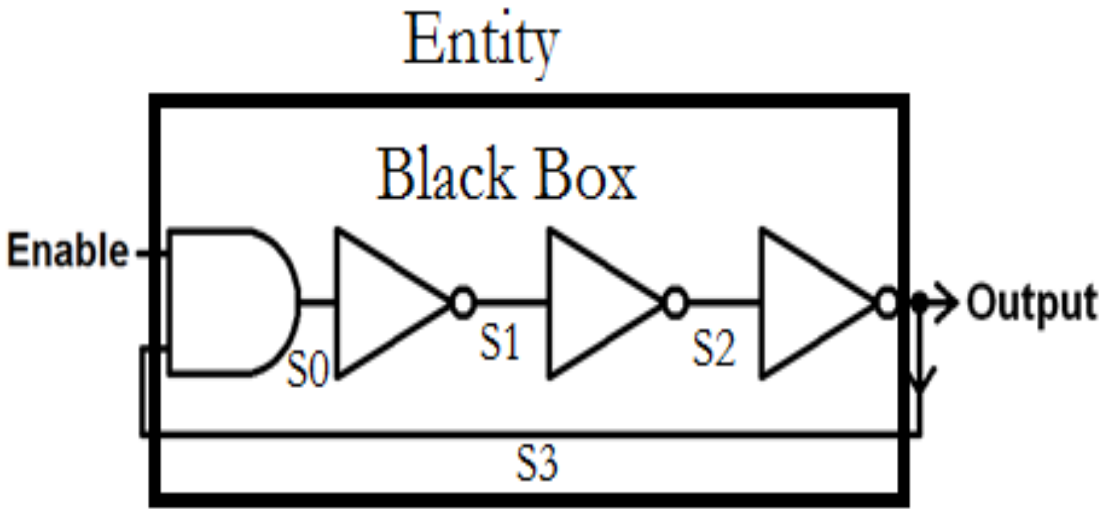
VHDL RO Implementation

```
Entity Three_stages_RO is
port (Enable: in std_logic;  Output: out std_logic);
Three_stages_RO;
Architecture Behavioral of Three_stages_RO is
signal s0, s1, s2, s3: std_logic;
begin
LUT1_inst : LUT2
 generic map (INIT => X"7")
 port map ( O => S0, -- LUT general output
I0 => Enable, -- LUT input
I1 => s3 -- LUT input);
```

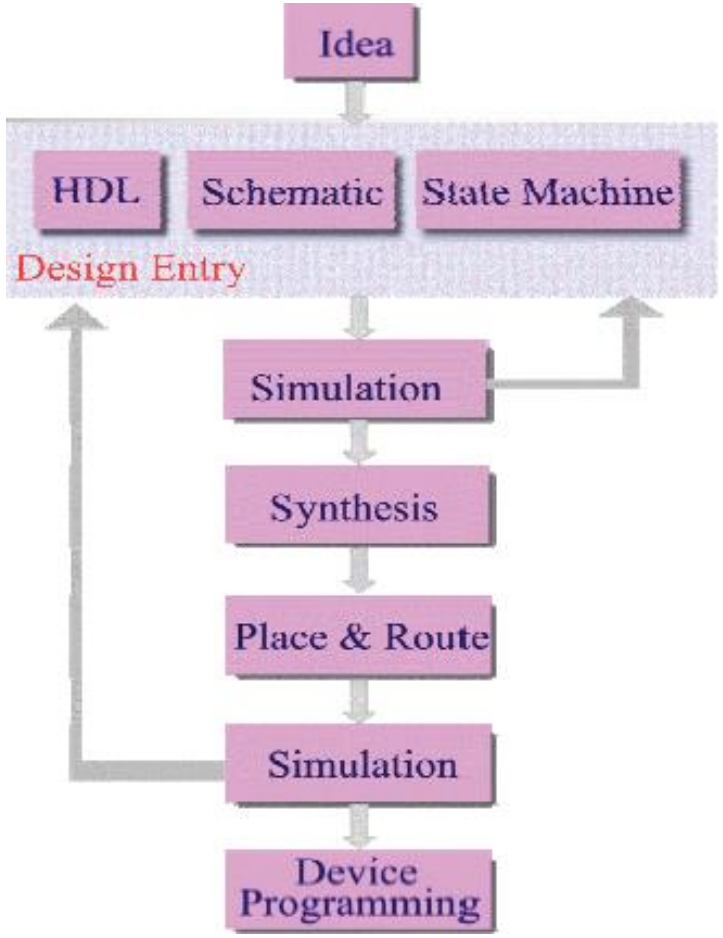
Can you Build LUT1? This is Part of your homework today

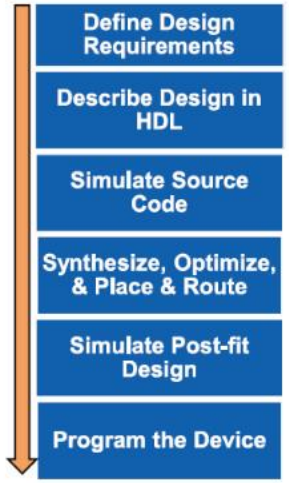
End

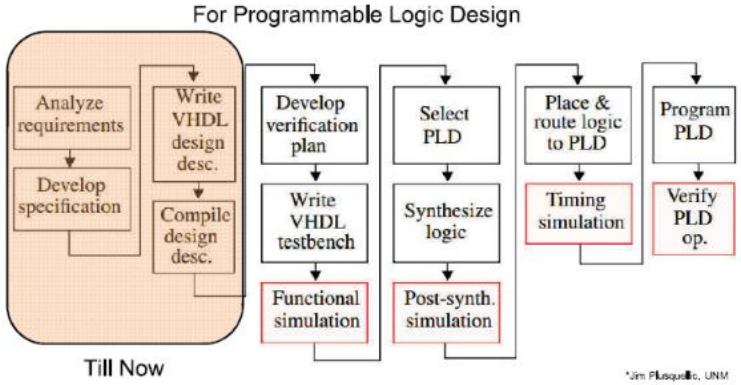




Steps to Implement Digital Logic on FPGA







Hardware Description Language

- Hardware Description Language - What and Why ?
- HDL Starter- General idea, Tools, Simulation and Synthesis ideas, etc.
- Sample Examples
- Combinational Circuits
- Sequential circuits
- Finite State Machines

Hardware Description Language - What and Why

- Specialized computer language for programing the structure, design and operation of electronic circuits.
  - Mostly digital circuits
  - Analog & Mixed Signal HDLs are also available.
- HDL offers hardware realization based on "Black-box Behavior".
- Can express designs in structural, behavioral or register-transfer-level architectures for the same circuit functionality
- Better control on simulation, synthesis, and test.
- Fast and cost effective!
- Allows designs to be ported from one EDA environment to another
- Supports complex designs with hierarchy - from gate to system level
- Allows us to validate the design prior to fabrication

Which one to choose - VHDL or Verilog? D

1. VHDL (VHSIC HDL): Very High Speed Integrated Circuit HDL
   - Developed by the Department of Defense (DOD) in 1981
   - IEEE got rights and publish as standards in 1986/87
   - Revised (current popular) standard was published in 1993 by VHDL international (VI)

2. Verilog HDL:
   - Developed by Gateway Design Automation in 1980
   - Acquired by Cadence and made public 1989/90
   - Become IEEE standard in 1995, regulated by Open Verilog International (OVI)

What to Choose - VHDL or Verilog?

- VHDL and Verilog are **two different hardware description languages**, not simply different toolsets or design methodologies.
- Both HDLs provide control from **low-level hardware details to higher levels of abstraction**.
- Both require **simulation and synthesis** to verify correct operation and generate hardware implementations.
- **Which One Should You Choose?**
  - **Personal preference:** Choose the language you are more comfortable and productive with.
  - **EDA tool availability:** Consider which language is better supported by your available design and simulation tools.
  - **Commercial and business considerations:** Industry adoption, project requirements, vendor support, and organizational preferences may influence the choice
  - The choice of HDL can influence the **toolset, design methodology, performance, and optimization** available to the designer.
  - .

What to Choose - VHDL or Verilog? - Example

- ❑ Bothe are HDL and they can function the same to program an FPGA
- ❑ Differences in: Libraries, Some Capabilities, e.g., Design reusability, Managing large designs, High level design constructs), Compilation, and so on

- **Example VHDL**
  - Provides a representation **closer to the middle-to-low level of hardware abstraction**.
  - **Preferred when the design needs to be partitioned into small blocks and the blocks need to be connected together** (**structural design**).
  - **Easy to use for both high-level behavioral modeling and structural design**.
  - Provides **greater data-type flexibility** and supports several **advanced language features**.
- **While Verilog**
  - Its syntax and programming style are **closer to high-level programming languages**, such as **C**.
  - Verilog has one primary building block: the **module**.
  - It is **easier to use below the RTL level**, with built-in support for **gate-level and transistor-level descriptions**.
    - Generally, Verilog is well suited for **designs below the RTL level**, particularly when describing gates and lower-level hardware structures.

VHDL vs Verilog Code – 8 bits Counter Example

VHDL

```
1.  LIBRARY IEEE;
2.  USE IEEE.STD_LOGIC_1164.ALL;
3.  USE IEEE.STD_LOGIC_ARITH.ALL;
4.  USE IEEE.STD_LOGIC_UNSIGNED.ALL;

5.  ENTITY Counter1 IS
6.    PORT(
7.              Clock, Reset,UPDOWN     : IN STD_LOGIC;
8.               Max_count     : IN STD_LOGIC_VECTOR(7 downto 0);
9.              Count          : OUT     STD_LOGIC_VECTOR(7 downto 0)
10.             );
11. END Counter1;

12. ARCHITECTURE behaviour OF Counter1 IS
13.   SIGNAL internal_count    : STD_LOGIC_VECTOR(7 downto 0);
14. BEGIN
15.   Count <= internal_count;
16.   PROCESS(Reset,Clock)
17.             BEGIN
18.                 IF reset='0' THEN
19.                     internal_count<="00000000";
20.                 ELSIF clock 'EVENT AND clock='0' THEN
21.                     IF updown='0' THEN
22.                           IF internal_count<Max_count THEN
23.                                   internal_count<=internal_count+1;
24.                           ELSE
25.                                   internal_count<="00000000";
26.                           END IF;
27.                     ELSIF updown='1' THEN
28.                          IF "00000000"<internal_count THEN
29.                                   internal_count<=internal_count-1;
30.                          ELSE
31.                                   internal_count<=Max_count;
32.                          END IF;
33.                     END IF;
34.                 END IF;
35.   END PROCESS;
36. END behaviour;
```

Verilog

```
1.  module counter2 (updown,clock,reset,MaxCount,Count);
2.  output[7:0] Count;
3.  input[7:0] MaxCount;
4.  input clock, reset, updown;
5.  reg[7:0] Cnt;

6.  assign Count=Cnt;

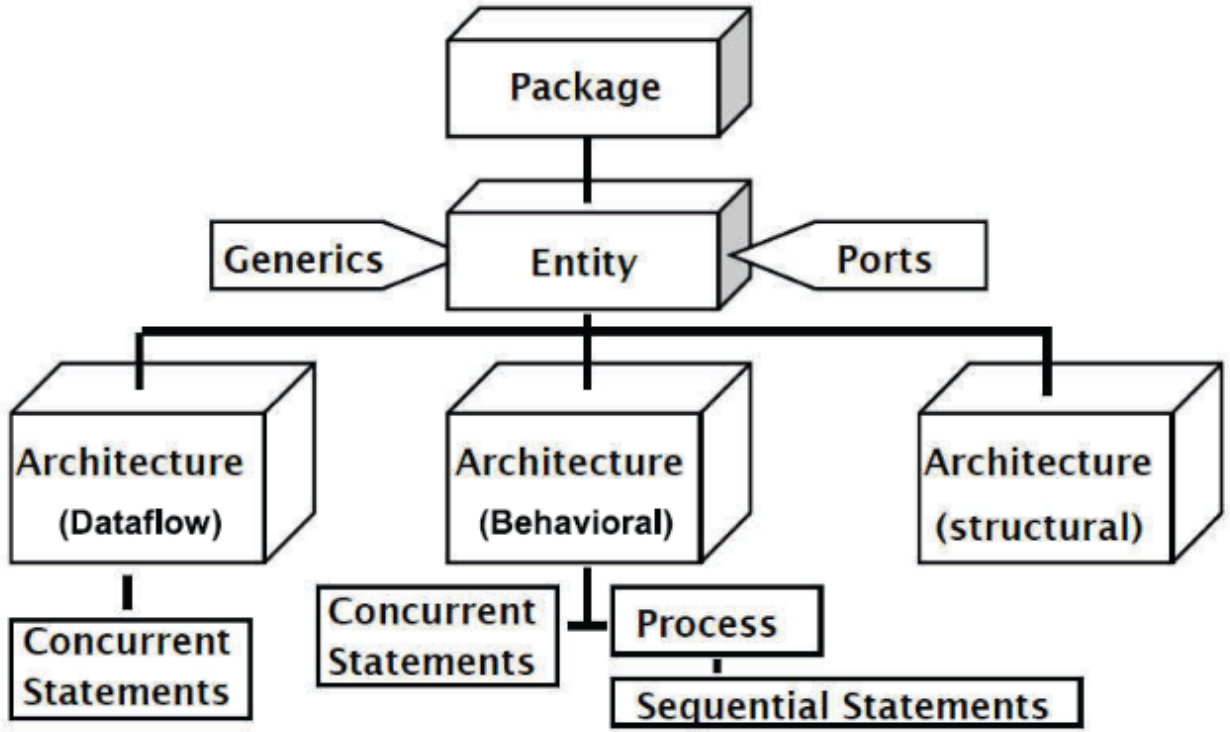
7.  always @ (negedge clock or negedge reset)
8.  begin
9.  if(~reset)
10.   Cnt=8'b0000_0000;
11. else if(updown)
12.   if (Cnt<MaxCount)
13.         Cnt=Count+1;
14.   else
15.         Cnt=8'b0000_0000;
16. else if(~updown)
17.   if (8'b0000_0000<Cnt)
18.         Cnt=Cnt-1;
19.   else
20.         Cnt=MaxCount;
21. end
22. endmodule
```

Resources

- **FPGA Design Tools**
  - **AMD Xilinx Vivado Design Suite**
    - The primary design tool for **AMD/Xilinx FPGA devices**.
    - Supports a wide range of **current and previous FPGA device families**.
    - Supports **VHDL, Verilog, and SystemVerilog**.
    - Provides tools for **simulation, synthesis, implementation, timing analysis, and FPGA programming**.
    - To download Vivado, select **"Download Vivado"** and sign in or create a free AMD account.
    - **Download:** https://www.amd.com/en/products/software/adaptive-socs-and-fpgas/vivado.html
- **EDA Playground**
  - **Online and versatile** HDL simulation platform.
  - Supports **VHDL, Verilog, and SystemVerilog**.
  - No installation required.
  - **Link:** EDA Playground: http://www.edaplayground.com/home

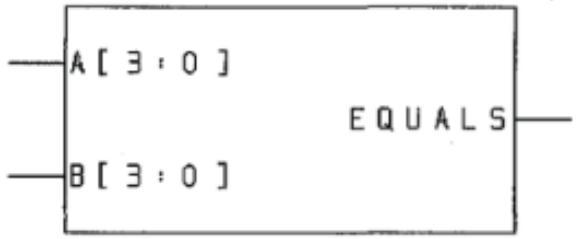
- **Books & Online Resources**
- **VHDL: Programming by Example** — Douglas Perry (ISBN: 0071400702)
- **VHDL for Programmable Logic** — Kevin Skahill (ISBN: 0201895730)
- **VHDL Starter's Guide** — Sudhakar Yalamanchili (ISBN: 0131457357)
- **Online:** YouTube, tutorials, lectures, and example codes

VHDL Hierarchy



VHDL – Entity and Port Declaration

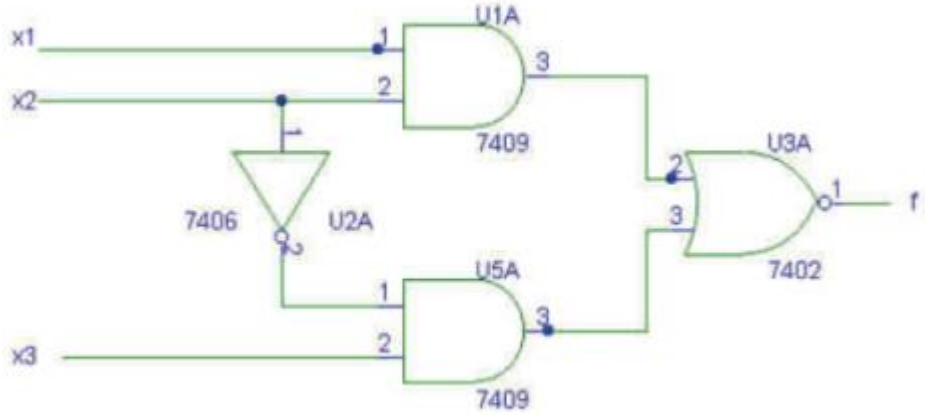
- Entity declaration describes I/O and parameterized values of design entity
  - Analogous to Black-box/Schematic Symbol
  - Describes component's connection to the rest of the design



- Ports: A Data Object – refers to I/O signal in an entity
  - Analogous to Pin
  - Consists of 'Name+Mode+Data Type'

- Mode: Describes the direction of data transferring through a port.
  - IN: Only incoming direction (e.g. clock input, control inputs, etc.)
  - OUT: Only outgoing direction from source to output port; NOT readable within entity i.e. cannot be used as feedback
  - BUFFER: Out + internal feedback
  - INOUT: Bidirectional signals

VHDL – Simple Logic Example




```vhdl
-- logic_ckt is an example
--

library ieee;
use ieee.std_logic_1164.all;

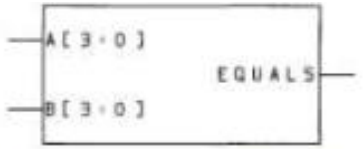
ENTITY logic_ckt IS
PORT (
    x1, x2, x3: IN std_logic;
    f:          OUT std_logic);
END logic_ckt;

ARCHITECTURE arch_logic_ckt OF logic_ckt IS
BEGIN
    f <= (x1 AND x2) NOR (NOT x2 AND x3);
END arch_logic_ckt;
```

* M. Tehranipoor

VHDL – 4-bit comparator example (combinational logic)

- Architecture describes an Entity's function
  - Contents of Entity; components within the black-box
  - Logic/Behavior of the Black-box




```vhdl
-- eqcomp4 is a four bit equality comparator with ieee_std_logic

library ieee;
use ieee.std_logic_1164.all;

entity eqcomp4 is port(
    a, b:   in std_logic_vector(3 downto 0);
    equals: out std_logic );
end eqcomp4;

architecture dataflow of eqcomp4 is
begin
    equals <= '1' when (a = b) else '0'; -- equals is active high
end dataflow;
```

- Can be written in various styles
  - Dataflow
  - Behavioral
  - Structural
- Possible mix-and-match in styles possible
- Allows control on design over different level of abstraction

- Note: Name given to architecture does not effect the outcome

  Style of design description does not necessarily effect synthesis

VHDL – 4-bit comparator example (Behavioral Example)

- Behavioral Architecture
  - High level descriptions, resembles to high level programing language.
  - Uses "Process" Statement.
  - Allows sequential statements; useful for nested operation.

- Process: Design constructs for embodying algorithms
  - Can be considered as event(s)
  - Allows multiple 'process' (events) to encounter
  - Syntax: "Label(optional)+Process+Sensitivity List"
  - Affecting parameters should be included into the Sensitivity List

```
1  -- eqcomp4 is a four bit equality comparator
2  -- with behavioral architecture
3
4  library ieee;
5  use ieee.std_logic_1164.all;
6
7  entity eqcomp4 is port(
8      a, b:   in std_logic_vector(3 downto 0);
9      equals: out std_logic);
10 end eqcomp4;
11
12 architecture behavioral of eqcomp4 is
13 begin
14 comp: process (a, b)
15     begin
16         if a = b then
17             equals <= '1';
18         else
19             equals <= '0';
20         end if;
21     end process comp;
22 end behavioral;
```

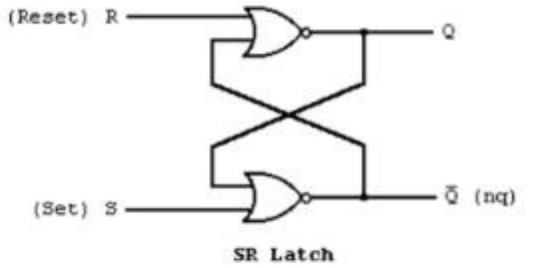
VHDL – (Dataflow Example)

- Execution order of concurrent signal assignment is dependent upon which signals change.
- A concurrent signal assignment statement is sensitive to transitions in any of the signals on the right-side of the <= signal assignment operator
- The textual order of the concurrent signal assignment statements (CSAs) do NOT effect the results.

```
1  -- initially a, b, c, d equals '1'
2  -- hence w, x, y, z equals '1'
3  -- what if b changes to '0'?
4
5
6  architecture simple of example is
7  begin
8      w <= x and y;
9      x <= y or c;
10     y <= a and b;
11     z <= x and d;
12 end simple;
13
14 -- line 10 will execute first
15 -- what next?
16
```

Sample Combinational Example

```
1  -- start  : r='0',s='0',q='1',nq='0'
2  -- what if r changes to '1'?
3
4  architecture sample of sr_latch is
5  begin
6      q  <= r nor nq;
7      nq <= s nor q;
8  end sample;
9
10 -- line 6 will execute first
11
12 -- start  : r='0',s='0',q='1',nq='0'
13 -- round 1: r='1',s='0',q='1',nq='0', The value '0' is scheduled on q.
14 -- round 2: r='1',s='0',q='0',nq='0', The value '1' is scheduled on nq.
15 -- round 3: r='1',s='0',q='0',nq='1', No new events are scheduled.
16 -- round 4: r='0',s='0',q='0',nq='1', No new events are scheduled.
```



SR Latch

SR Latch Example

*http://www.gmvhdl.com/simulate.htm

VHDL – (Dataflow Example)

- Dataflow Architecture with Boolean expression

```
1  -- eqcomp4 is a four bit equality comparator
2  -- with Dataflow (Boolean) architecture
3
4  library ieee;
5  use ieee.std_logic_1164.all;
6
7  entity eqcomp4 is port(
8      a, b:   in std_logic_vector(3 downto 0);
9      equals: out std_logic);
10 end eqcomp4;
11
12 architecture bool of eqcomp4 is
13 begin
14 equals <=       NOT(a(0) XOR b(0) )
15             AND NOT(a(1) XOR b(1) )
16             AND NOT(a(2) XOR b(2) )
17             AND NOT(a(3) XOR b(3) );
18
19 end bool;
```

VHDL – 4-bit comparator example (combinational logic)

- Commonly used Data Type for Ports are from IEEE Standard Logic Family
  - Requires IEEE.STD_LOGIC_1164.ALL package
  - Most useful and well supported for synthesis

```
1  -- eqcomp4 is a four bit equality comparator with ieee_std_logic
2
3  library ieee;
4  use ieee.std_logic_1164.all;
5
6  entity eqcomp4 is port(
7      a, b:   in std_logic_vector(3 downto 0);
8      equals: out std_logic );
9  end eqcomp4;
10
11 architecture dataflow of eqcomp4 is
12 begin
13     equals <= '1' when (a = b) else '0'; -- equals is active high
14 end dataflow;
```

VHDL – 4-bit comparator example (Data Flow logic)

- Dataflow Architecture with Boolean expression

```
1  -- eqcomp4 is a four bit equality comparator
2  -- with Dataflow (Boolean) architecture
3
4  library ieee;
5  use ieee.std_logic_1164.all;
6
7  entity eqcomp4 is port(
8      a, b:   in std_logic_vector(3 downto 0);
9      equals: out std_logic);
10 end eqcomp4;
11
12 architecture bool of eqcomp4 is
13 begin
14 equals <=       NOT(a(0) XOR b(0) )
15             AND NOT(a(1) XOR b(1) )
16             AND NOT(a(2) XOR b(2) )
17             AND NOT(a(3) XOR b(3) );
18
19 end bool;
```

VHDL – 4-bit comparator example (Structural)

- Structural description consists of VHDL netlist
  - Hierarchical
  - Consists of 'components' (entity used inside another entity)
  - All entity must be declared (using 'gatespkg' package includes the standard gates (entities))

```
1  -- eqcomp4 is a four bit equality comparator
2  -- with structural architecture
3
4  library ieee;
5  use ieee.std_logic_1164.all;
6
7  entity eqcomp4 is port(
8      a, b:   in std_logic_vector(3 downto 0);
9      equals: out std_logic);
10 end eqcomp4;
11
12 use work.gatespkg.all;
13
14 architecture structural of eqcomp4 is
15 signal x : std_logic_vector(0 to 3);
16 begin
17   u0: xnor2 port map (a(0) ,b(0) ,x(0));
18   u1: xnor2 port map (a(1) ,b(1) ,x(1));
19   u2: xnor2 port map (a(2) ,b(2) ,x(2));
20   u3: xnor2 port map (a(3) ,b(3) ,x(3));
21   u4: and4  port map (x(0) ,x(1) ,x(2) ,x(3) ,equals);
22 end stuctural;
```

VHDL Example 4 x 1 Mux

```
1  -- mux4to1 is a four to one MUX
2  --
3
4  library ieee;
5  use ieee.std_logic_1164.all;
6
7  entity mux4to1 is
8  port(
9      a, b, c, d:     in std_logic_vector(3 downto 0);
10     s:  in std_logic_vector(1 downto 0);
11     x:  out std_logic_vector(3 downto 0));
12 end mux4to1;
13
14 architecture arcmux4to1 of mux4to1 is
15 begin
16     with s select
17         x <= a when "00",
18         x <= b when "01",
19         x <= c when "10",
20         x <= d when others;
21 end arcmux4to1;
```

```
1  -- mux4to1 is a four to one MUX
2  --
3
4  library ieee;
5  use ieee.std_logic_1164.all;
6
7  entity mux4to1 is
8  port(
9      a, b, c, d:     in std_logic_vector(3 downto 0);
10     s:  in std_logic_vector(1 downto 0);
11     x:  out std_logic_vector(3 downto 0));
12 end mux4to1;
13
14 architecture arcmux4to1 of mux4to1 is
15 begin
16         x <= a when (s = "00") else
17              b when (s = "01") else
18              c when (s = "10") else
19              d;
20 end arcmux4to1;
```

```
-- what if we replace line 20 with
-- x <= "11" when others;
-- x <= d when others;
```

VHDL Priority Encoder Example

```
1  -- prior_encode is a 4-bit Priority Encoder
2
3  LIBRARY IEEE;
4  USE IEEE.STD_LOGIC_1164.ALL;
5
6  ENTITY prior_encode IS PORT (
7      d   :   IN  STD_LOGIC VECTOR (3 DOWNTO 0);
8      f   :   OUT STD_LOGIC VECTOR (1 DOWNTO 0);
9      z   :   OUT STD_LOGIC);
10 END prior_encode;
11
12 ARCHITECTURE sample OF prior_encode IS
13 BEGIN
14     f <= "11" WHEN d(3) = '1' ELSE
15          "10" WHEN d(2) = '1' ELSE
16          "01" WHEN d(1) = '1' ELSE
17          "00";
18     z <= '0' WHEN d = "0000" ELSE '1';
19
20 END sample;
21
```

VHDL 4 x 1 Adder

```
1  -- adder4 is a four bit adder
2
3  LIBRARY ieee;
4  USE  ieee.std_logic_1164.all;
5  USE  ieee.std_logic_arith.all;
6  USE  ieee.std_logic_signed.all;
7
8  ENTITY adder4 IS    PORT (
9      Cin  : IN  STD_LOGIC;
10     a, b : IN  STD_LOGIC_VECTOR(3 DOWNTO 0);
11     s    : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
12     Cout : OUT STD_LOGIC);
13 END adder;
14
15 ARCHITECTURE example OF adder4 IS
16 SIGNAL Sum : STD_LOGIC_VECTOR(4 DOWNTO 0);
17 BEGIN
18     Sum <= a + b + Cin;
19     s <= Sum(3 DOWNTO 0);
20     Cout <= Sum(4);
21 END example;
22
```

16- bit Adder VHDL code

```
1  -- adder16 is a 16 bit adder that uses adder4 as component
2  -- only architecture level is shown here
3
4  architecture behavioral of adder16 is
5  signal c : bit_vector (15 downto 0);
6  begin
7      array : for i in 0 to 15 generate
8          begin
9              first : if i=0 generate
10         begin
11             cell :  component full_adder
12                     port map(a(i), b(i), cin, s(i), c(i));
13         end generate first;
14
15         other : if i/=0 generate
16         begin
17             cell : component full_adder
18                 port_map(a(i), b(i), c(i-1), s(i), c(i));
19         end generate other;
20     end generate array;
21
22     cout <= c(15);
23
24 end architecture behavioral;
25
```

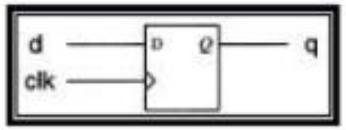
Sequential Circuit D FF vs D latch – VHDL

```vhdl
-- D-FlipFlop with rising clock edge

library ieee;
use ieee.std_logic_1164.all;

entity dff_logic is port (
    d, clk: in std_logic;
    q :     out std_logic) ;
end dff_logic;


architecture arch_dff_logic of dff_logic is
begin
    process (clk) begin
        if (clk'event and clk = '1') then
            q <= d;
        end if;
    end process;
end arch_dff_logic;
```

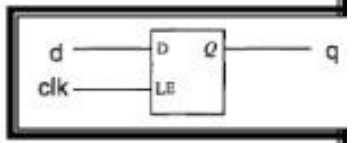



```vhdl
-- Level Sensitive Latch

library ieee;
use ieee.std_logic_1164.all;

entity dff_logic is port (
    d, clk: in std_logic;
    q :     out std_logic) ;
end dff_logic;


architecture arch_dff_logic of dff_logic is
begin
    process (clk, d) begin
        if (clk = '1') then
            q <= d;
        end if;
    end process;
end arch_dff_logic;
```



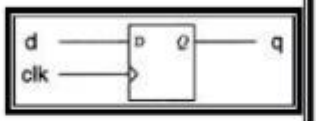
DFF with Rising Edge

```vhdl
-- D-FlipFlop with rising clock edge

library ieee;
use ieee.std_logic_1164.all;

entity dff_logic is port (
    d, clk: in std_logic;
    q :     out std_logic) ;
end dff_logic;


architecture arch_dff_logic of dff_logic is
begin
    process (clk) begin
        if (clk'event and clk = '1') then
            q <= d;
        end if;
    end process;
end arch_dff_logic;
```




```vhdl
-- D-FlipFlop with rising clock edge

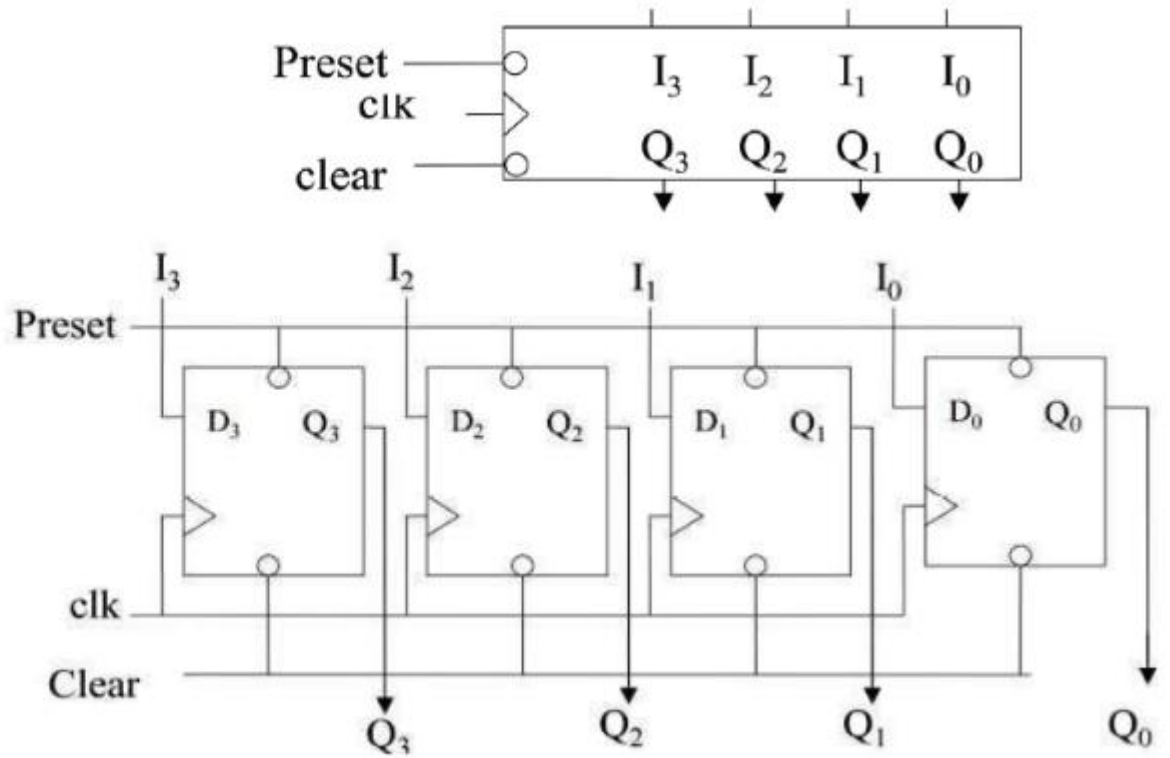
library ieee;
use ieee.std_logic_1164.all;

entity dff_logic is port (
    d, clk: in std_logic;
    q :     out std_logic) ;
end dff_logic;


architecture arch_dff_logic of dff_logic is
begin
    process (clk) begin
        if (rising_edge(clk)) then
            q <= d;
        end if;
    end process;
end arch_dff_logic;
```

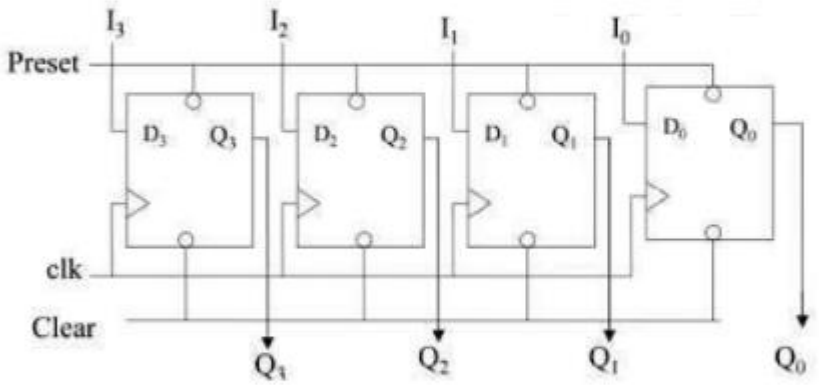
4- bit Register VHDL Code

- A simple register consists of N flip-flops driven by a common clock signal.
- Has N inputs and N outputs in addition to the clock signal.
- Asynchronous Preset and Set does not depend on clock



* M. Tehranipoor

4-bit register with Aschronous present and clear signals




```vhdl
-- Simple 4-Bit Register

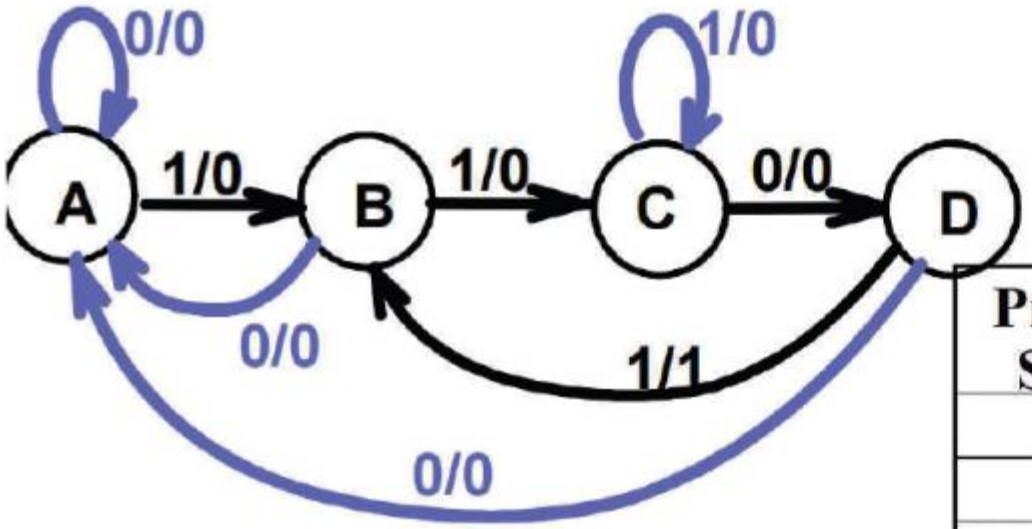
Library ieee;
USE ieee.std_logic_1164.all;

ENTITY simple_register IS
    GENERIC (N : INTEGER := 4);
    PORT (  I : IN STD_LOGIC_VECTOR (N-1 DOWNTO 0);
            Clock, Clear, Preset : IN STD_LOGIC;
            Q : OUT STD_LOGIC_VECTOR (N-1 DOWNTO 0));
END simple_register;

ARCHITECTURE arch_simple_register OF simple_register IS
BEGIN
    PROCESS (Preset, Clear, Clock) BEGIN
        IF Preset = '0' THEN
            Q <= (OTHERS => '1');;
        ELSIF Clear = '0' THEN
            Q <= (OTHERS => '0');
        ELSIF (Clock'EVENT AND Clock = '1') THEN
            Q <= I;
        END IF;
    END PROCESS;
END arch_simple_register;
```

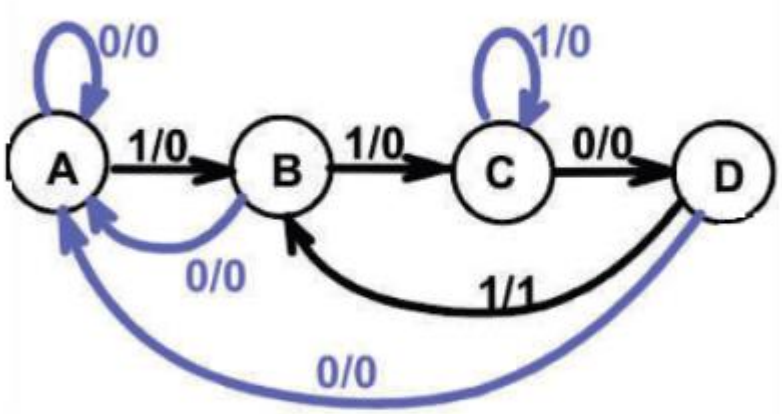
* M. Tehranipoor

Sequential FSM to detect 1101- VHDL code



Present State	Next State x=0	Next State x=1	Output x=0	Output x=1
A	A	B	0	0
B	A	C	0	0
C	D	C	0	0
D	A	B	0	1

Sequential FSM to detect 1101- VHDL code




```vhdl
-- seq_rec is a FSM that detects 1101 sequence in a bit stream

-- This can be implemented in 3 steps
-- Step 1: implement positive edge-triggered flipflop with asynchronous reset
-- Step 2: implement output as function of input X and state
-- Step 3: next state-function implemented as a function of input X and state


-- Define entity
library ieee;
use ieee.std_logic_1164.all;

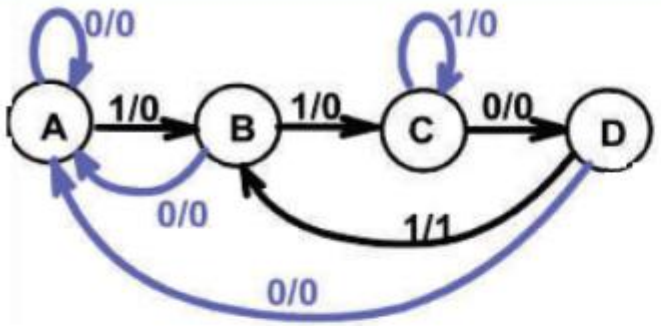
entity seq_rec is port (
    CLK, RESET, X: in std_logic;
    Z: out std_logic);
end seq_rec;

-- Define Architecture
architecture process_3 of seq_rec is
    type state_type is (A, B, C, D);    -- for states
    signal state, next_state: state_type;

begin


--process 1: implements positive edge-triggered flipflop with asynchronous reset
state_register: process (CLK, RESET)
begin
    if(RESET = '1') then
        state <= A;
    elsif (CLK'event and CLK = '1') then
        state <= next_state;
    end if;
end process;
```

* M. Tehranipoor




```
35
36 --process 2: implement output as function of input X and state
37 output_function: process (X, state)
38 begin
39     case state is
40     when A => Z <= '0';
41     when B => Z <= '0';
42     when C => Z <= '0';
43     when D =>
44         if X = '1' then
45             Z <= '1';
46         else
47             Z <= '0';
48         end if;
49     end case;
50 end process;
51
```

```
52 --process 3: implement next state-function
53 --as a function of input X and state
54
55 next_state_function: process (X, state)
56 begin
57     case state is
58     when A =>
59         if X = '1' then
60             next_state <= B;
61         else
62             next_state <= A;
63         end if;
64     when B =>
65         if X = '1' then
66             next_state <= C;
67         else
68             next_state <= A;
69         end if;
70     when C =>
71         if X = '1' then
72             next_state <= C;
73         else
74             next_state <= D;
75         end if;
76     when D =>
77         if X = '1' then
78             next_state <= B;
79         else
80             next_state <= A;
81         end if;
82     end case;
83 end process;
84
85 end process_3;
```

* M. Tehranipoor

Thank you!
Please download and install **AMD Xilinx Vivado**, then complete the **homework assignment**.

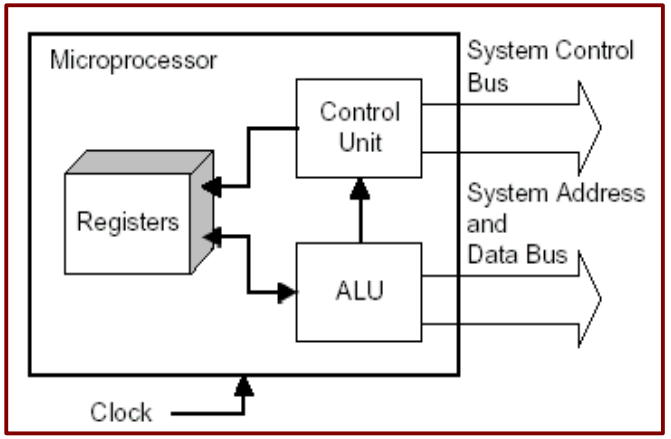
The ARM Cortex-M4 Processor Architecture and Assembly Operation

The slides have been adapted from materials provided by ARM website, **Jonathan W. Valvano (The University of Texas at Austin)**, **NPETL Online Certificate Notes (IIT)**, and **Tennessee Tech University (TTU) ECE 3130 course materials**

The Architecture for the Digital World® ARM

Microprocessor?

- It is basically the entire CPU fabricated on a single chip.
  - Consists of a set of registers to store temporary data.
  - Consists of an arithmetic logic unit (ALU), where all arithmetic and logical computations are carried out.
  - Consists of some mechanism to interface external devices (memory and I/O) through buses (address, data and control).
  - Consists of a control unit that synchronizes the operation.



Schematic Diagram of Microprocessor

ARM

A Microprocessor (CPU) Architecture ?

- A Microprocessor mainly consists of **arithmetic and Logic Unit (ALU), control unit (CU)**, and **registers**.
  - The ALU performsthe arithmetic and logic (ALU) operations requested by the program.
  - A simple ALU that can perform only four operations (addition, subtraction, AND, OR) is shown in Figure 1.1.

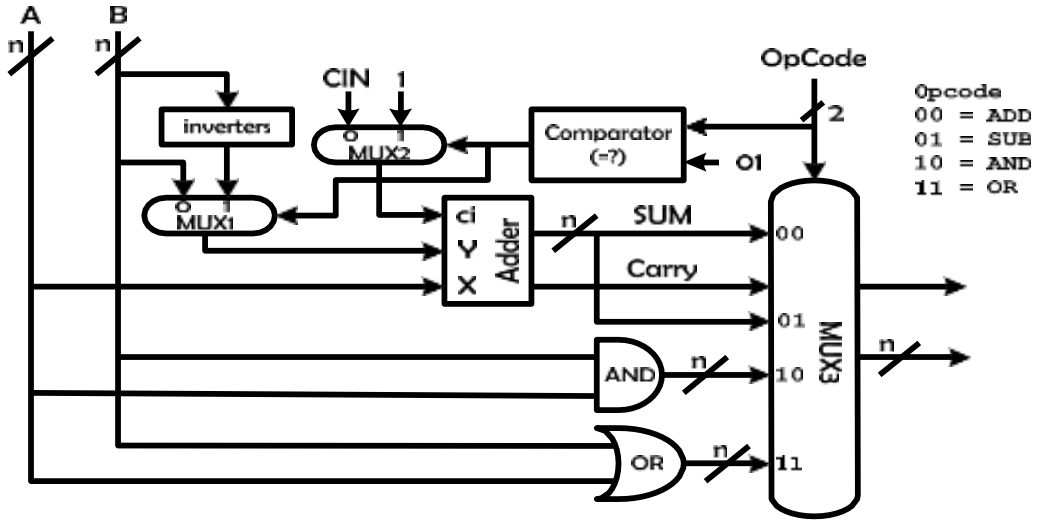
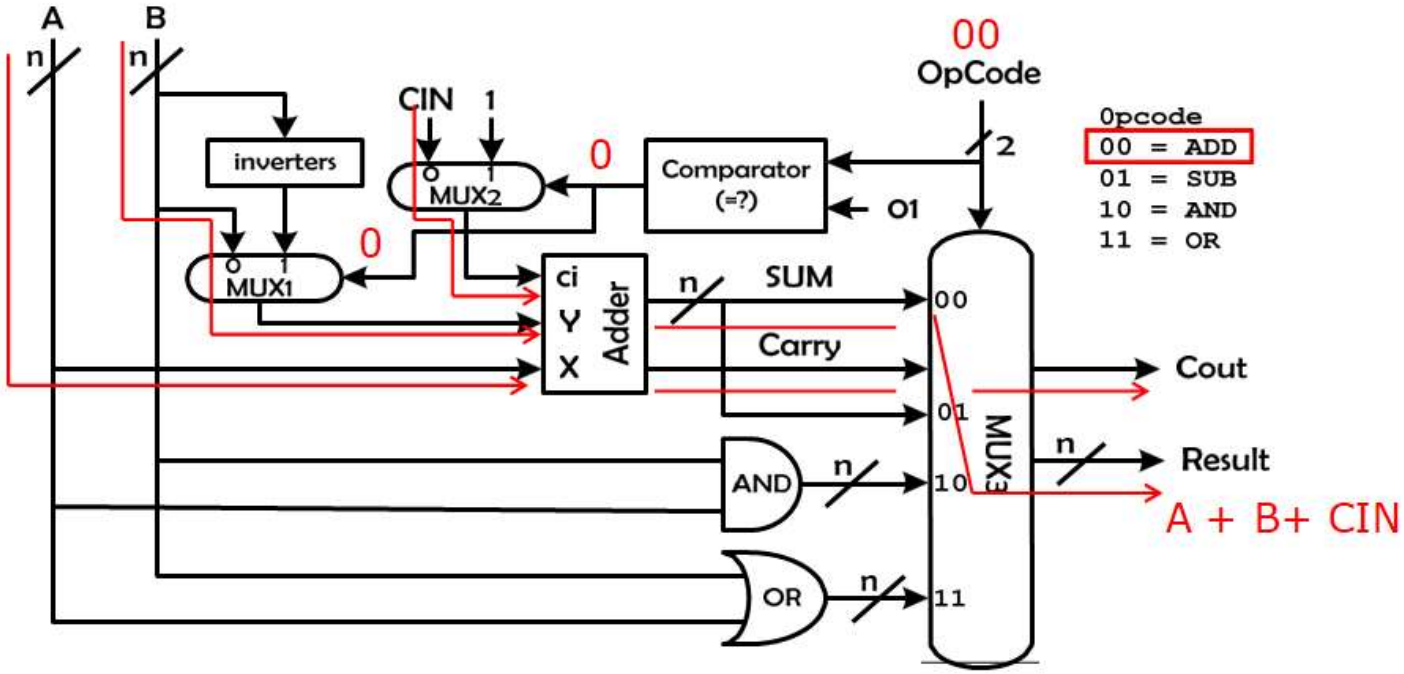


Figure 1.1 An ALU that implements ADD, SUB, AND, and OR operations

ARM

The ALU operations Example: **Opcode = ADD (00)**

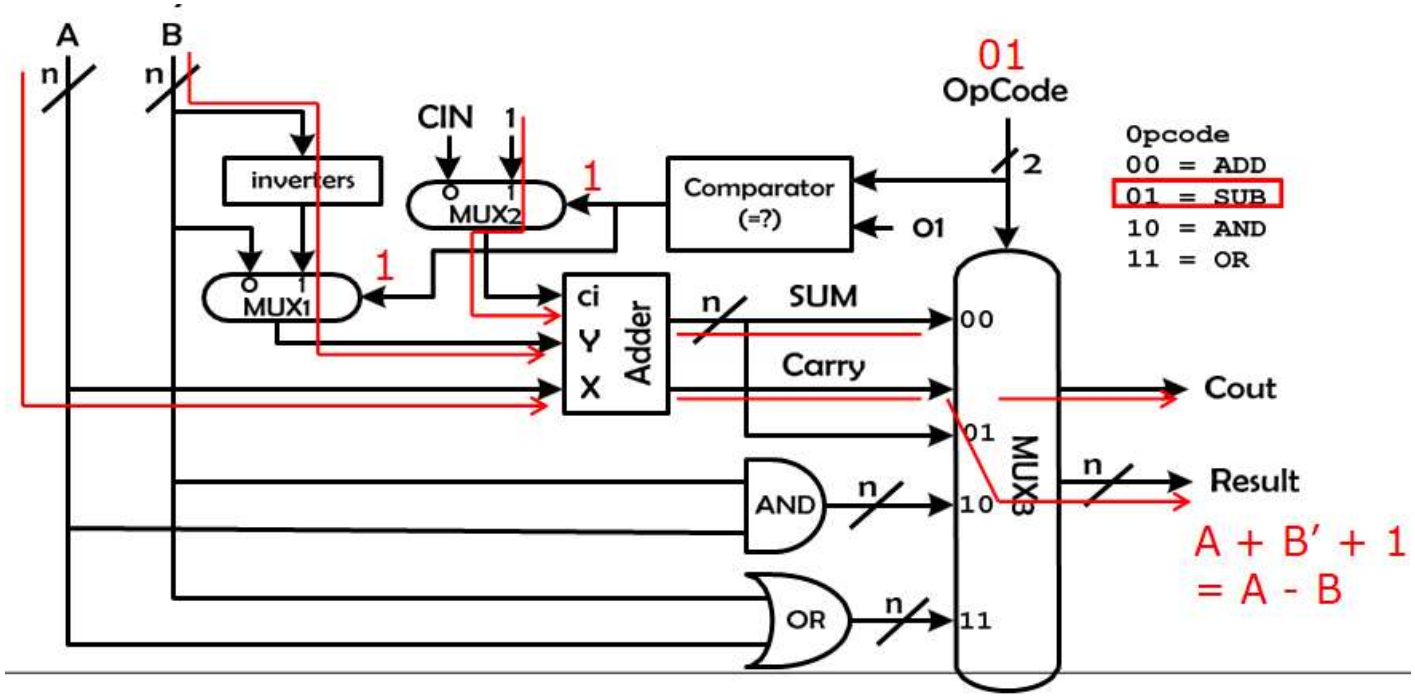
- The comparator output = 0, MUX2 output = CIN, MUX1 output = B, the adder inputs are n-bit A and n-bit B.
- The result of the adder is A + B + CIN, CIN is the input carry.
- MUX3 selects the output of the adder and the carry.



ARM

The ALU operations Example: **Opcode = SUB (00)**

- The comparator output = 1, MUX2 output = 1, MUX1 output = B' (one' complement of B), the adder inputs are n-bit A and n-bit B'.
- The result of the adder is A + B' + 1 = A - B. Remember B' + 1 = - B.
- B' + 1 is the two's complement of B.
- MUX3 selects the output of the adder and the carry (called borrow).



ARM

The binary number system has two digits 0 and 1

- The hexadecimal number system uses 16 digits: 0, 1, .., 9, A, B, C,.., F
- The decimal number system uses 10 digits: 0, 1, .., 9
- Computer hardware uses binary numbers to perform all operations.
- Humans are used to decimal number system.
- Hexadecimal numbers have shorter representations than the binary system.
- Conversion between the different number systems is needed.

ARM

A prefix is used to indicate the base of a number

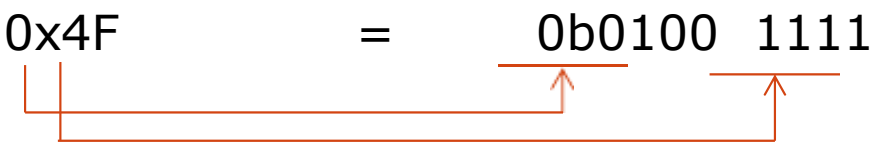
Base	Prefix	Example
Binary	0b	0b10001010
Decimal	None	12345678
Hexadecimal	0x	0x392

Number like 10 can be binary, hexadecimal and decimal. We use a prefix to define it.

- Convert %1000101 to Hexadecimal



- Convert &4F to Binary



- For unsigned numbers, they are always considered positive
- For signed number

B010 0000

- If the most significant bit (B) = 1 → negative, otherwise positive

ARM

Signed and Unsigned Binary numbers

Computer does not know whether a bit sequence represents a signed or an unsigned number -> **It is the programmer's responsibility!**

Unsigned number Example

0b1111 = 1 + 2 + 4 + 8 = 15

0b111 = 1 + 2 + 4 = 7

Unsigned N-bit number can have numbers from 0 to 2^N-1

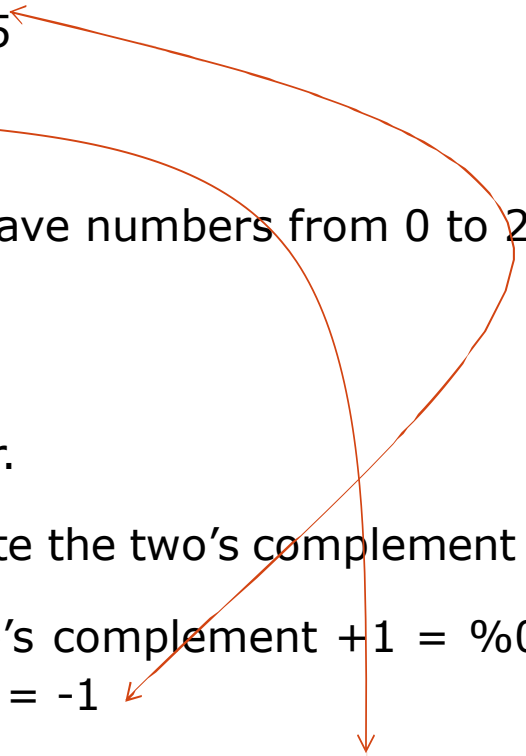
Signed number Example

0b1111 is a negative number.

To convert to decimal, calculate the two's complement

The two's complement = one's complement +1 = %0000 + 1 =0b0001 = 1 → then %1111 = -1

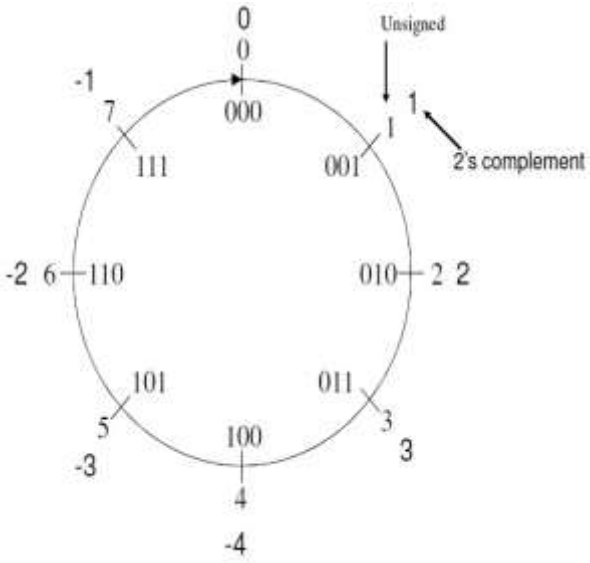
0b0111 is a positive number = 1 + 2 + 4 = 7.



ARM

Signed and Unsigned Binary numbers

- Unsigned 8-bit number can have numbers from 0 to 255
  00000000 to 11111111
- Signed 8-bit number can have numbers from -128 to +127 to 10000000 to 01111111
- Signed N-bit number can have numbers from -2^{N-1} to $2^{N-1}-1$
- Unsigned 3-bit number can represent numbers from 0 (= 000) to 7 (=111)
- Signed 3-bit number can represent numbers from -4 (= %100) to 3 (= 011)



ARM

Classification of CPU Architecture

- About the instruction set architecture (ISA) of the CPU.
    a) Complex Instruction Set Computer (CISC)
        - Typically used in desktops, laptops and servers (courtesy Intel).
    b) Reduced Instruction Set Computer (RISC)
        - Typically used in microcontrollers, that are used to build embedded systems.
- Two different types of memory:
    a) Random Access Memory (RAM)
        - Volatile; used for data memory in microcontrollers.
    b) Read Only Memory (ROM)
        - Non-volatile; used for program memory in microcontrollers.

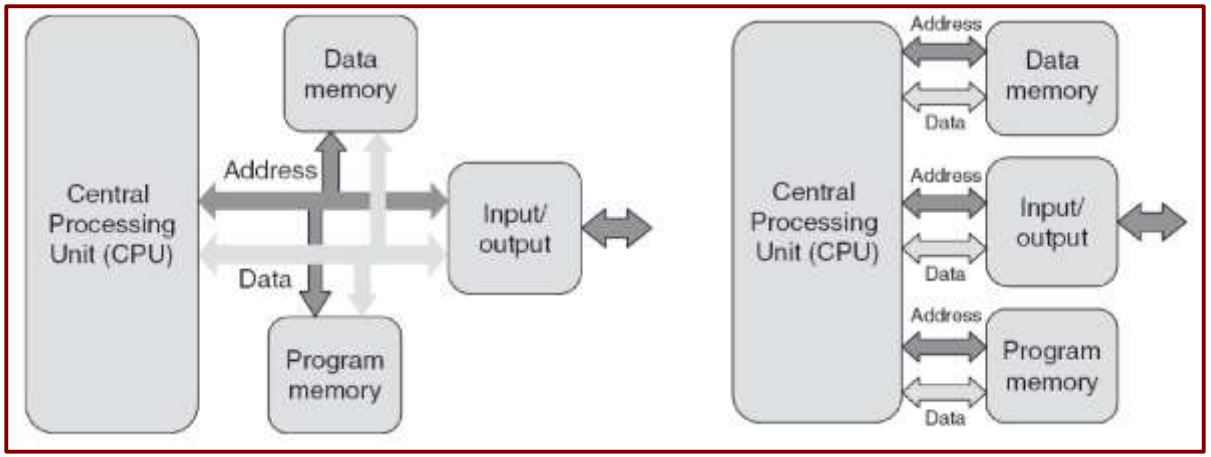
ARM

Classification of CPU Architecture

- Broadly two types of architectures:
  - a) Von Neumann Architecture (CISC)
    - Both instructions and data are stored in the same memory.
    - This model is followed in conventional computing systems.
  - b) Harvard Architecture (RISC based)
    - Instructions and data are stored in separate memories.
    - Typically followed in microcontrollers, used for building embedded systems.
    - Instructions are stored in a ROM (permanent), while stored temporary data are in RAM.

Von Neumann Architecture

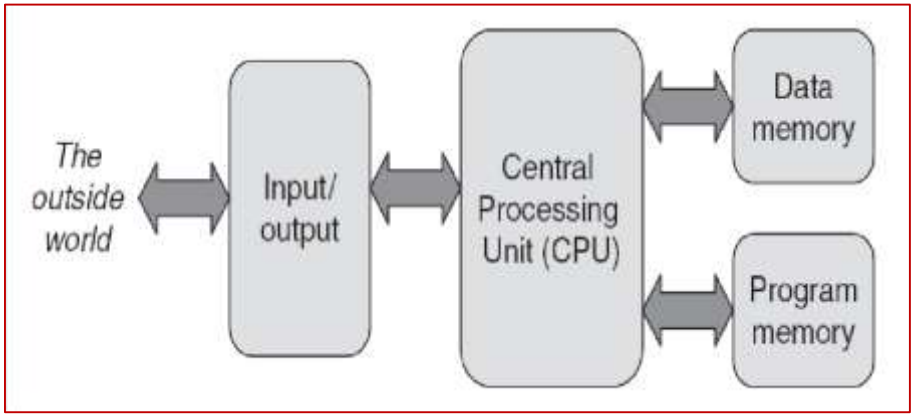
Harvard Architecture



ARM

Basic Operation of a Computing System

- The central processing unit (CPU) carries out all computations.
  - Fetches instructions from the program memory and executes it; may require access to data in data memory.
- The input/output block provides interface with the outside world.
  - Allows users to interact with the computing system, and also observe the output results.



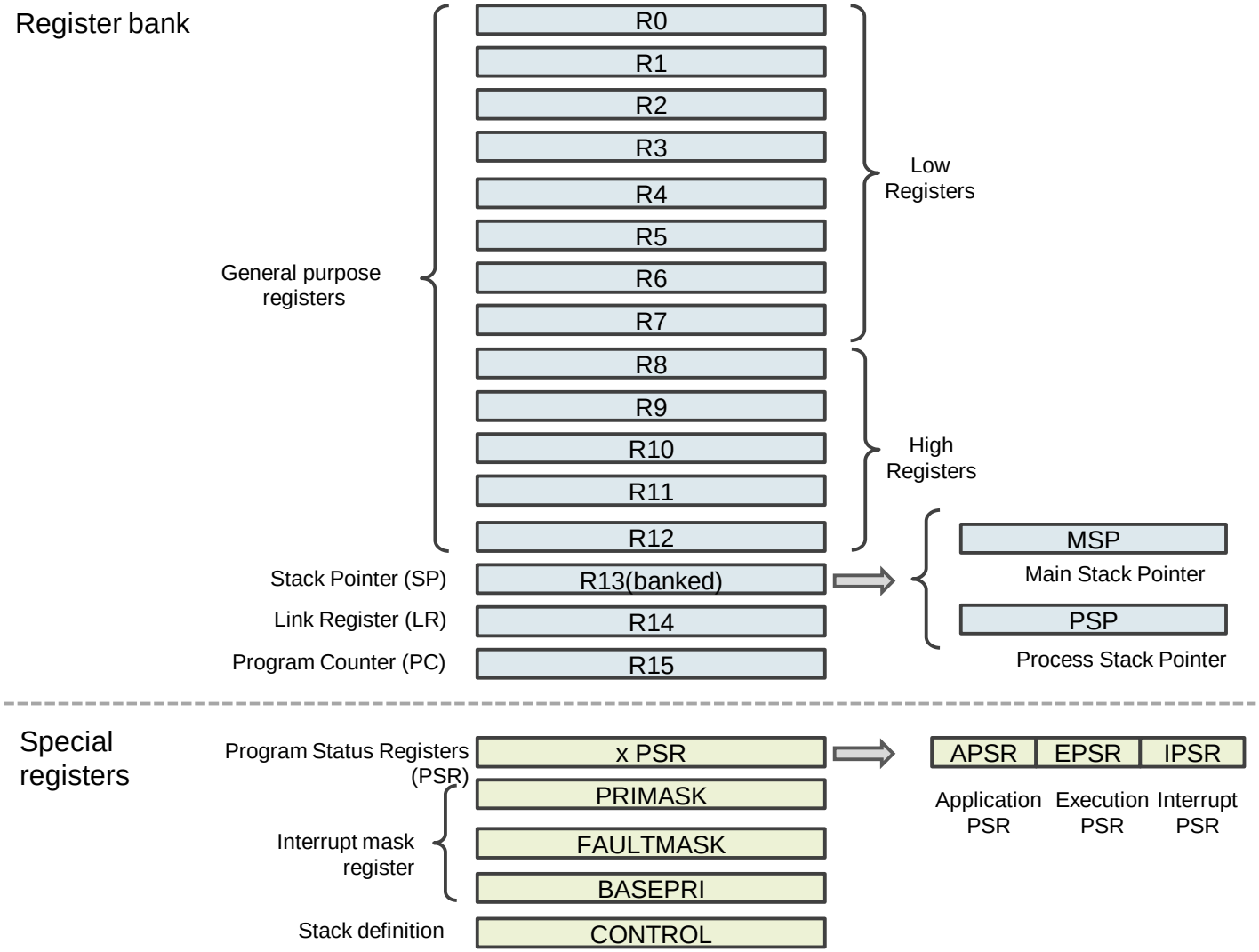
ARM

Cortex-M4 Registers

- Processor registers
  - The internal registers are used to store and process temporary data within the processor core
  - All registers are inside the processor core, hence they can be accessed quickly
  - Load-store architecture
    - To process memory data, they have to be first loaded from memory to registers, processed inside the processor core using register data only, and then written back to memory if needed
- Cortex-M4 registers
  - Register bank
    - Sixteen 32-bit registers (thirteen are used for general-purpose);
  - Special registers

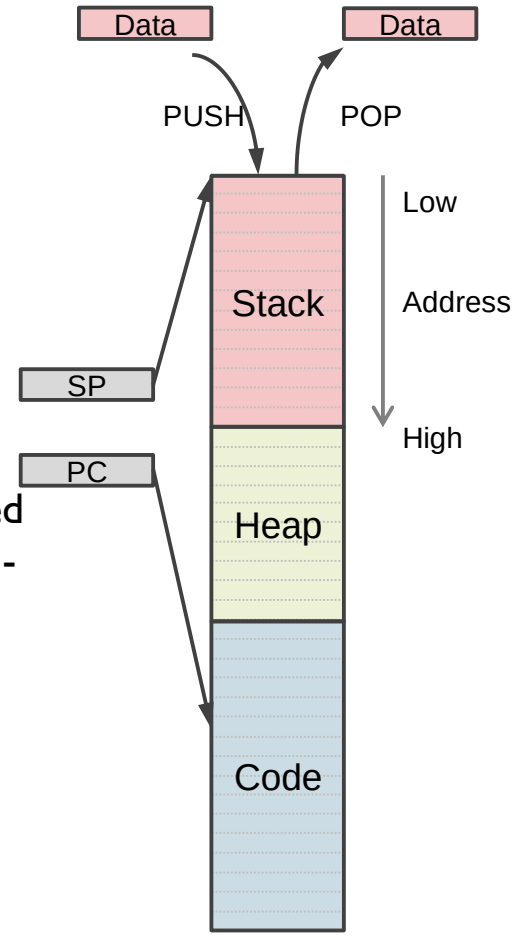
ARM

Cortex-M4 Registers



Cortex-M4 Registers

- R0 – R12: general purpose registers
  - Low registers (R0 – R7) can be accessed by any instruction
  - High registers (R8 – R12) sometimes cannot be accessed e.g. by some Thumb (16-bit) instructions
- R13: Stack Pointer (SP)
  - Records the current address of the stack
  - Used for saving the context of a program while switching between tasks
  - Cortex-M4 has two SPs: Main SP, used in applications that require privileged access e.g. OS kernel, and exception handlers, and Process SP, used in base-level application code (when not running an exception handler)
- Program Counter (PC)
  - Records the address of the current instruction code
  - Automatically incremented by 4 at each operation (for 32-bit instruction code), except branching operations
  - A branching operation, such as function calls, will change the PC to a specific address, meanwhile it saves the current PC to the Link Register (LR)



Cortex-M4 Registers

- APSR
  - N: negative flag – set to one if the result from ALU is negative
  - Z: zero flag – set to one if the result from ALU is zero
  - C: carry flag – set to one if an unsigned overflow occurs
  - V: overflow flag – set to one if a signed overflow occurs
  - Q: sticky saturation flag – set to one if saturation has occurred in saturating arithmetic instructions, or overflow has occurred in certain multiply instructions
- IPSR
  - ISR number – current executing interrupt service routine number
- EPSR
  - T: Thumb state – always one since Cortex-M4 only supports the Thumb state
  - IC/IT: Interrupt-Continuable Instruction (ICI) bit, IF-THEN instruction status bit

ARM

Useful Resources

- Architecture Reference Manual:

  http://infocenter.arm.com/help/index.jsp?topic=/com.arm.doc.ddi0403c/index.html

- Cortex-M4 Technical Reference Manual:

  http://infocenter.arm.com/help/topic/com.arm.doc.ddi0439d/DDI0439D_cortex_m4_processor_r0p1_trm.pdf

- Cortex-M4 Devices Generic User Guide:

  http://infocenter.arm.com/help/topic/com.arm.doc.dui0553a/DUI0553A_cortex_m4_dgug.pdf

ARM

- Categories of ARM instructions
- Data processing instructions

ARM

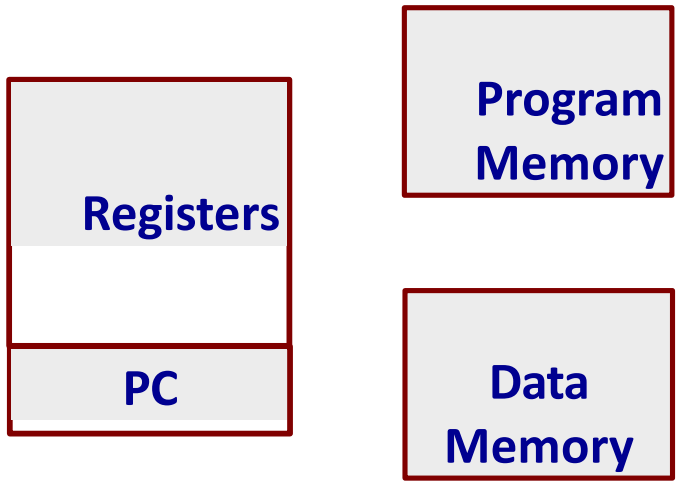
The ARM Instruction Set

- ARM instruction can be categorized into three groups:

a) Data processing instructions
   - Operate on values in registers

b) Data transfer instructions
   - Move values between registers and memory

c) Control flow instructions
   - Change the value of the program counter (PC)



ARM

(a) Data Processing Instructions

- All operands are 32-bits in size:
  - Either registers
  - Or literals (immediate values) specified as part of the instruction
- The result, if any, is also 32-bit in size and goes into a specified register.
  - One exception: long multiply, that generates 64-bit results.
- All operand and result registers are independently specified as part of the instruction.

ARM

- **Arithmetic instructions:**

```
ADD    r0,r1,r      ; r0 = r1 + r2
ADC    r0,r1,r2     ; r0 = r1 + r2 + C
SUB    r0,r1,r      ; r0 = r1 - r2
SBC    r0,r1,r      ; r0 = r1 - r2 + C
RSB    r0,r1,r      ; r0 = r2 - r1
RSC    r0,r1,r      ; r0 = r2 - r1 + C
                                (C is carry bit)
```

- All operations can be viewed as either unsigned or 2's complement signed.
  - Means the same thing.

ARM

- **Bit-wise logical instructions:**

```
AND     r0,r1,r2        ; r0 = r1 and  r2
ORR     r0,r1,r2        ; r0 = r1 or   r2
EOR     r0,r1,r2        ; r0 = r1 xor  r2
BIC     r0,r1,r2        ; r0 = r1 and  not r2
```

- BIC is the acronym for "*bit clear*"
  - Each 1-bit in r2 clears the corresponding bit in r1.

ARM

- **Register-register move instructions:**

```
MOV     r0,r2           ; r0 = r2
MVN     r0,r2           ; r0 = not r2
```

- MVN is the acronym for "*move negated*"
  - Each 1-bit in r2 clears the corresponding bit in r0.
- In the instruction encoding, the first operand r1 is not specified, as these are unary operations.

ARM

- **Comparison instructions:**

```
CMP    r1,r2          ; set cc on (r1 - r2)
CMN    r1,r2          ; set cc on (r1 + r2)
TST    r1,r2          ; set cc on (r1 and r2)
TEQ    r1,r2          ; set cc on (r1 xor r2)
```

- All these instructions affect the condition codes (N, Z, C, V) in the current program status register (CPSR).
  - These instructions do not produce result in any register (r0).

ARM

- **Specifying immediate operands:**

```
ADD     r1,r2,#2        ; r1 = r2 + 2
SUB     r3,r3,#1        ; r3 = r3 - 1
AND     r6,r4,#&0f      ; r6 = r4[3:0]
```

- Notations:
  - # indicates immediate value
  - & indicates hexadecimal notation
- Allowed immediate values:
  - 0 to 255 (8 bits), rotated by any number of bit positions that is multiple of 2.

ARM

- **Shifted register operands:**
  - The second source operand may be shifted either by a constant number of bit positions, or
    by a register-specified number of bit positions.

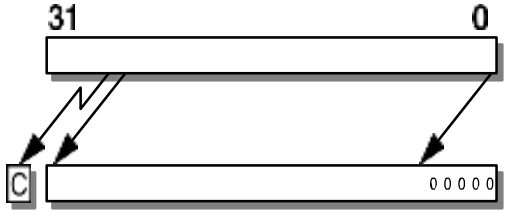
```
ADD     r1,r2,r3,LSL #3     ; r1 = r2 + (r3 << 3)
ADD     r1,r2,r3,LSL r5     ; r1 = r2 + (r3 << r5)
```

- Various shift and rotate options:
  - **LSL**: logical shift left
  - **LSR**: logical shift right
  - **ROR**: rotate right
  - **RRX**: rotate right extended by 1 bit

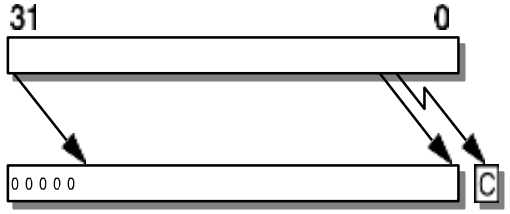
ASL: arithmetic shift left

ASR: arithmetic shift right

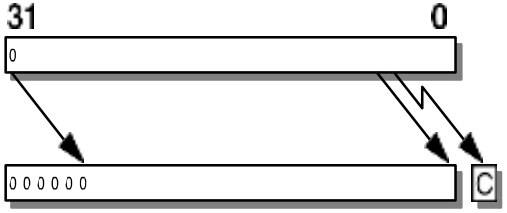
ARM



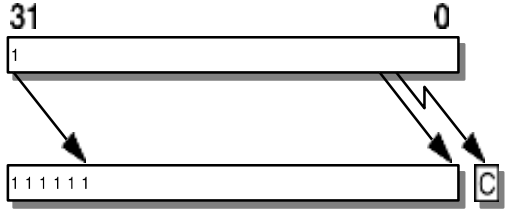
LSL # 5



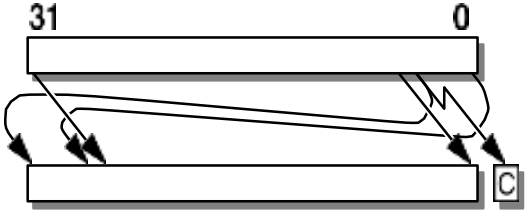
LSR # 5



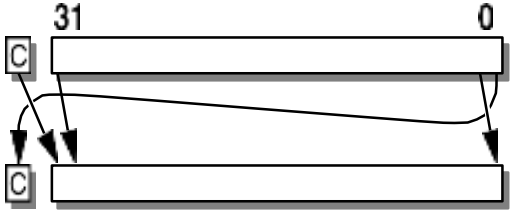
ASR # 5 – positive operand



ASR # 5 – negative operand



ROR # 5



RRX

ARM

- **Multiplication instruction**

```
MUL    r1,r2,r3              ; r1 = (r2 x r3)[31:0]
```

  - Only the least significant 32-bits are returned.
  - Immediate operands are not supported.

- **Multiply-accumulate instruction:**

```
MLA    r1,r2,r3,r4           ; r1 = (r2 x r3 + r4)[31:0]
```

  - Required in digital signal processing (DSP) applications.

- Multiplication with 64-bit results is also supported.

ARM

(b) Data Transfer Instructions

- ARM instruction set supports three types of data transfers:
  a) Single register loads and stores
     - Flexible, supports byte, half-word and word transfers
  b) Multiple register loads and stores
     - Less flexible, multiple words, higher transfer rate
  c) Single register-memory swap
     - Mainly for system use (for implementing locks)

ARM

All ARM data transfer instructions use *register indirect addressing*.

- Before any data transfer, some register must be initialized with a memory address.

Example:

```
LDR     r0,[r1]          ;   r0 = mem[r1]
STR     r0,[r1]          ;   mem[r1] = r0
```


ARM

- **Single register loads and stores**
  - The simplest form uses register indirect without any offset:

```
LDR     r0,[r1]                 ; r0 = mem[r1]
STR     r0,[r1]                 ; mem[r1] = r0
```

  - An alternate form uses register indirect with offset (limited to 4 Kbytes):

```
LDR    r0,[r1,#4]   ; r0 = mem[r1+4]
STR    r0,[r1,#12]  ; mem[r1+12] = r0
```

  - We can also use auto-indexing in addition:

```
LDR    r0,[r1,#4]!     ; r0 = mem[r1+4],  r1 = r1 +  4
STR    r0,[r1,#12]!    ; mem[r1+12] = r0, r1 = r1 +  4
```


ARM

- We can use post indexing:

```
LDR     r0,[r1],#4      ; r0 = mem[r1],      r1 = r1 + 4
STR     r0,[r1],#12     ; mem[r1] = r0,      r1 = r1 + 12
```

- We can specify a byte or half-word to be transferred:

```
LDRB        r0,[r1]          ; r0 = mem8[r1]
STRB        r0,[r1]          ; mem8[r1] = r0
LDRSH       r0,[r1]          ; r0 = mem16[r1], Load Register
                               Signed Half word
STRSH       r0,[r1]          ; mem16[r1] = r0, Store
                               Register Signed Half word
```


ARM

Examples of addressing modes in multiple-register transfer:

ARM Multiple-Register

❑ Addressing Modes LDMIA / STMIA Increment After: ↑ Address increases AFTER the first transfer

❑LDMIB / STMIB Increment Before

↑ Address increases BEFORE the first transfer

❑LDMDA / STMDA Decrement After

↓ Address decreases AFTER the first transfer

❑ LDMDB / STMDB Decrement Before

↓ Address decreases BEFORE the first transfer

ARM

Multiple register loads and stores – Example

- ARM supports instructions that transfer between several registers and memory.
- Example: LDMIA r1,{r3,r5,r6} means: **Load Multiple — Increment After**

```
LDMIA r1,{r3,r5,r6}   ;   r3   =   mem[r1]
                      ;   r5   =   mem[r1+4]
                      ;   r6   =   mem[r1+8]
```

R3 ← [R1]
R5 ← [R1 + 4]
R6 ← [R1 + 8]
R1 ← R1 + 12

LDM = Load Multiple registers
- **IA** = Increment After
- **R1** = base register containing the starting memory address
- {R3,R5,R6} = registers that receive the values
- It loads **three 32-bit words from consecutive memory locations** into R3, R5, and R6.
  - For **LDMIB**, the addresses will be **r1+4**, **r1+8**, **and r1+12**.
  - The list of destination registers may contain any or all of r0 to r15.
- Block copy addressing
  - Supported with addresses that can *increment (I)* or *decrement (D)*, before (B) or *after (A)* each transfer.

ARM

Multiple register loads and stores – Example

Assume: R9 = 0x100C
The instruction stores {R0, R1, R5}
The ! means UPDATE R9 after the operation.
R9′ = the new/updated value of R9.

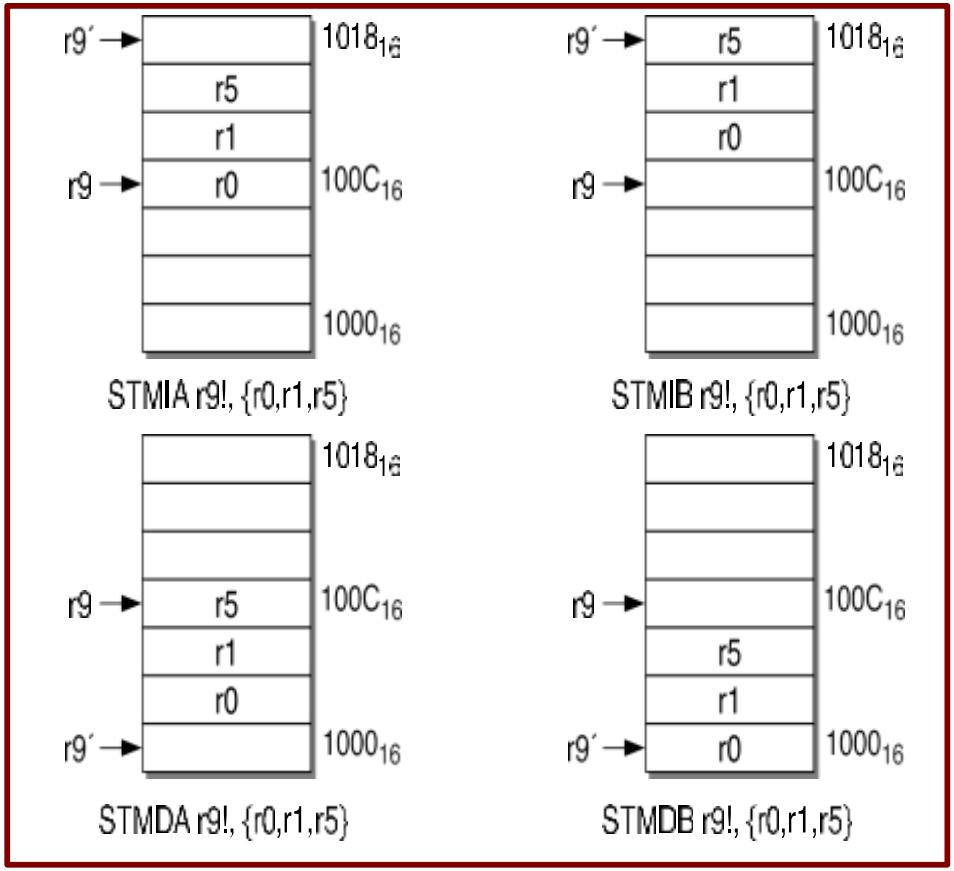
1. STMIA R9!, {R0,R1,R5}: Increment After

   First store at the current R9 address:

   0x100C → R0
   0x1010 → R1
   0x1014 → R5

   Final:
   R9′ = 0x1018

   Think: Store first → then move UP



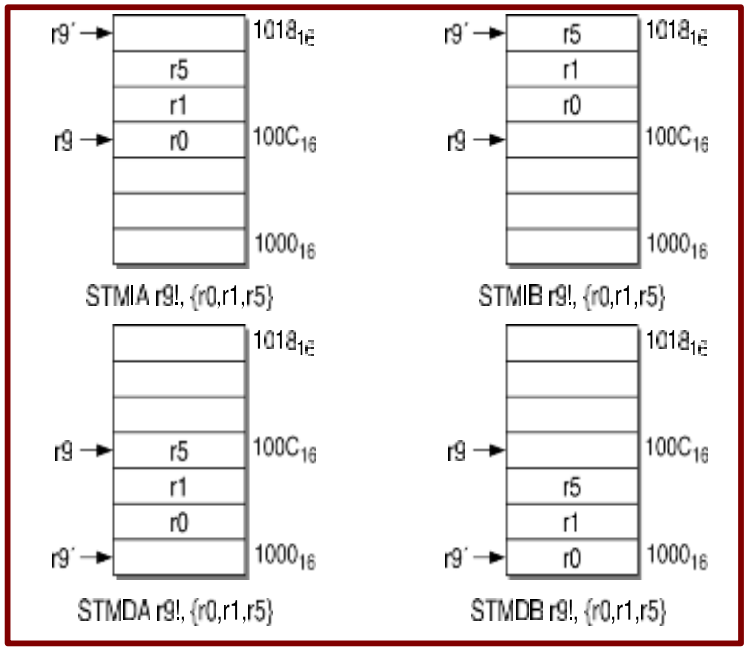
ARM

2. STMIB R9!, {R0,R1,R5}
   Increment Before

   Increase the address BEFORE the first store:

   0x1010 → R0
   0x1014 → R1
   0x1018 → R5Final:
   R9′ = 0x1018

   Think: Move UP first → then store



ARM

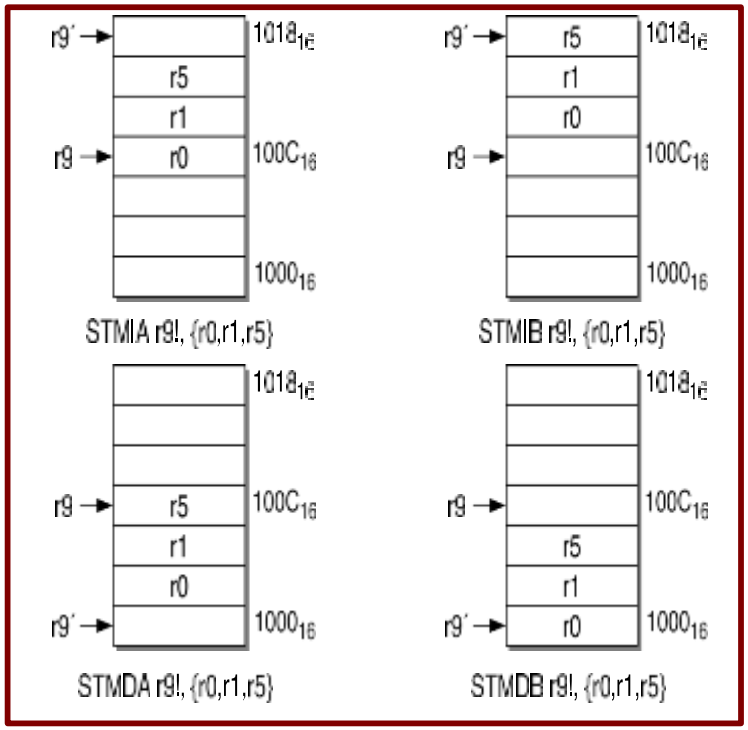
3. STMDA R9!, {R0,R1,R5}
   Decrement After

   First store at the current R9 address, then move DOWN:

   0x100C → R5
   0x1008 → R1
   0x1004 → R0

Final:
   R9′ = 0x1000

   Think: Store first → then move DOWN



ARM

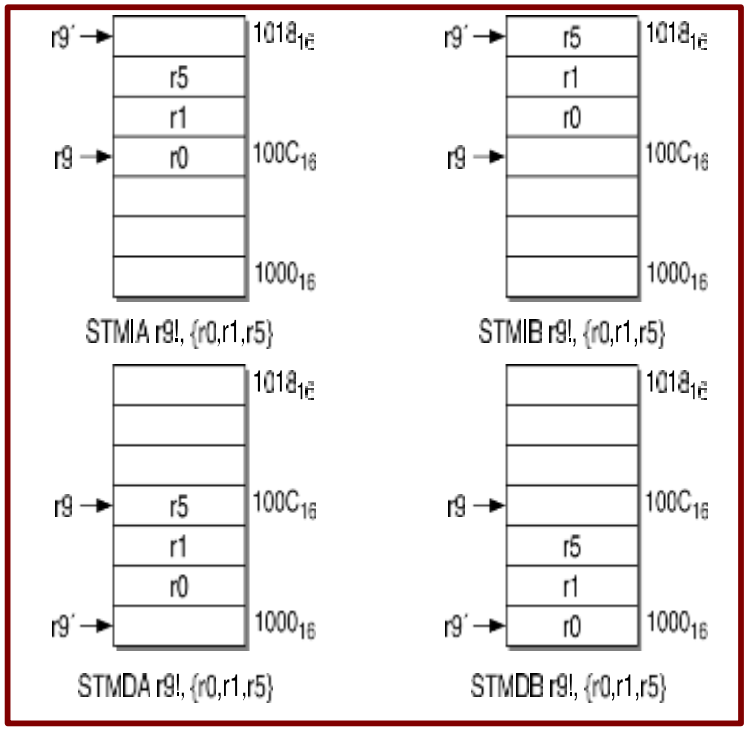
4. STMDB R9!, {R0,R1,R5}
   Decrement Before

   Decrease the address BEFORE the first store:

   0x1000 → R0
   0x1004 → R1
   0x1008 → R5

   Final:
   R9′ = 0x1000

   Think: Move DOWN first → then store



SUMMARY:

Instruction	Meaning	First Address Final R9

ARM

- **SUMMARY: Instruction Meaning First Address Final R9**

Instruction	Meaning	First Address	Final R9
STMIA	Increment After	0x100C	0x1018
STMIB	Increment Before	0x1010	0x1018
STMDA	Decrement After	0x100C	0x1000
STMDB	Decrement Before	0x1000	0x1000

EASY WAY TO REMEMBER:

I = Increment = Address goes UP ↑
D = Decrement = Address goes DOWN ↓
A = After = Change address AFTER first transfer
B = Before = Change address BEFORE first transfer

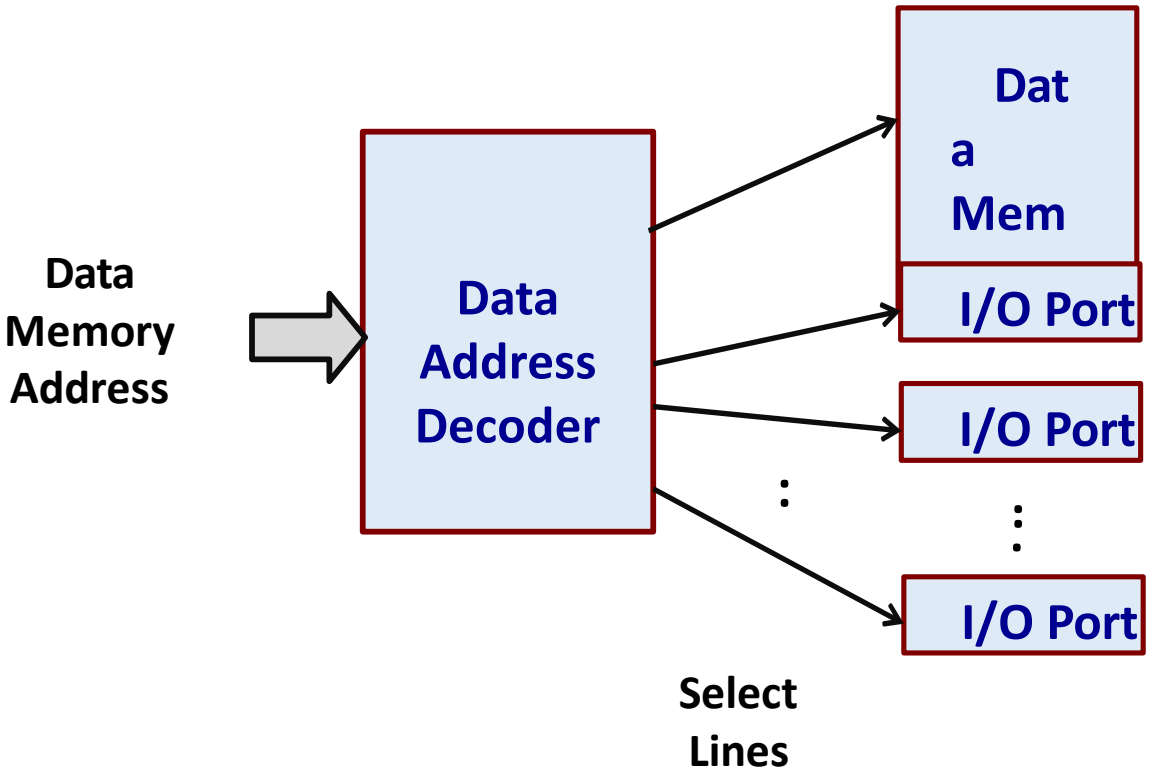
STM = Store: Registers → Memory
LDM = Load: Memory → Registers

IMPORTANT:
The ! means write-back/update the base register.

STMIA R9, {R0,R1,R5} → R9 does NOT change
STMIA R9!, {R0,R1,R5} → R9 IS updated

ARM

Memory Mapped I/O in ARM



ARM

Bit-band Operations

- Bit-band operation allows a single load/store operation to access a single bit in the memory, for example, to change a single bit of one 32-bit data:
  - Normal operation without bit-band (read-modify-write)
    - Read the value of 32-bit data
    - Modify a single bit of the 32-bit value (keep other bits unchanged)
    - Write the value back to the address
  - Bit-band operation
    - Directly write a single bit (0 or 1) to the "bit-band alias address" of the data
- Bit-band alias address
  - Each bit-band alias address is mapped to a real data address
  - When writing to the bit-band alias address, only a single bit of the data will be changed

ARM

Read-Modify-Write (RMW)

This assembly program demonstrates a **Read-Modify-Write (RMW)** operation, which is common in embedded systems programming. The steps below explain the code line by line:

```
;Read-Modify-Write Operation

LDR     R1, =0x20000000  ;Setup address
LDR     R0, [R1]         ;Read
ORR     R0, R0, #0x8     ;Modify bit
STR     R0, [R1]         ;Write back
```

1. LDR R1, =0x20000000

•**LDR** (Load Register) is used to load an address into register R1.

•R1 is now pointing to the memory address 0x20000000.This address could represent a specific memory-mapped register or peripheral in an embedded system.

2. LDR R0, [R1]

•This instruction **loads the value** stored at the memory address in R1 (which is 0x20000000) into register R0.

•In simple terms, it reads the value from memory address 0x20000000 and puts it into R0.

3. ORR R0, R0, #0x8

•**ORR** (Logical OR) performs a bitwise OR operation between the current value in R0 and the immediate value 0x8.

•This operation sets bit 3 (the 4th bit, since bit counting starts at 0) of R0 to 1, without affecting other bits.

•This is the **modify** part of the RMW operation, where a specific bit is being set in the value read from memory.

4. STR R0, [R1]

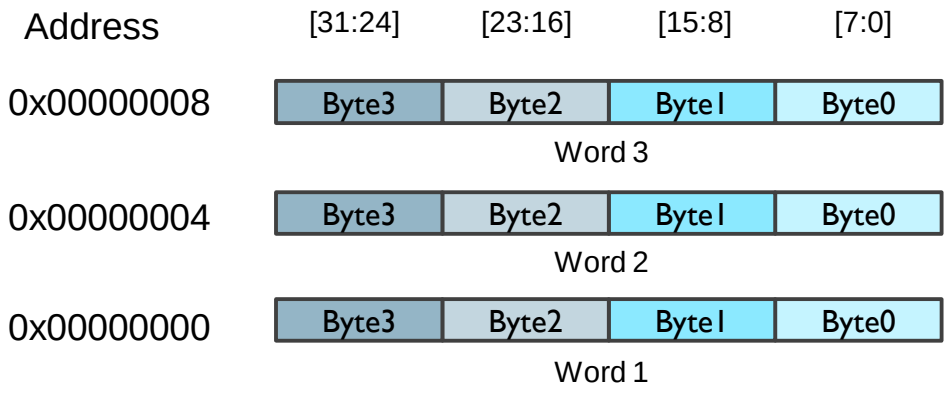
•**STR** (Store Register) writes the modified value in R0 back into the memory location at address R1 (which is 0x20000000).

•This is the **write** part of the RMW process, where the updated value is saved back to the memory.

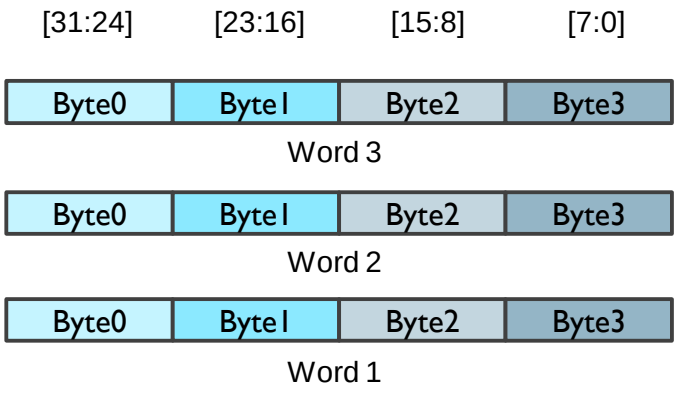
ARM

Cortex-M4 Endianness

- Endian refers to the order of bytes stored in memory
  - Little endian: lowest byte of a word-size data is stored in bit 0 to bit 7
  - Big endian: lowest byte of a word-size data is stored in bit 24 to bit 31
- Cortex-M4 supports both little endian and big endian
- However, "Endianness" only exists at the hardware level.



Little endian 32-bit memory



Big endian 32-bit memory

ARM

- No separate instructions for input/output.
- The I/O ports are treated as data memory locations.
  - Each with a unique (memory) address.
- Data input is done using the LDR instruction.
- Data output is done with the STR instruction.

ARM

(c) Control Flow Instructions

- These instructions change the order of instruction execution.
  - Normal flow is sequential execution, where PC is incremented by 4 after executing every instruction.
- Types of conditional flow instructions:
  - Unconditional branch
  - Conditional branch
  - Branch and Link
  - Conditional execution

ARM

- **Unconditional branch instruction:**

```
                B       Target
                ..
                .
Target          ..
                .
                ..
```

- **Conditional branch instruction:**

```
                MOV     r2,#0
LOOP            ...
                ...
                ADD     r2,r2,#1
                CMP     r0,#20
                BNE     LOOP
                ...
```


ARM

Cortex-M4 Registers

- APSR, combined Program Status Register
  - Provides information about program execution and ALU flags



ARM

ARM CONDITIONAL BRANCH INSTRUCTIONS

B, BAL
B = Branch
BAL = Branch Always

BEQ, BNE
BEQ = Branch if Equal
BNE = Branch if Not Equal

BPL, BMI
BPL = Branch if Plus (N = 0)
BMI = Branch if Minus (N = 1)

BCC, BCS
BCC = Branch if Carry Clear (C = 0)
BCS = Branch if Carry Set (C = 1)

BVC, BVS
BVC = Branch if Overflow Clear (V = 0)
BVS = Branch if Overflow Set (V = 1)

ARM

ARM BRANCH INSTRUCTIONS — COMPARISONS

BGT, BGE
BGT = Branch if Greater Than
BGE = Branch if Greater Than or Equal

BLT, BLE
BLT = Branch if Less Than
BLE = Branch if Less Than or Equal

EASY WAY TO REMEMBER:

EQ = Equal
NE = Not Equal

PL = Plus
MI = Minus

CC = Carry Clear
CS = Carry Set

VC = Overflow Clear
VS = Overflow Set

GT = Greater Than
GE = Greater Than or Equal

LT = Less Than
LE = Less Than or Equal

- **Example Branch and link instruction**
  - Used for calling subroutines in ARM.
  - The return address is saved in register r14 (called *link register*).
  - To return from the subroutine, we have to jump back to the address stored in r14.

```
            BL      MYSUB           ; Branch to subroutine
            ...                     ; Return here
            ...
MYSUB       ...                     ; Subroutine starts here
             ...
            MOV     pc,r14          ; Return
```

Nested subroutine calls cannot be used in this way.

ARM

- We can use R14: Link Register (LR) to save/restore the return address and registers. An example showing nested subroutine calls and return.

```
            BL      MYSUB1
            ...

MYSUB1      STMFD   r13!,{r0-r2,r14}
            BL      MYSUB2
            ...
            LDMFD   r13!,{r0-r2,pc}

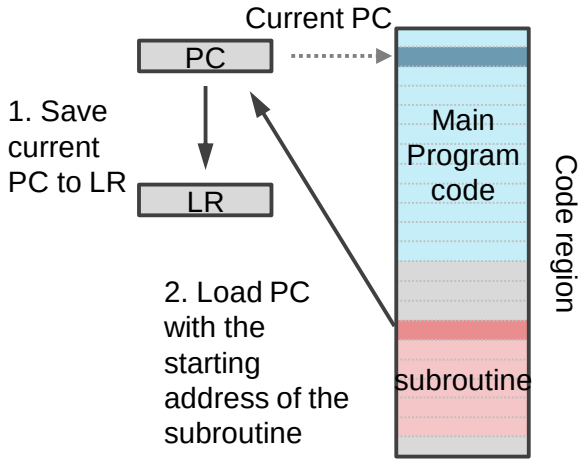
MYSUB2      ...
            ...
            MOV     pc,r14
```

Nested subroutine calls can be used.

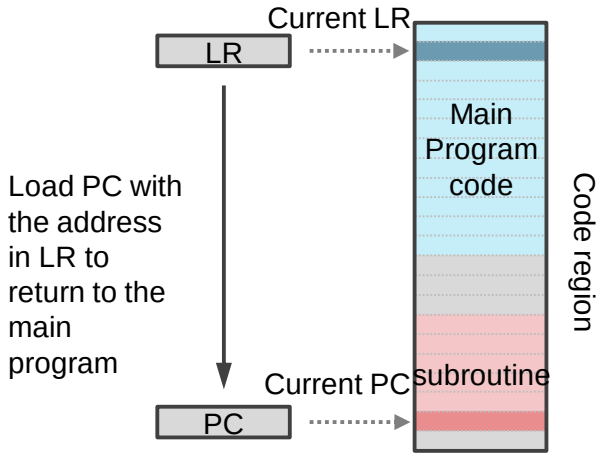
ARM

Cortex-M4 Registers

- R14: Link Register (LR)
  - The LR is used to store the return address of a subroutine or a function call
  - The program counter (PC) will load the value from LR after a function is finished



Call a subroutine

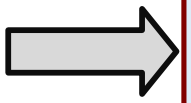


Return from a subroutine to the main program

ARM

- **Conditional execution**
  - A unique feature of the ARM instruction set.
  - All instructions can be made conditional, i.e. will get executed only when a specified
    condition is true.
  - Helps in removing many short branch instructions (improves performance and code density)
  - An example: `if (r2 != 10) r5 = r5 +10 - r3`

```
        CMP     r2,#10
        BEQ     SKIP
        ADD     r5,r5,r2
        SUB     r5,r5,r3
SKIP    ...
```



```
CMP     r2,#10
ADDNE   r5,r5,r2
SUBNE   r5,r5,r3
```


ARM

- Various instruction postfix supported for conditional execution:

Postfix	Condition
CS	Carry set
EQ	Equal (zero set)
VS	Overflow set
GT	Greater than
GE	Greater than or equal
PL	Plus (positive)
HI	Higher than
HS	Higher or same (i.e. CS)

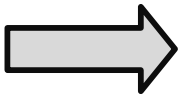
Postfix	Condition
CC	Carry clear
NE	Not equal (zero clear)
VC	Overflow clear
LT	Less than
LE	Less than or equal
MI	Minus (negative)
LO	Lower than (i.e. CC)
LS	Lower or same

ARM

- Another example:

```
if ((r1 == r3) && (r5 == r6)) r7
= r7 + 10
```

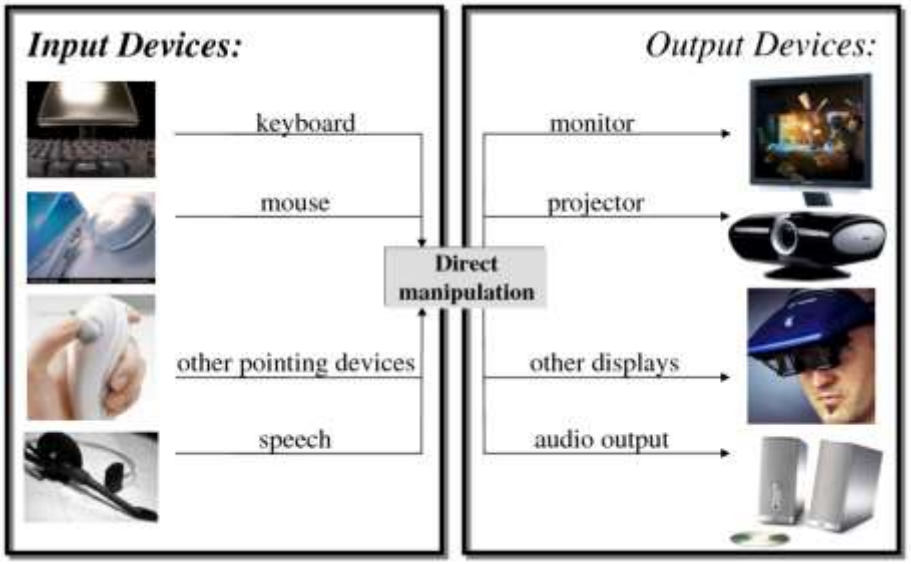
```
        CMP   r1,r3
        BNE   SKIP
        CMP   r5,r6
        BNE   SKIP
        ADD   r7,r7,#10
SKIP    ...
```



```
CMP   r1,r3
CMPEQ r5,r6
ADDEQ r7,r7,#10
```


ARM

Next Lecture: Input and Output (I/O)



Next Lecture

In the next lecture, we will discuss:

ADC (Analog-to-Digital Converter)

DAC (Digital-to-Analog Converter)

Some **Communication protocols used for interfacing**

Common interfacing nomenclature and terminology

We will look at how embedded systems **convert signals and communicate with sensors, actuators, and other devices**.

- SPI: Serial Peripheral Interface: Serial communication bus, widely used for attaching lower-speed peripheral ICs to microcontrollers over short distance.
- I2C: Inter-Integrated Circuit
- PWM: Pulse Width Modulation
- CAN: Controller Area Network
- A robust vehicle bus standard, designed to allow microcontrollers and devices to communicate with each other without a host computer.

ARM

Digital Logic Interface Voltages

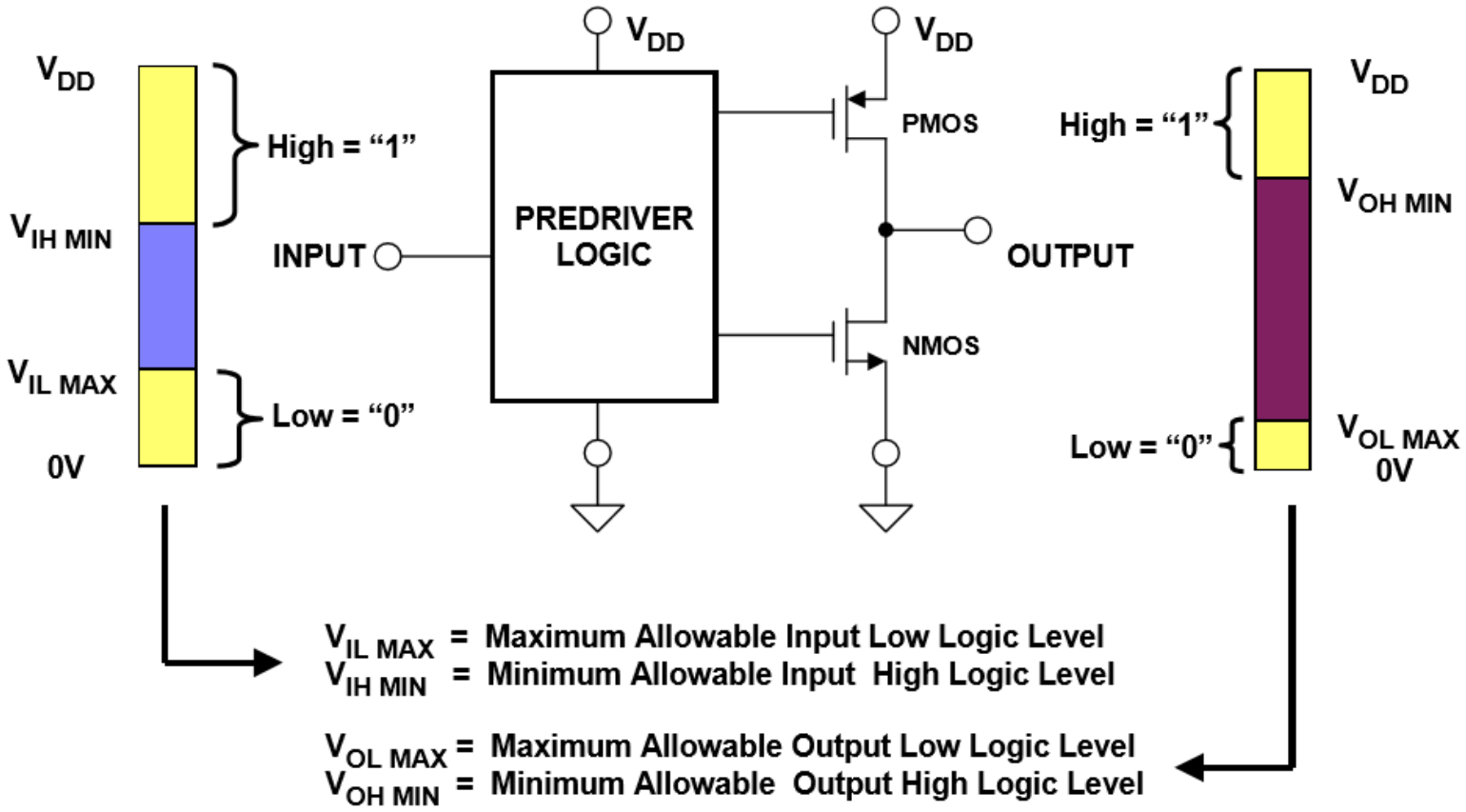
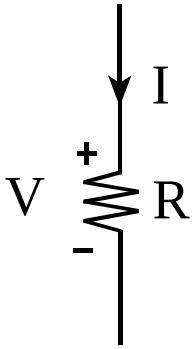


Figure 2: Typical CMOS IC Output Driver Configuration

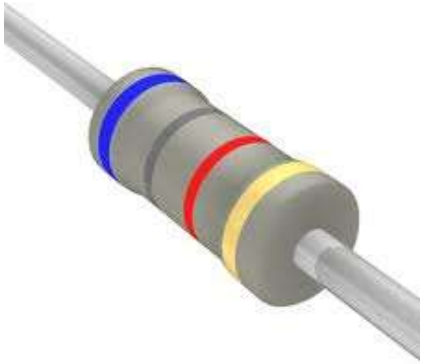
Ohm's Law

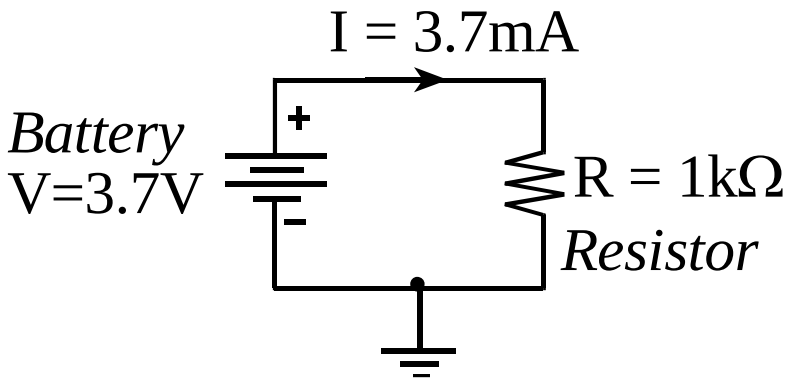


$V = I \times R$ (Voltage = Current × Resistance)

$I = V / R$ (Current = Voltage / Resistance)

$R = V / I$ (Resistance = Voltage / Current)





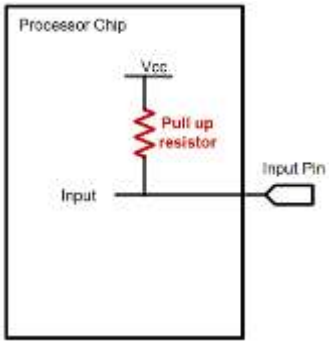
Resistive (Real) Power Formulas:

$P = V \times I$	Power = Voltage × Current
$P = V^2 / R$	Power = Voltage^2 / Resistance
$P = I^2 \times R$	Power = Current^2 × Resistance

ARM

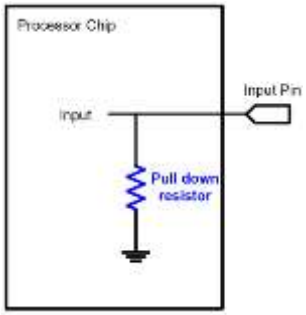
GPIO Input: Pull Up/Down Resistors

- **Pull-Up and Pull-Down Switches**
- A digital input can have three states: High, Low, and High-Impedance (also called floating, tri-stated, HiZ)



Pull-Up

If external input is HiZ, the input is read as a valid HIGH.



Pull-Down

If external input is HiZ, the input is read as a valid LOW.

Pull-Up and Pull-Down switches are essential configurations in digital circuits to ensure that an input pin is never left floating and always has a defined logic state.

Pull-Up Resistor:

- A pull-up resistor connects the input pin to a high voltage level (usually the power supply, VCC).
- When the switch is open, the input pin reads as logic **HIGH**.
- When the switch is closed, the input pin is pulled to **ground (LOW)**.

Pull-Down Resistor:

- A pull-down resistor connects the input pin to the ground (GND).
- When the switch is open, the input pin reads as logic **LOW**.
- When the switch is closed, the input pin is pulled to a **high voltage (HIGH)**.

Use Cases: Pull-Up and Pull-Down Switch configurations
These configurations help to avoid **floating inputs**, which can lead to unpredictable behavior in digital circuits.
Pull-Up Switches: Typically used when a button press or switch closure should produce a **LOW** signal.
Pull-Down Switches: Typically used when a button press or switch closure should produce a **HIGH** signal.

ARM

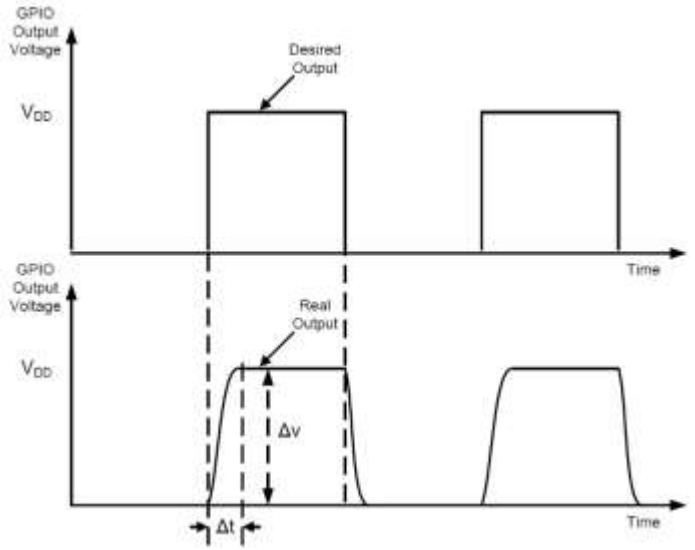
GPIO Output: Slew Rate

Slew Rate: Maximum rate of change of the output voltage
Slew Rate is a key parameter in analog electronics, especially for amplifiers and operational amplifiers (op-amps). It refers to the maximum rate of change of the output voltage over time and is typically expressed in **volts per microsecond (V/µs)**.

$$Slew\ Rate = max\left(\frac{\Delta V}{\Delta t}\right)$$

Importance:

- A high slew rate allows the output to be toggled at a fast speed.
- For example: a higher slew rate means an amplifier can respond more quickly to fast-changing input signals (signals with very high-frequency).
- If the input signal changes faster than the amplifier's slew rate, then the output may distort or "lag" behind the input, especially for high-frequency signals applications

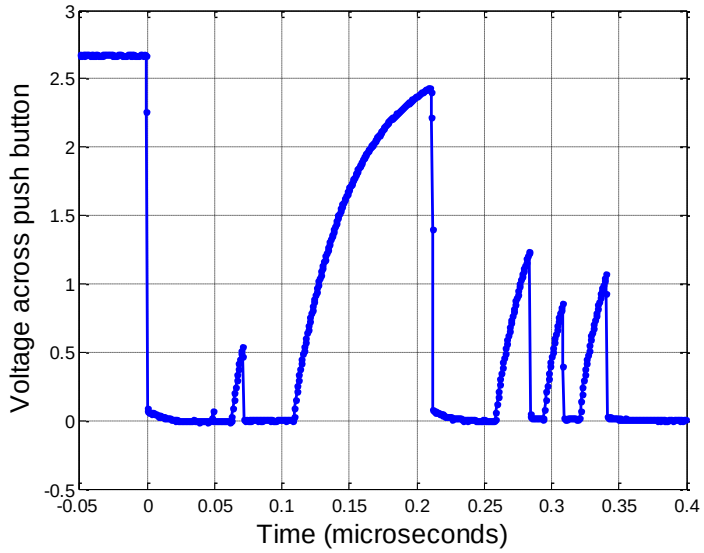


ARM

I/O Debouncing

- Example signal when a switch or button is pressed:

• When a button is pressed or released, the contact between the metal parts can "bounce," leading to multiple transitions that can be mistakenly detected as multiple presses.
•I/O debouncing refers to the process of eliminating unwanted, erratic, or spurious signals that arise when an input signal is toggled. This typically occurs with mechanical switches or buttons
Example:
Imagine pressing a mechanical button connected to a microcontroller. Ideally, when the button is pressed, the signal changes from high to low (or vice versa), but in reality, due to the bouncing, the signal can fluctuate rapidly before settling. This rapid fluctuation can result in the microcontroller detecting multiple presses when only one was intended.
Solution: Debouncing techniques can be implemented using either hardware or software:
Hardware debouncing: A small circuit that can capacitor noise is placed across the button to smooth out the signal and eliminate noise.
Software debouncing: The microcontroller ignores signal changes for a brief period (e.g., 10-50 milliseconds) after the first detection, allowing the signal to settle.
This ensures that only a single signal change is registered, leading to more accurate and stable input readings.



Example Code for Configuring a Switch on TM4C123 (Tiva C)

- The following is the typical code C for Configuring a Switch on TM4C123 (Tiva C): Assume we have a switch connected to Pin PF4, which corresponds to SW1 on Tiva C LaunchPad

```
#include <stdint.h>

#include "tm4c123gh6pm.h"

void PortF_Init(void) {

// 1. Enable clock for Port F

    SYSCTL_RCGCGPIO_R |=
    SYSCTL_RCGCGPIO_R5;

// 2. Allow time for clock to stabilize

   while ((SYSCTL_PRGPIO_R &
    SYSCTL_PRGPIO_R5) == 0) {}

// 3. Set PF4 as input
   GPIO_PORTF_DIR_R &= ~0x10;
// 4. Enable digital function on PF4
   GPIO_PORTF_DEN_R |= 0x10;
// 5. Enable pull-up resistor on PF4
    GPIO_PORTF_PUR_R |= 0x10; }
int main(void) { PortF_Init(); // Initialize Port
F and switch
while (1) { if ((GPIO_PORTF_DATA_R &
0x10) == 0) {
 // Switch is pressed (PF4 is low) }
else {
// Switch is not pressed (PF4 is high) } } }
```


ARM

Explication of the Code:

\\ 1. SYSCTL_RCGCGPIO_R |= SYSCTL_RCGCGPIO_R5;

- **SYSCTL_RCGCGPIO_R**: a register in the System Control (SYSCTL) module of the microcontroller.
- The **Run Mode Clock Gating Control Register for GPIO** (RCGCGPIO) controls whether the clock to the GPIO ports is enabled or disabled.
- **SYSCTL_RCGCGPIO_R5**: This is a predefined constant that corresponds to the bit for **GPIO Port F**. In the TM4C123 microcontroller, each GPIO port (A, B, C, D, E, F) has a bit in this register. Bit 5 corresponds to **Port F**.
- SYSCTL_RCGCGPIO_R |= SYSCTL_RCGCGPIO_R5; The bitwise OR assignment operator (|=) is used to set a specific bit in the register without affecting other bits.
- In this case, it is used to ensure that the clock for **Port F** is turned on by setting **bit 5** to 1.

ARM

Explication of the Code:

```
// 2. Allow time for clock to stabilize
While ((SYSCTL_PRGPIO_R & SYSCTL_PRGPIO_R5) == 0) {}
```

- The line of code is to ensures that the microcontroller waits until **GPIO Port F** is completely ready to be used after the clock has been enabled for it.
- After enabling the clock for a GPIO port (SYSCTL_RCGCGPIO_R |= SYSCTL_RCGCGPIO_R5), there may be a slight delay before the peripheral is fully operational.
- The while loop continuously checks whether **Port F** is ready by reading the **SYSCTL_PRGPIO_R** register.
- In another word the While loop allow the necessary waits until **bit 5** (for Port F) is set to 1, which indicates that the port is now fully initialized and ready to use.

ARM

Explication of the Code:

/ 3. Set PF4 as input GPIO_PORTF_DIR_R &= ~0x10;

- GPIO_PORTF_DIR_R is the **data direction register** for **Port F**.
- Each bit in this register controls the direction of the corresponding pin (input or output).
- ~0x10 is the bitwise NOT of 0x10, which is 11101111 in binary. This is needed to clear bit 4 (PF4) in the GPIO_PORTF_DIR_R register, configuring PF4 as an **input**.
- **Result**: PF4 is set to be an input pin.

ARM

Explication of the Code:

// 4. Enable digital function on PF4 GPIO_PORTF_DEN_R |= 0x10;

- The GPIO_PORTF_DEN_R is the digital enable register.
- It enables the digital function for each pin.
- Setting bit 4 (0x10 in hexadecimal) is needed to enable the digital function for the device connected at the pin PF4.

Result:

PF4 is enabled as a digital input pin for any device connected to this pin to read date from this device .

ARM

Explication of the Code:

// 5. Enable pull-up resistor on PF4 GPIO_PORTF_PUR_R |= 0x10;

- GPIO_PORTF_PUR_R is the pull-up resistor enable register.
- Setting bit 4 enables an internal pull-up resistor for PF4.
- A pull-up resistor ensures that when no active signal (such as a button press) is present, the input is pulled to a high state (logic 1).
- **Result:** PF4 has an internal pull-up resistor, so the input is high by default unless the switch is pressed the input will be grounded to 0.

ARM

Explication of the Code: Main Function and Polling of PF4

int main(void) { PortF_Init(); // Initialize Port F and switch
while (1)
{ if ((GPIO_PORTF_DATA_R & 0x10) == 0) { // Switch is pressed (PF4 is low) } else { //
Switch is not pressed (PF4 is high) } } }

- PortF_Init() is the main function that contains the code to initialize PF4 as an input with a pull-up resistor (as described earlier).
- while (1) creates an **infinite loop** where the program continuously checks the status of PF4.

Checking the Switch State:
if ((GPIO_PORTF_DATA_R & 0x10) == 0) {

- GPIO_PORTF_DATA_R is the **data register** for Port F, which holds the input/output values of the pins.
- 0x10 (binary: 00010000) isolates bit 4 (PF4) using a bitwise AND operation.
- If the result is 0, it means PF4 is **low**, indicating that the switch (connected to PF4) is pressed.

ARM

Explication of the Code: Main Function and Polling of PF4

// Switch is pressed (PF4 is low)

Switch Pressed:

•If PF4 is low, the switch is pressed. We can place the code that needs to be performed here to respond to the button press (such as toggling an LED).

else { // Switch is not pressed (PF4 is high) }

Switch Not Pressed:

• If PF4 is high, the switch is not pressed, thanks to the internal pull-up resistor keeping or make sure the input is always remine high.

ARM

Course Name: Embedded System

Lecture 5: Analog to Digital Conversion

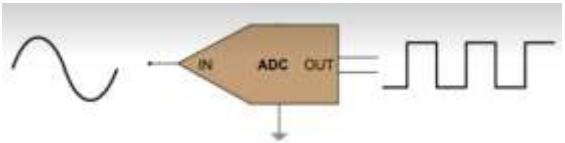
The slides have been adapted from materials provided by ARM website, **Jonathan W. Valvano (The University of Texas at Austin)** **NPETL Online Certificate Notes (IIT)**, and **Tennessee Tech University (TTU) ECE 3130 course materials, and other internet sources**

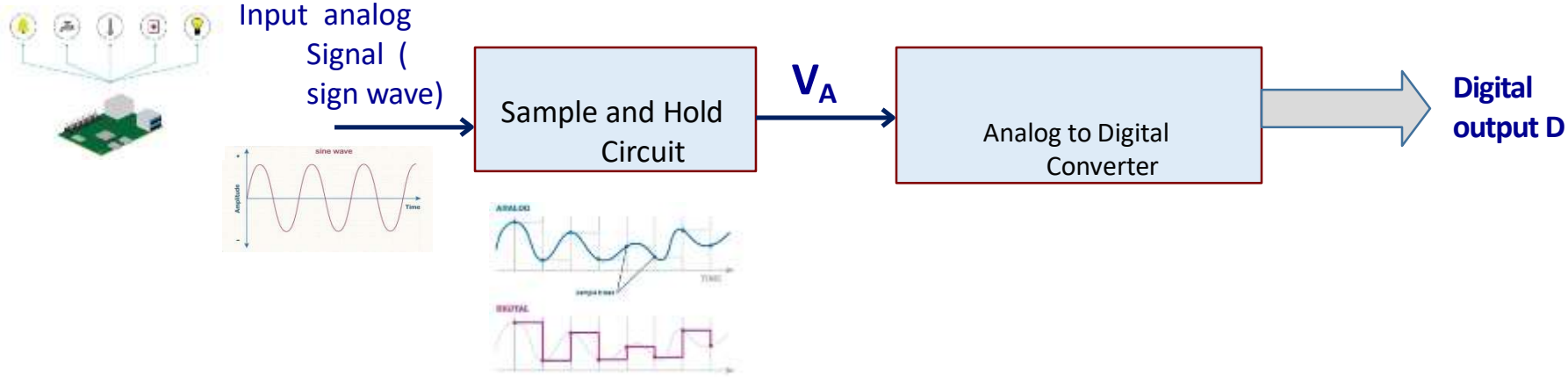
- ❑ Introduction to analog-to-digital conversion
- ❑ Choice of signal sampling rate
- ❑ Flash-type, counter-type and tracking-type ADC
- ❑ Successive-approximation type ADC
- ❑ A/D conversion in ARM microcontrollers

Introduction

- An analog-to-digital converter (ADC) takes an analog input V_A as input, and generates a digital word D as output, where $D \alpha V_A$.
- Various types of ADC are possible:
  a) Flash-type ADC
  b) Counter-type ADC
  c) Tracking-type ADC
  d) Successive approximation ADC
  e) Dual-slope ADC

- If the input signal changes during the conversion, the final value D may be erroneous.
- We need a *sample-and-hold circuit* to sample the input voltage, and hold it to a constant value while the conversion is in progress.





Overall Schematic of ADC

ADC Applications



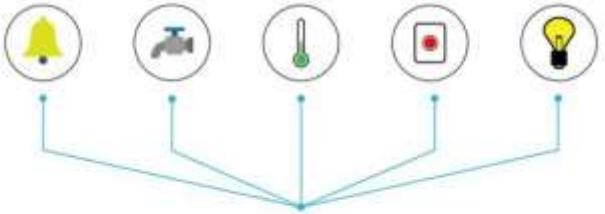


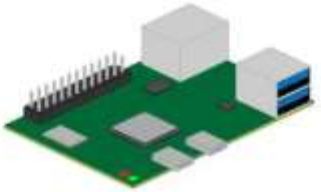


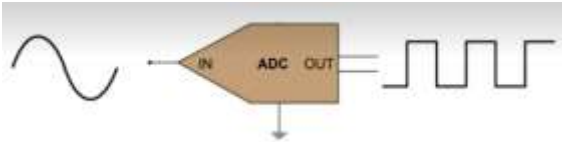






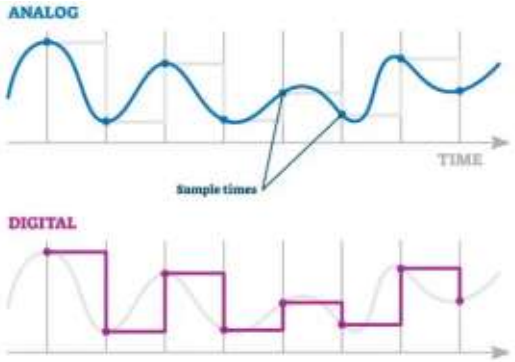






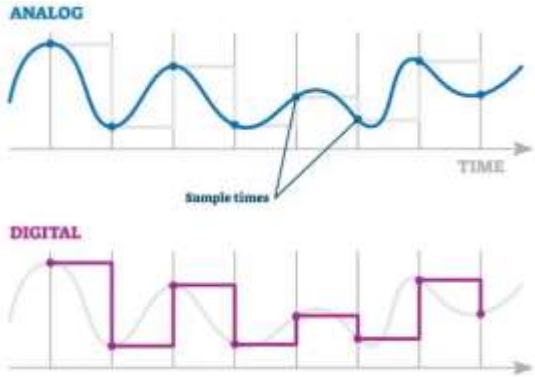
Digital vs. Analog Inputs: Signal Characteristics

Attribute	Digital Input	Analog Input
Signal	Discrete values (0 or 1)	Continuous range of values
Waveform	Usually square wave	Often sine or continuously varying wave
Signal Levels	Limited, defined levels	Many possible values
Noise	Generally less sensitive to noise	Generally more sensitive to noise
Bandwidth	Depends on data rate and switching speed	Depends on signal frequency/content
Processing	Directly processed by digital systems	Usually requires an ADC for digital processing

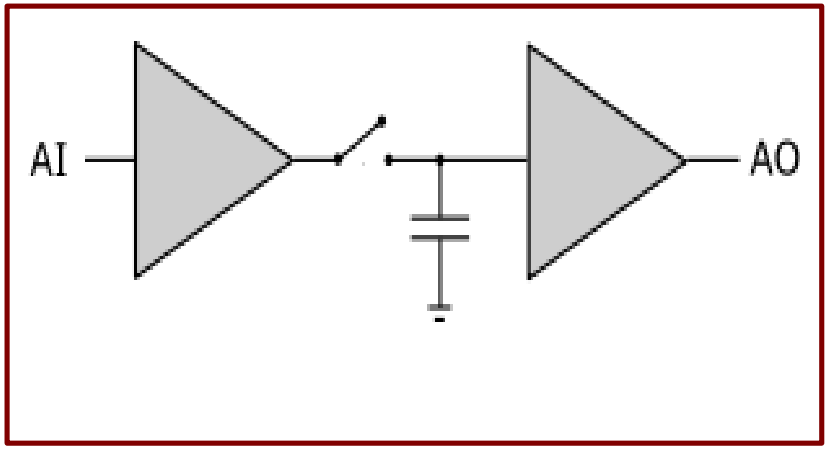


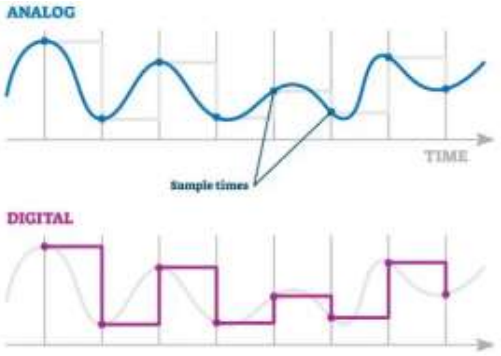
Digital vs. Analog Inputs: Signal Characteristics

Attribute	Digital Input	Analog Input
Typical Components	Logic gates, switches, microcontrollers	Sensors, amplifiers, filters
Conversion	DAC may be needed for analog output	ADC needed for digital processing
Accuracy	Determined by logic thresholds and digital resolution	Affected by sensor accuracy, noise, and ADC resolution
Power	Depends on switching speed and circuit design	Depends on analog circuit design
Applications	Computers, keyboards, switches, digital sensors	Temperature, pressure, light, sound
Examples	Push button, digital switch, GPIO signal	Microphone, thermistor, light sensor



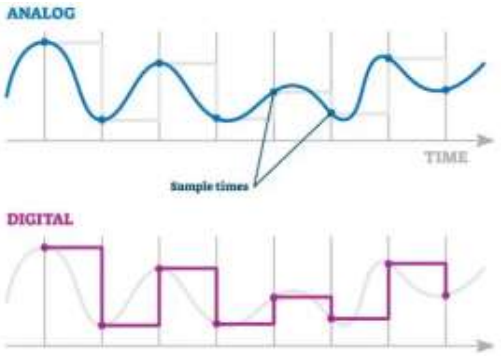
Sample-and-Hold Circuit





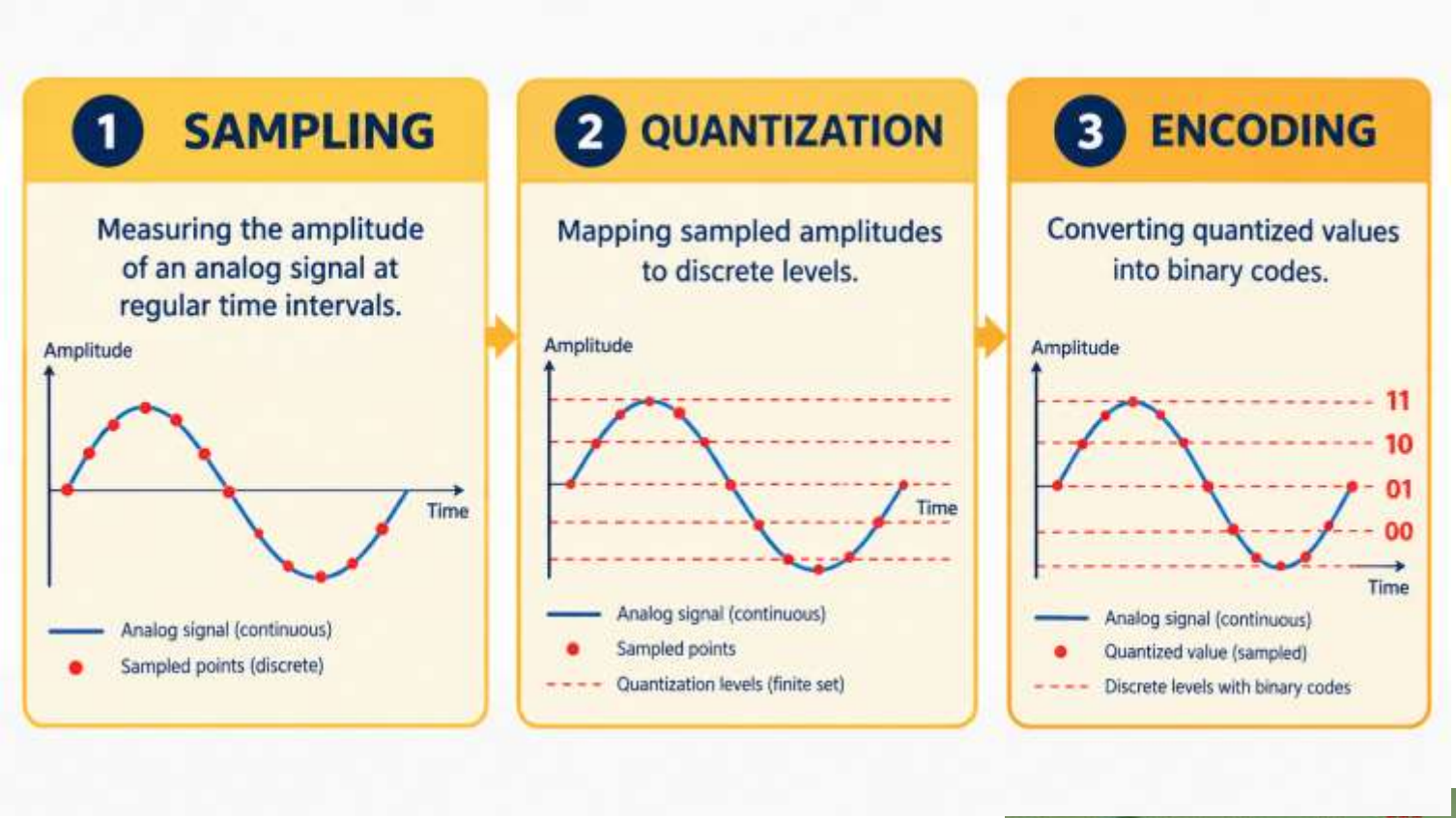
- The input voltage *AI* is sampled by closing the switch; the capacitor charges to the voltage.
- During conversion, the switch is opened again, so that *AO* remains reasonably constant.

How Fast We Must Sample?



- A very important decision.
  - Guided by *Nyquist Theorem*.
- What is Nyquist Theorem?
  - A *band-limited* analog signal that has been sampled can be perfectly reconstructed from an infinite sequence of samples if the sampling rate f_s exceeds $2f_{max}$ samples per second, where f_{max} is the highest frequency in the original signal.
    - If the analog signal does contain frequency components larger than $f_s/2$, then there will be an *aliasing error*.
    - Aliasing is when the digital signal appears to have a different frequency than the original analog signal.

ADC Basic Operation





ADC Resolution (Minimum Step Size)

ADC Resolution

- **Resolution** is the **smallest change in input voltage that an ADC can detect or represent.**
- It is determined by the number of **bits** used by the ADC.
- The resolution corresponds to the voltage represented by **1 Least Significant Bit (LSB).**
- For an **N-bit ADC**:
- **Resolution tells us how small a voltage change the ADC can distinguish.**
  A smaller voltage step means **higher resolution.**

$$\text{Resolution} \approx \frac{V_{range}}{2^N}$$

Example: For example, if one ADC step represents **0.4 mV**, the ADC resolution is **0.4 mV per LSB.**
If the ADC has a resolution of **0.4 mV**, an increase of 0.4 mV corresponds approximately to **one ADC count (1 LSB).**

Important distinction: The **sample-and-hold circuit samples and holds the analog voltage**; the **ADC's quantizer/encoder converts that voltage into a digital code.** Resolution is primarily determined by the ADC's bit depth and input voltage range.

Resolution (Step)

- Related to full scale if the ADC is linear.
  - $LSB = A / 2^n$, for an n-bit ADC.
  - For a linear 8-bit ADC with a 1V full scale input, Resolution $= 1.0 / 2^8 = 3.9$ mV

    Percentage Resolution = Resolution / (Full-scale voltage) x 100 %

1. ADC Resolution – Example

ADC Resolution (Minimum Step Size)

Resolution = the **minimum change in analog input voltage that an ADC can detect.**

$$\boxed{\text{Resolution} = \frac{V_{max} - V_{min}}{2^N}}$$

where **N** = number of ADC bits.

Example:

For a 12-bit ADC with a 0–3.3 V input range:

$$\frac{3.3}{4096} \approx 0.806 \text{ mV}$$

So, **one ADC step (1 LSB) ≈ 0.806 mV.**

2. Conversion Time and Bandwidth

- How often can a conversion be done?
  - A few ns to a few ms depending on the technology.
- Input bandwidth
  - Maximum input signal bandwidth
    - Sample and hold input circuitry
    - Conversion frequency

3. Conversion Rate

How often can a conversion be done? — Conversion Rate

This means **how many analog-to-digital conversions the ADC can perform per second.**

Example:

If an ADC takes **1 µs** to complete one conversion:

$$f_{conversion} = \frac{1}{1\mu s} = 1\,MHz$$

So the ADC can perform approximately **1 million conversions per second.**

4. Conversion time

Conversion time can be in Few ns to few ms

- **Shorter conversion time → higher conversion frequency.**
- Different ADC technologies have different conversion speeds.
- **Example:**
  - Fast ADC → conversion may take **10 ns**
  - Medium-speed ADC → **1 µs**
  - Slow ADC → **1 ms**

5. Input Bandwidth

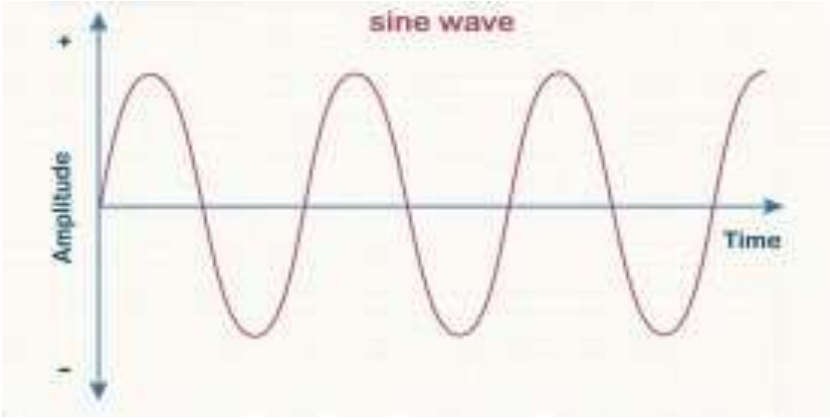
- **Input bandwidth** is the range of frequencies that the ADC/input circuitry can properly handle.

Example:

- Suppose an ADC system has an input bandwidth of **100 kHz**.
  - The means, it basically can properly process signals containing frequencies up to approximately **100 kHz**.
  - A **10 kHz** sensor signal → within the bandwidth.
  - A **500 kHz** signal → outside the specified bandwidth.

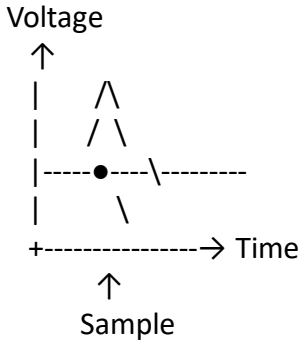
5. Maximum Input Signal Bandwidth

- This is the **highest frequency of the input signal that the ADC system is designed to handle**.
- **Example:**
- If BW=100 kHz the maximum specified input frequency is **100 kHz**.
- For a sine wave:
  - 1 kHz → ✓
  - 20 kHz → ✓
  - 100 kHz → at the limit
  - 500 kHz → ✗



6. Sample-and-Hold Input Circuitry

- At the sampling instant, the circuit captures the voltage,
- For example:
  - Vin=1.85V
  - It then **holds 1.85 V** while the ADC converts it into a digital number.



7. Conversion Frequency

6. Conversion Frequency

Conversion frequency = number of ADC conversions performed per second.

$$\boxed{f_s = \frac{1}{T_{conversion}}}$$

Example:

If one conversion takes:

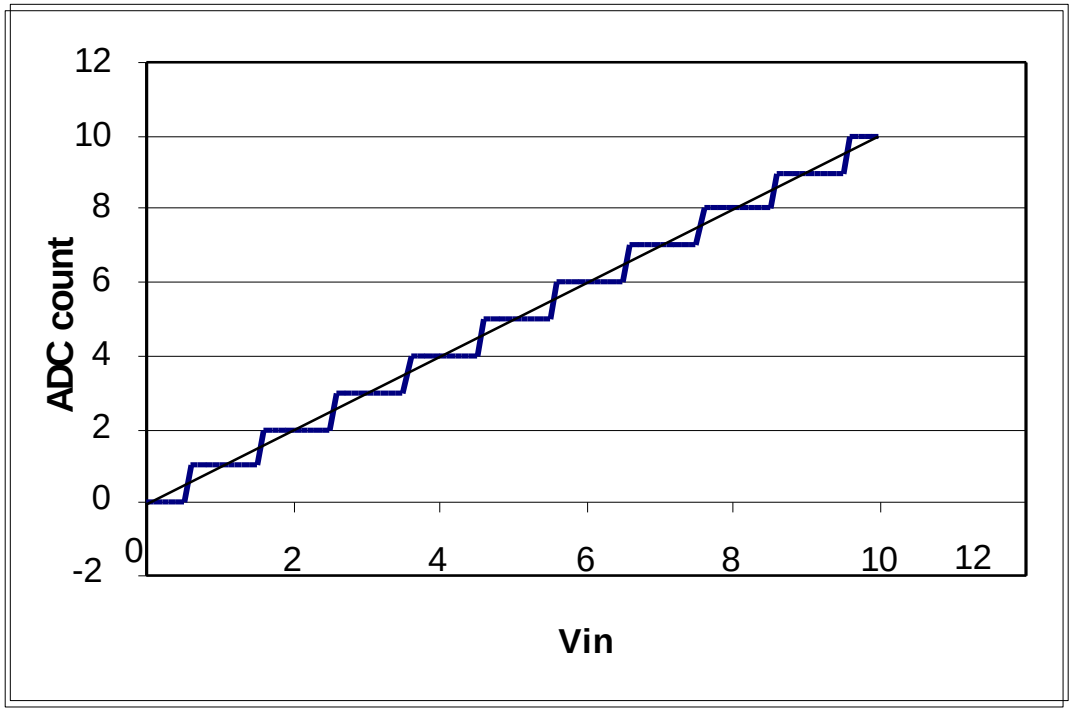
$$T = 10\mu s$$

then:

$$f_s = \frac{1}{10\mu s} = 100\,kHz$$

So the ADC can perform **100,000 conversions per second.**

ADC Transfer Characteristics Offset and Non-linearity Example



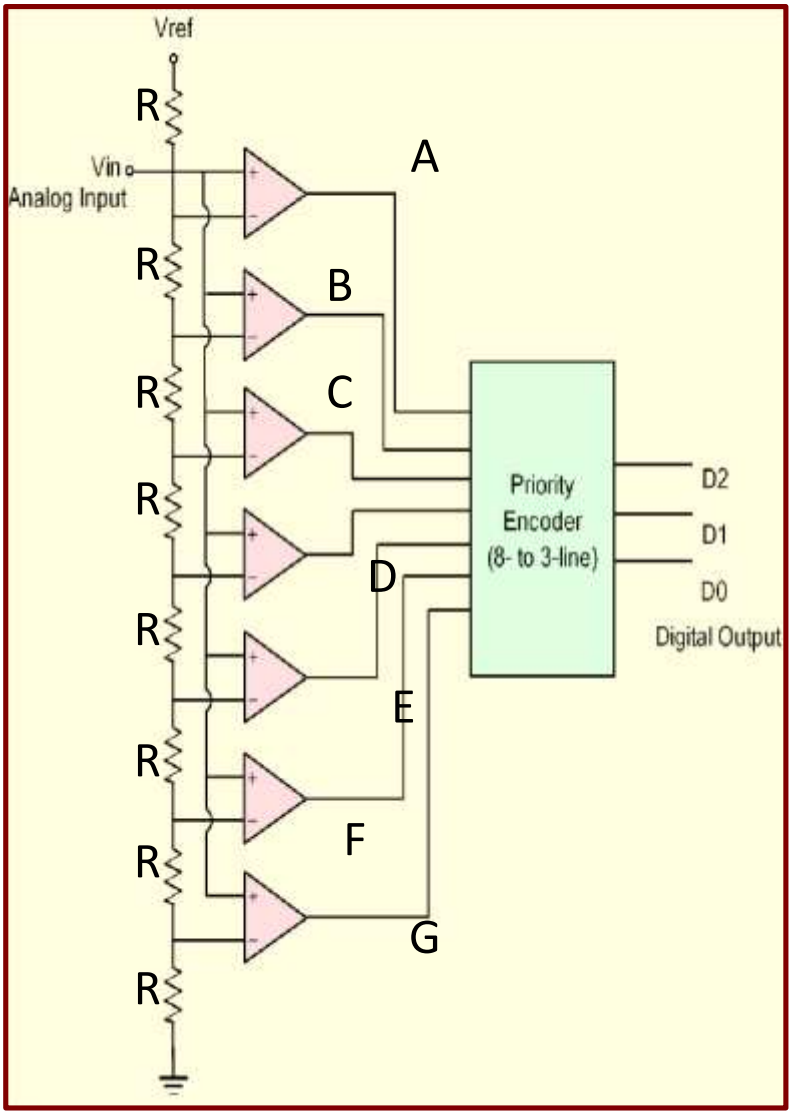
- This curve is for ideal ADC.
- Possible errors:
  - Offset
  - Non-linearity

Summary

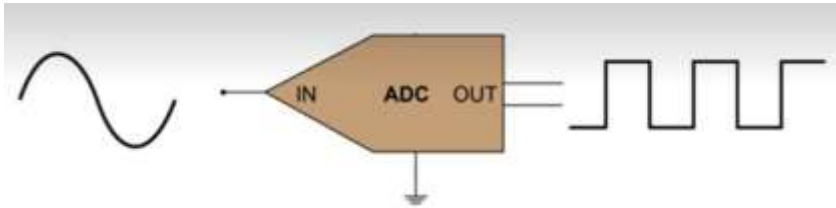
Term	Simple meaning	Example
Conversion time	Time for one conversion	10 µs
Conversion frequency	Conversions per second	100 kHz
Input bandwidth	Frequency range the input can handle	0–100 kHz
Maximum input bandwidth	Highest specified input frequency	100 kHz
Sample & Hold	Captures and holds input voltage	Holds 1.85 V
Conversion rate	How quickly conversions can be repeated	100,000 conversions/s

(a) Flash-type ADC

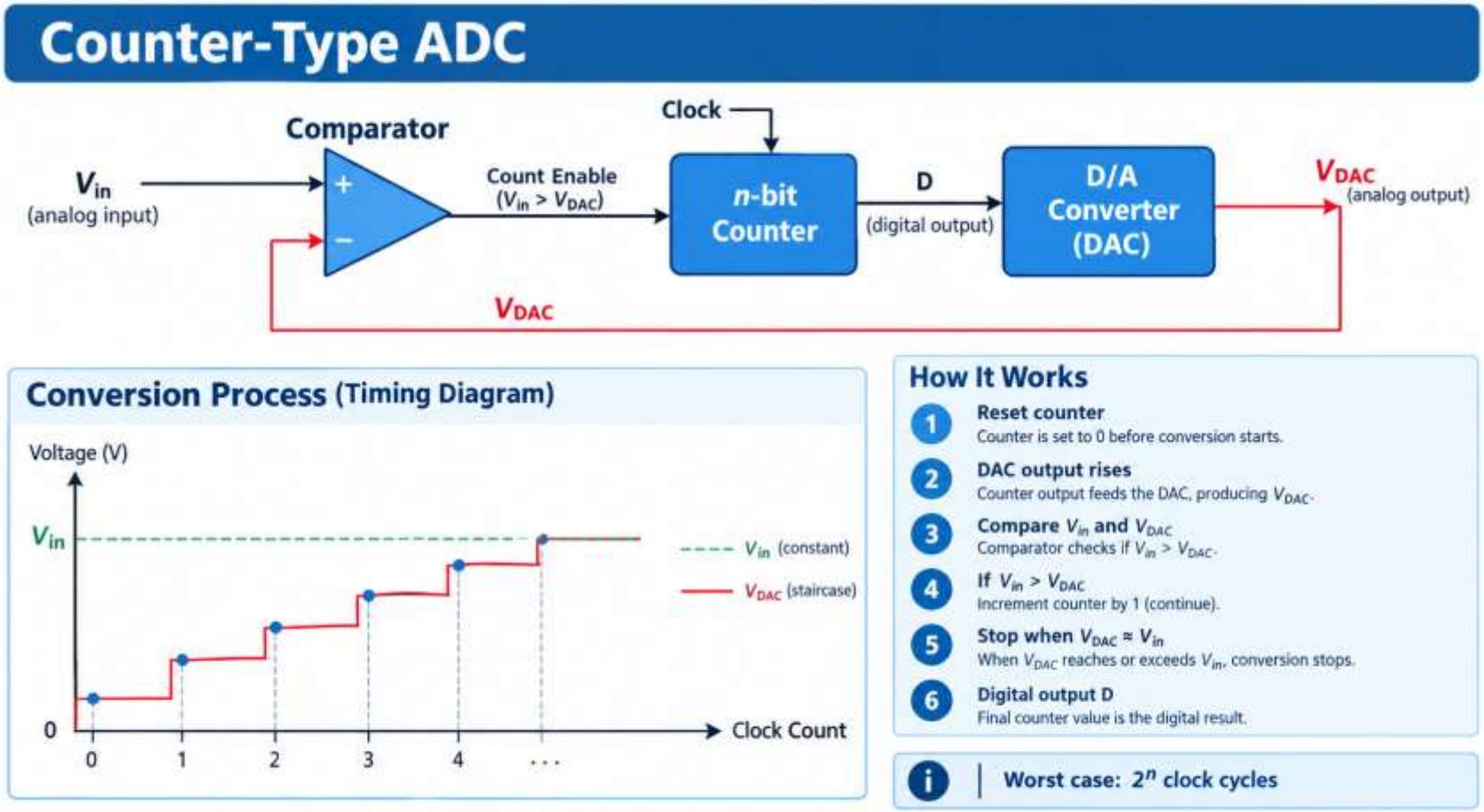
- This type of ADC provides the fastest conversion.
- Requires large amount of hardware.
  - For an n-bit converter, we require 2^n-1 comparators, 2^n resistors, and one 2^n-input priority encoder.
- We illustrate the design for a 3-bit ADC.



A	B	C	D	E	F	G	D2	D1	D0
0	0	0	0	0	0	0	**0**	**0**	**0**
0	0	0	0	0	0	1	**0**	**0**	**1**
0	0	0	0	0	1	X	**0**	**1**	**0**
0	0	0	0	1	X	X	**0**	**1**	**1**
0	0	0	1	X	X	X	**1**	**0**	**0**
0	0	1	X	X	X	X	**1**	**0**	**1**
0	1	X	X	X	X	X	**1**	**1**	**0**
1	X	X	X	X	X	X	**1**	**1**	**1**



(b) Counter-Type ADC



What was wrong with the counter method?

In a **counter-type ADC**, the counter starts at:

$$0000$$

and counts upward:

$$0000 \rightarrow 0001 \rightarrow 0010 \rightarrow 0011 \rightarrow \cdots$$

The DAC converts each digital value into a voltage, and the comparator checks:

$$V_{in} \quad \text{vs.} \quad V_{DAC}$$

The problem is that it may have to try **many values** before reaching the correct one.

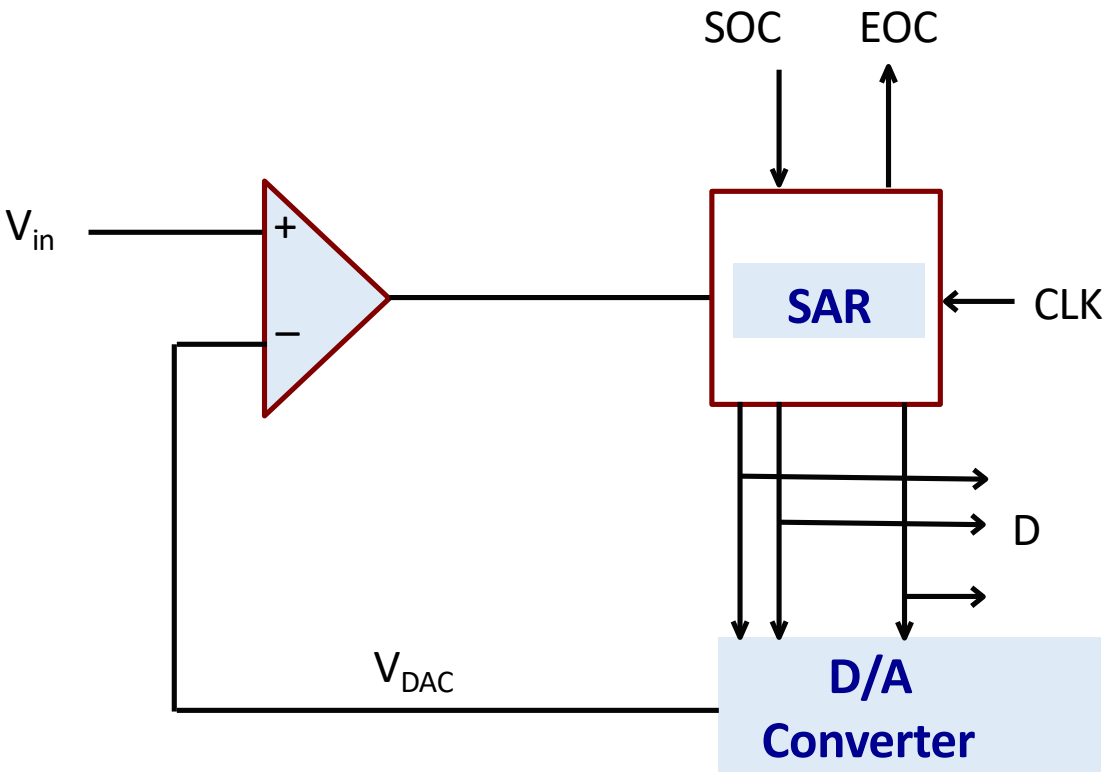
For an n-bit ADC, it can take up to:

$$2^n$$

steps.

(C) Successive Approximation Type ADC

- The principle of operation is analogous to *binary search*.
  - Suppose we are searching for a number (here V_{in}) in a sorted list.
  - We look into the middle of the list – effectively the size of the list reduces by half.
  - For 2^n steps, the number of steps required is only n.
- What modifications are required?
  - We use a successive approximation register (SAR) instead of a counter.
  - The SAR emulates binary search.
  - For example, in 4 bits, it starts with 1000 (i.e. 8).
    - If the input is smaller, the next value is set as 0100 (i.e. 4).
    - If the input is larger, the next value is set at 1100 (i.e. 12).



- SOC: Start of Conversion
- EOC: End of Conversion

Binary Search idea



(C) Successive Approximation Type ADC

The SAR ADC uses binary search instead

The modification is:

Replace the counter with a Successive Approximation Register (SAR).

The SAR does **not simply count upward.**

Instead, it tries the **middle value first**, just like binary search.

For a **4-bit ADC**, the possible values are:

$$0 \text{ to } 15$$

The middle is approximately 8, so the SAR first tries:

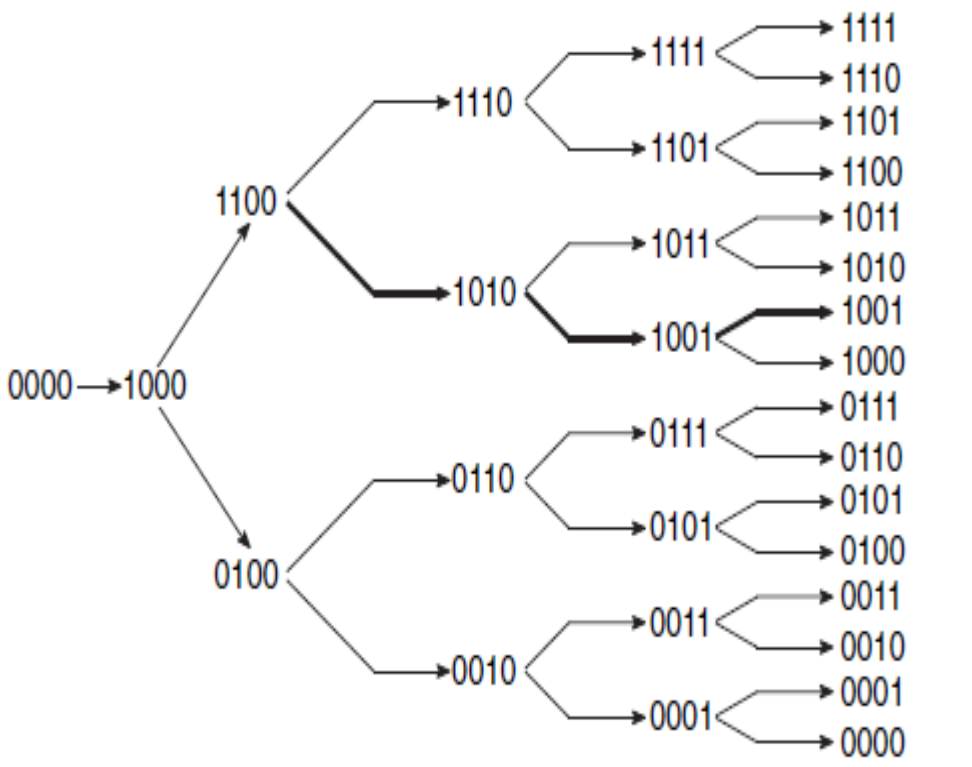
$$\boxed{1000_2 = 8}$$

The DAC converts 1000 into its corresponding analog voltage.

Then the comparator asks:

Is V_{in} greater than or less than V_{DAC}?





Conversion process in a successive approximation type A/D converter.

- An example for a 4-bit ADC, where the expected digital output is 1001.
- Only 4 comparison steps are required, once per bit position.

4-Bit SAR ADC Example

- Suppose we have a **4-bit ADC**.
  - A 4-bit ADC has:
  - **2^4 = 16 possible values**
  - The digital values range from: **0 to 15**
  - The SAR begins by setting the MSB to 1.
  - **Initial trial: 1000 = 8**
  - The DAC converts this digital value into an analog voltage.
  - The comparator then compares: **Vin with VDAC**

4-Bit SAR ADC Example

1- SAR = 1000, 1000 in binary = 8

2- The DAC converts 1000 into its corresponding analog voltage.

The comparator checks:

Is Vin greater than or less than VDAC?

If Vin < VDAC: The trial value of 8 is too high.

Therefore, we search the lower half.

The SAR clears the 8-bit position.

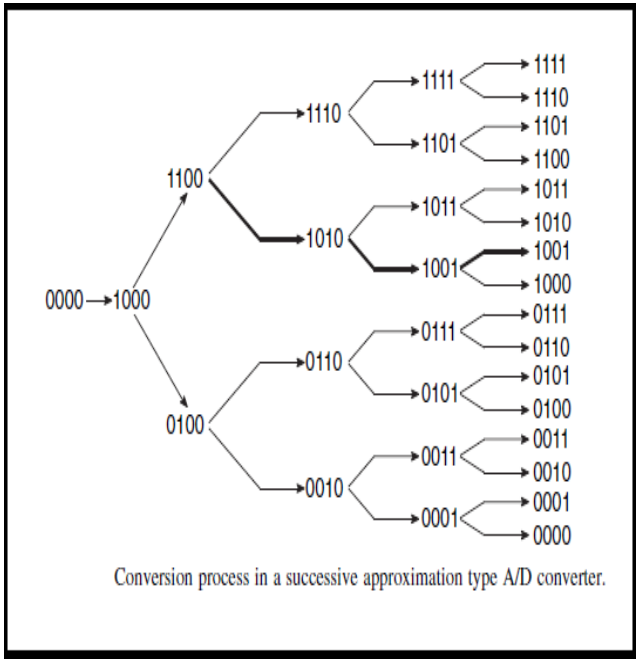
Next trial: 0100 = 4

If Vin > VDAC: The trial value of 8 is too low.

Therefore, we search the upper half.

The SAR keeps the 8-bit position and tests the next bit.

Next trial: 1100 = 12

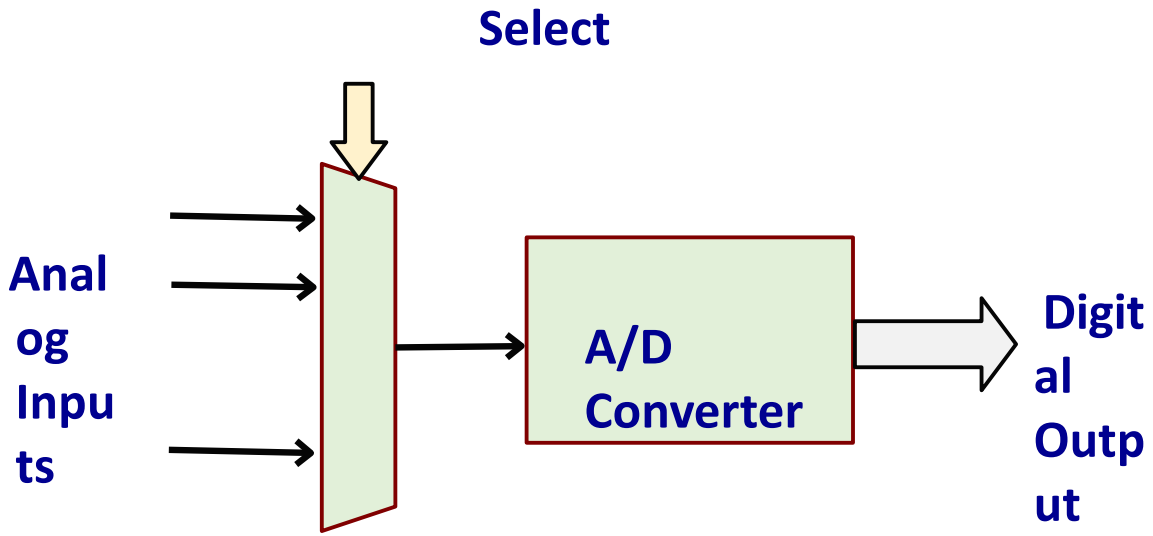


Conversion process in a successive approximation type A/D converter.

A/D Conversion in ARM Microcontrollers

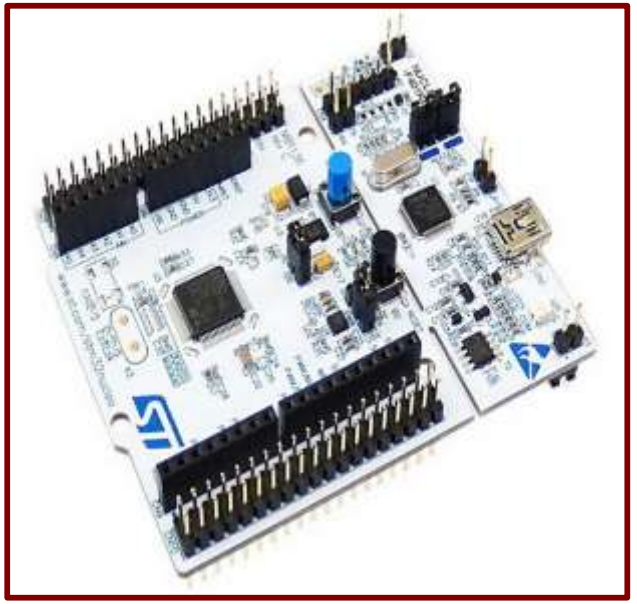
- **ADC in LPC2148 ARM7 Microcontroller**
  - Built-in 10-bit successive-approximation ADC.
  - With V_{REF} = 3.3V, step size = 3.3 / (2^{10} – 1) = 3.23 mV
  - This determines the accuracy of the measurement.
- Some specific details:
  - Two built-in ADC modules, ADC0 and ADC1.
  - ADC0 has 6 channels, and ADC1 has 8 channels.
  - ADC operating frequency is 4.5 MHz (maximum).

- There is a Control Register that must be written into to determine the operating mode.
- Interrupt is generated after A/D conversion is complete.
- An analog multiplexer is used to select one of the input analog channels at the input of an ADC.

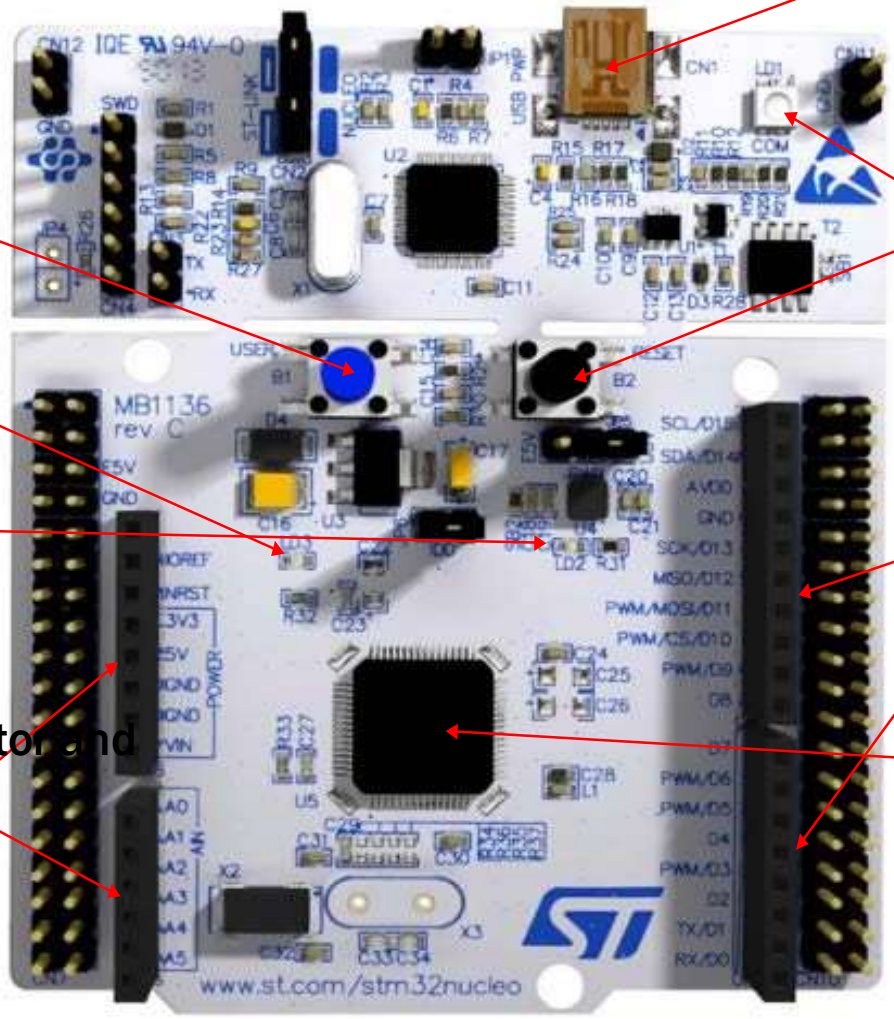


- **ADC in ARM Cortex-M4**
  - STM32F4xxx microcontrollers have up to 3 in-built 12-bit A/D converters, each of which has 19 channels.
  - With V_{REF} = 3.3V, step size = 3.3 / (2^{12} – 1) = 0.806 mV
  - There are 16 external channels, connected to I/O pins.
  - 6 of these are connected to dedicated Arduino connector pins, A0 to A5.
  - There are 3 internal channels:
    - V_{bat} : voltage on battery pin.
    - Temp sensor: used to measure difference in temperature.
    - V_{ref} : voltage reference for the A/D converter.

About the STM32F401 Nucleo Board



- Developed by ST Microelectronics.
- CPU: ATM Cortex© M4
- 512 KB Programmable Flash Memory
- 96 KB SRAM
- USB 2.0: Type A to Mini B
- Mbed Enabled (mbed.org)
- Supports a wide choice of Integrated Development Environments (IDEs) including IAR™, ARM® Keil®, GCC-based IDEs.



ST Link USB Type A to mini B

Reset Button

User Button

Red LED (power)

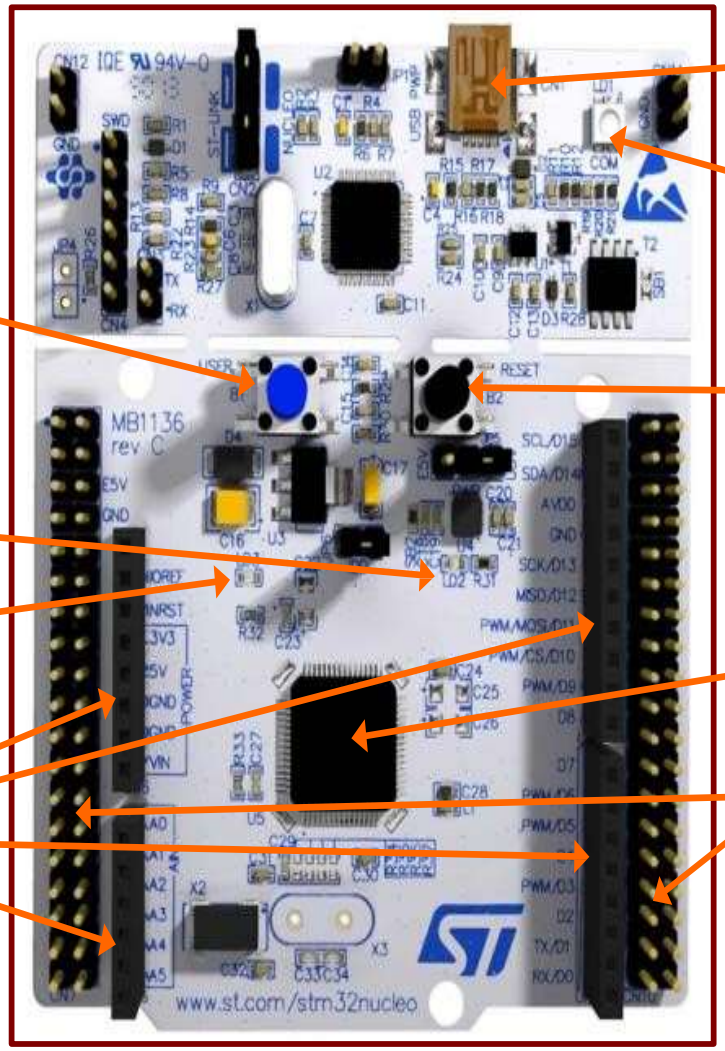
Red/Green LED (com)

Green LED (test)

Arduino Connector and Digital output

Arduino Connector and Analog input

STM32 Microcontroller


USB: Type-A to Mini-B
Red/Green LED (com)
User Button
Reset Button
Green LED (test)
Red LED (power)
STM32
Microcontroller
Port/SIgnal Connector
Arduino
Connector
MB1136
rev. C
www.st.com/stm32nucleo

Some Specific Details

- Contains STM32F401RET6 microcontroller with ARM Cortex-M4 CPU with FPU at 84 MHz clock, 512 KB of flash and 96 KB of SRAM with a variety of peripherals.
- Two types of extension connectors are available:
  - Arduino UNO Revision 3 connectivity.
  - STM Morpho extension pins for full access to all STM31F401RET6 I/Os.
- On-board ST-LINK/V2.1 interfacing with the PC, which provides:
  - Debug and programming port, to use the board with standard programmers.
  - Virtual Com port to send back traces to the PC.
  - Mass storage (USB Disk Drive) for drag-n-drop programming.

- Input/Output
  - 50 general-purpose I/O pins with external interrupt capability.
  - 12-bit ADC with 16 channels.
  - 7 general-purpose timers.
  - 2 watchdog timers.
  - USB 2.0 interface.
  - And several others …
- User LED (LD2)
- Two push buttons: USER and RESET.

Connecting the Board to the PC/Laptop

- Using the USB cable.





Understanding ADC on the ESP32

How an analog voltage becomes a digital number the microcontroller can use

1. Sensor

Light, temperature, pressure, sound, potentiometer

2. ADC

Samples voltage and converts it to a digital code

3. Software

Reads the number and makes a decision or calculation

Analog world → ESP32 ADC → digital world

How the ADC works

Think of the ADC as a measurement process.

Sampling

Take a measurement of the input voltage at a particular time.

Quantization

Map the measured voltage to one of a finite number of levels.

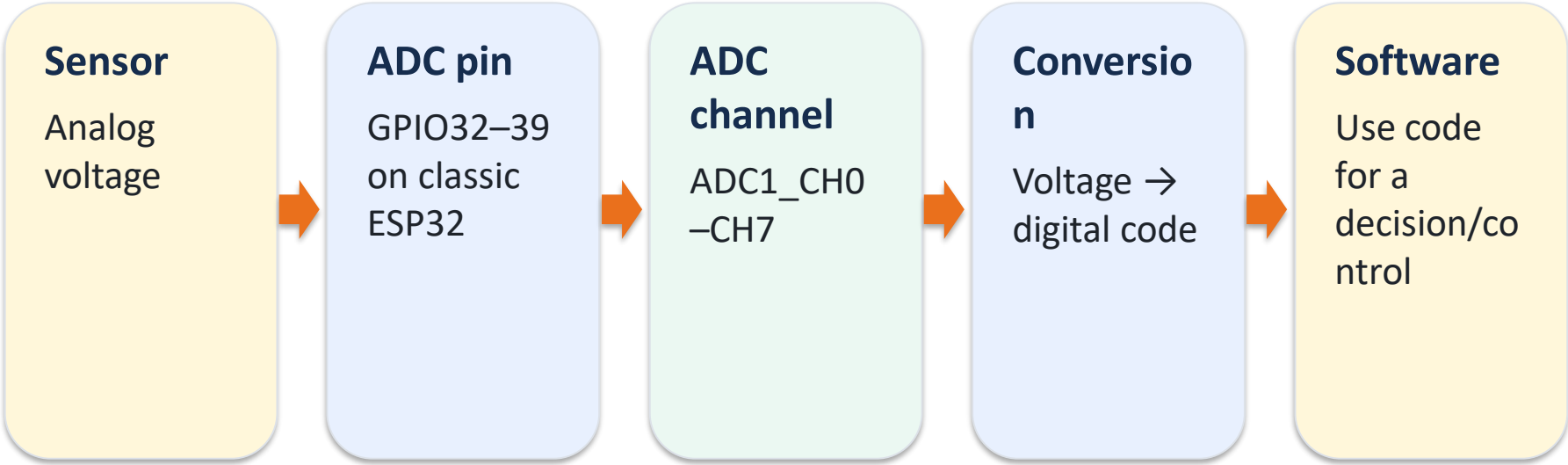
Encoding

Represent that level as a binary/digital number.

Example

An analog voltage is converted into a code such as 0, 1, 2, … 4095 when using a 12-bit result.

The ESP32 ADC in one picture



For the classic ESP32: GPIO34 = ADC1_CH6

ADC pin mappings cited from Espressif's ESP32 documentation.

Why do we need an ADC?

A microcontroller processes digital numbers, but many real-world sensors produce analog voltages.

Real world

Temperature

Light

Pressure

Sound

Position

Battery voltage

Analog signal

A voltage that can vary continuously.

Example:

0.8 V → 0.81 V → 0.82 V → …

Digital processor

The ESP32 works with numbers.

ADC converts the analog voltage into a digital code that software can read.

ESP32 ADC resolution: what does 12-bit mean?

For the classic ESP32, the ADC can provide a 12-bit raw result.

12 bits

$2^{12} = 4096$ possible codes

Code range:

0 → 4095

Low input

A lower input voltage produces a lower ADC code.

Conceptually:

0 V → near 0

Higher input

A higher input voltage produces a higher ADC code.

The exact voltage-to-code relationship depends on ADC configuration and calibration.

Important

Do not assume that every ESP32 ADC reading maps perfectly to voltage without calibration; ADC behavior is not perfectly ideal.

Classic ESP32 ADC pins — ADC1

For the original/classic ESP32, ADC1 has eight channels.

ADC channel	GPIO pin	Notes
ADC1_CH0	GPIO36	Input only
ADC1_CH1	GPIO37	Input only
ADC1_CH2	GPIO38	Input only
ADC1_CH3	GPIO39	Input only
ADC1_CH4	GPIO32	ADC input
ADC1_CH5	GPIO33	ADC input
ADC1_CH6	GPIO34	Input only
ADC1_CH7	GPIO35	Input only

Teaching shortcut

For the class experiment: GPIO34 = ADC1_CHANNEL_6.

ADC2 pins and an important ESP32 rule

The classic ESP32 also has ADC2, but ADC2 has a major limitation.

ADC2 channels

ADC2_CH0 → GPIO4

ADC2_CH1 → GPIO0

ADC2_CH2 → GPIO2

ADC2_CH3 → GPIO15

ADC2_CH4 → GPIO13

ADC2_CH5 → GPIO12

ADC2_CH6 → GPIO14

ADC2_CH7 → GPIO27

ADC2_CH8 → GPIO25

ADC2_CH9 → GPIO26

Why ADC1 is often preferred

On the classic ESP32, ADC2 cannot be used when Wi-Fi is being used.

So for Wi-Fi + analog sensing, choose an ADC1 pin when possible.

This is one reason GPIO32–39 are very useful for analog sensors.

Rule of thumb

Wi-Fi + ADC? Prefer ADC1 on the classic ESP32.

What does “GPIO34” really mean?

The same physical pin can have multiple functions; the ADC function selects the analog path.

GPIO34

Physical GPIO pin

Input only on classic ESP32

No software pull-up / pull-down

ADC1_CHANNEL_6

The ADC channel connected to GPIO34.

Software selects the ADC channel rather than treating it like a normal digital GPIO.

Sensor voltage

Connect the sensor's analog output here.

Example:

LDR → voltage → GPIO34 → ADC1_CH6 → digital code

ADC configuration: the important settings

When software initializes the ADC, it tells the ESP32 how to measure.

ADC unit / channel

Select which ADC is used and which channel.

Example:

ADC Unit 1

ADC1_CHANNEL_6

GPIO34 is connected to ADC1_CH6.

Bit width

Determines how many digital levels are available.

12-bit:

4096 levels

0–4095

More bits → finer digital resolution.

Attenuation

Changes the input range the ADC can handle.

Higher attenuation extends the measurable input range, but the exact usable range depends on the ESP32 ADC characteristics.

ESP-IDF: reading an ADC value

Conceptually, the program does four things.

```
1. Create / initialize the ADC unit
2. Configure the ADC channel
3. Read one conversion
4. Use the returned number

Example:

int adc_value = 0;
adc_oneshot_read(adc1_handle,
                 ADC_CHANNEL_6,
                 &adc_value);

printf("ADC = %d\n", adc_value);
```

Then what?

The number can be used to:

- Turn an LED on/off
- Display a value
- Trigger an alarm
- Control a motor
- Calculate an estimated voltage
- Feed a control algorithm

Example: potentiometer → ESP32 ADC → LED

A potentiometer makes the ADC concept easy to visualize.

1. Turn knob

Potentiometer changes the output voltage.

2. ADC input

Voltage enters an ADC1 pin such as GPIO34.

3. Digital code

ESP32 produces a number from the ADC.

4. Decision

Software compares the number and controls the LED.

Key idea

The ADC does not “understand” light, temperature, or position. It only measures an electrical voltage and produces a digital representation of that voltage.

ADC design considerations

A good ADC measurement is more than just calling a read function.

- Choose an ADC-capable GPIO pin that fits the ESP32 variant you are using.
- Keep the input voltage within the ADC's allowed range.
- Choose an appropriate attenuation setting.
- Consider noise, filtering, source impedance, and wiring.
- Use calibration when you need a more accurate voltage estimate.
- Avoid ADC2 on the classic ESP32 when Wi-Fi is active; use ADC1 instead.

Embedded Systems Lecture 7



Instructor: Fathi Amsaad

Credits: Some slides are adopted from Henk Corporaal's Computer Architecture and Organization course

and also from CS232: Computer Architecture II. University of Illinois at Urbana-Champaign and Tamzidul Hoque, Assistant Professor, Dept. of EECS, University of Kansas (hoque@ku.edu), office: Eaton 2038

Processing Unit

Functional units

- ALU = Arithmetic and Logic Unit
- E.g., ADD, SUB, Not, XOR, Multiply, Shift, Compare, …

Registers

- Small temporary storage
- Operands and results of functional units
- RISC-V and MIPS32 has 32 registers

Registers

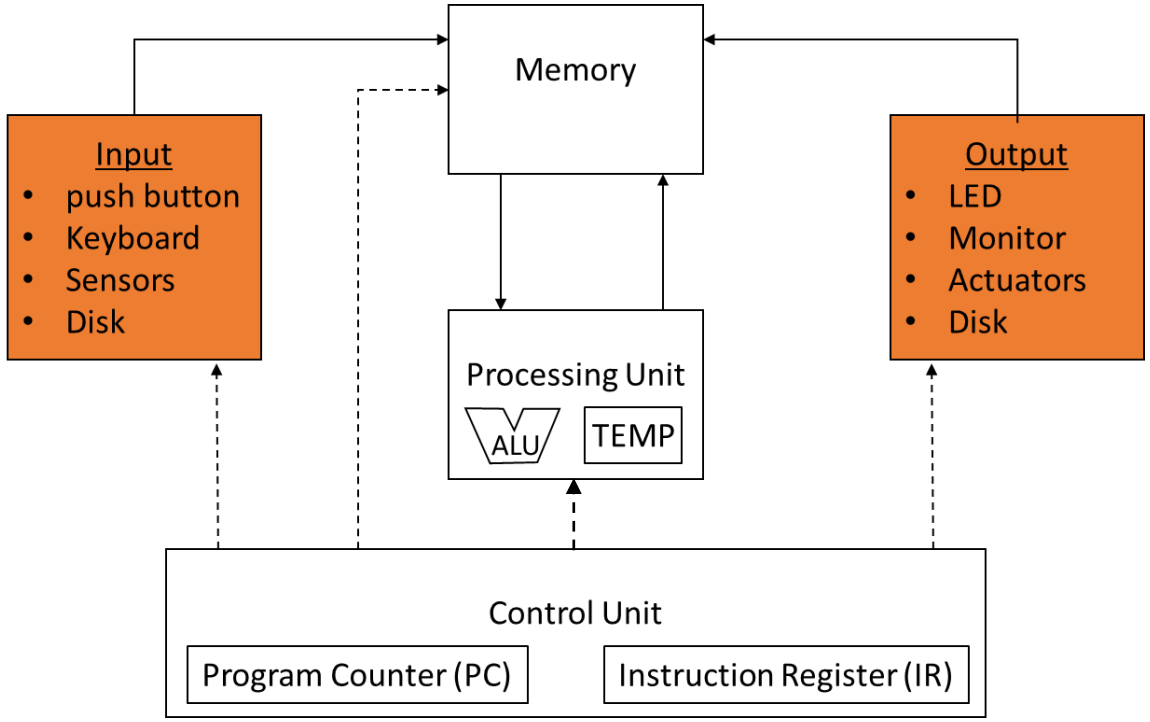
Registers are fast but small

- Memory is large but slow

Register file

- A structure in CPU that holds all the registers
- r0-r31 in MIPS

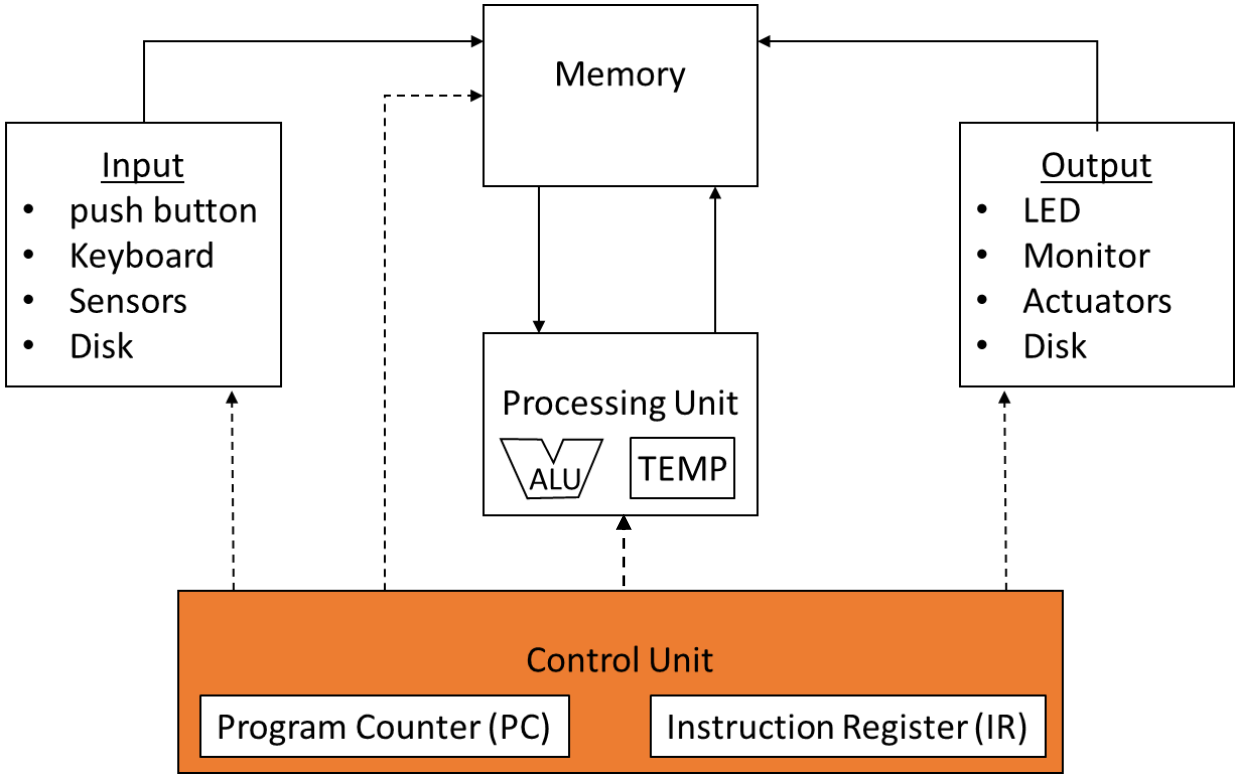
Input/Output



The Von Neumann computer model

- Input and output devices (i.e., peripherals) are used for getting data into and out of computer memory
- Usually each device has a set of registers to communicate with the CPU

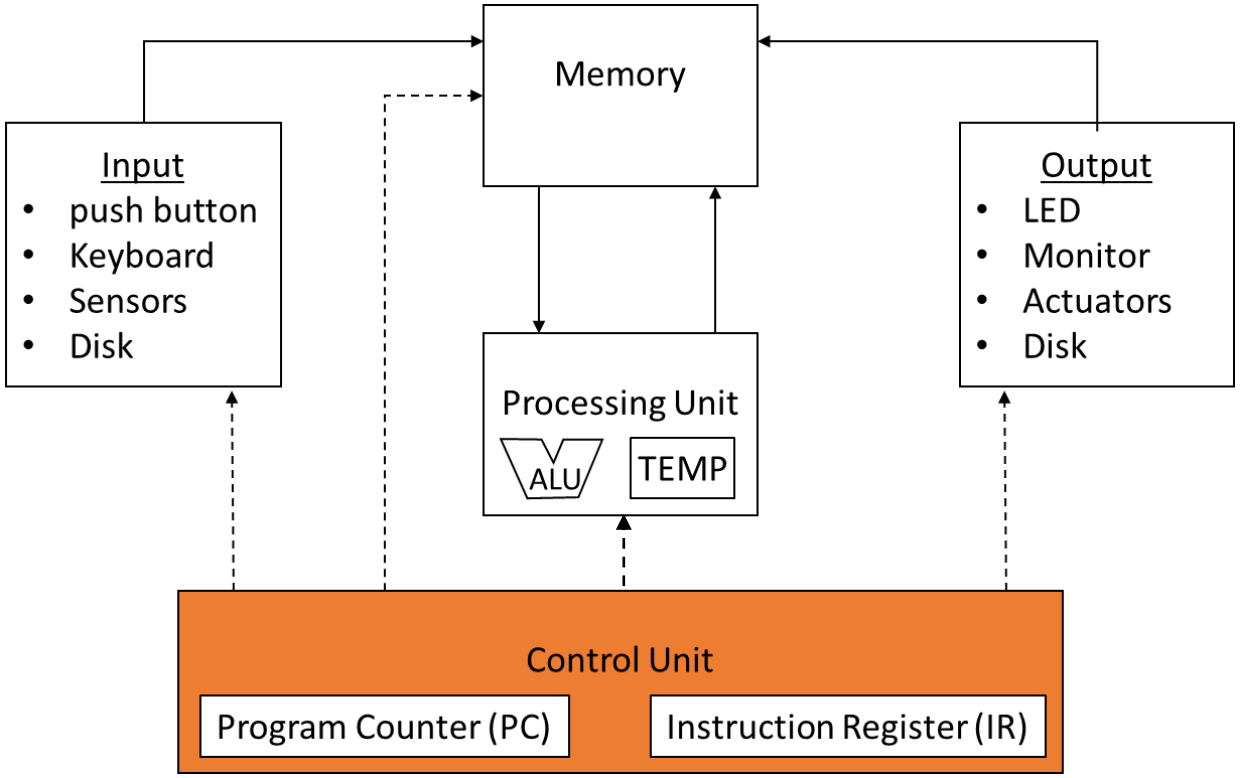
Control Unit



The Von Neumann computer model

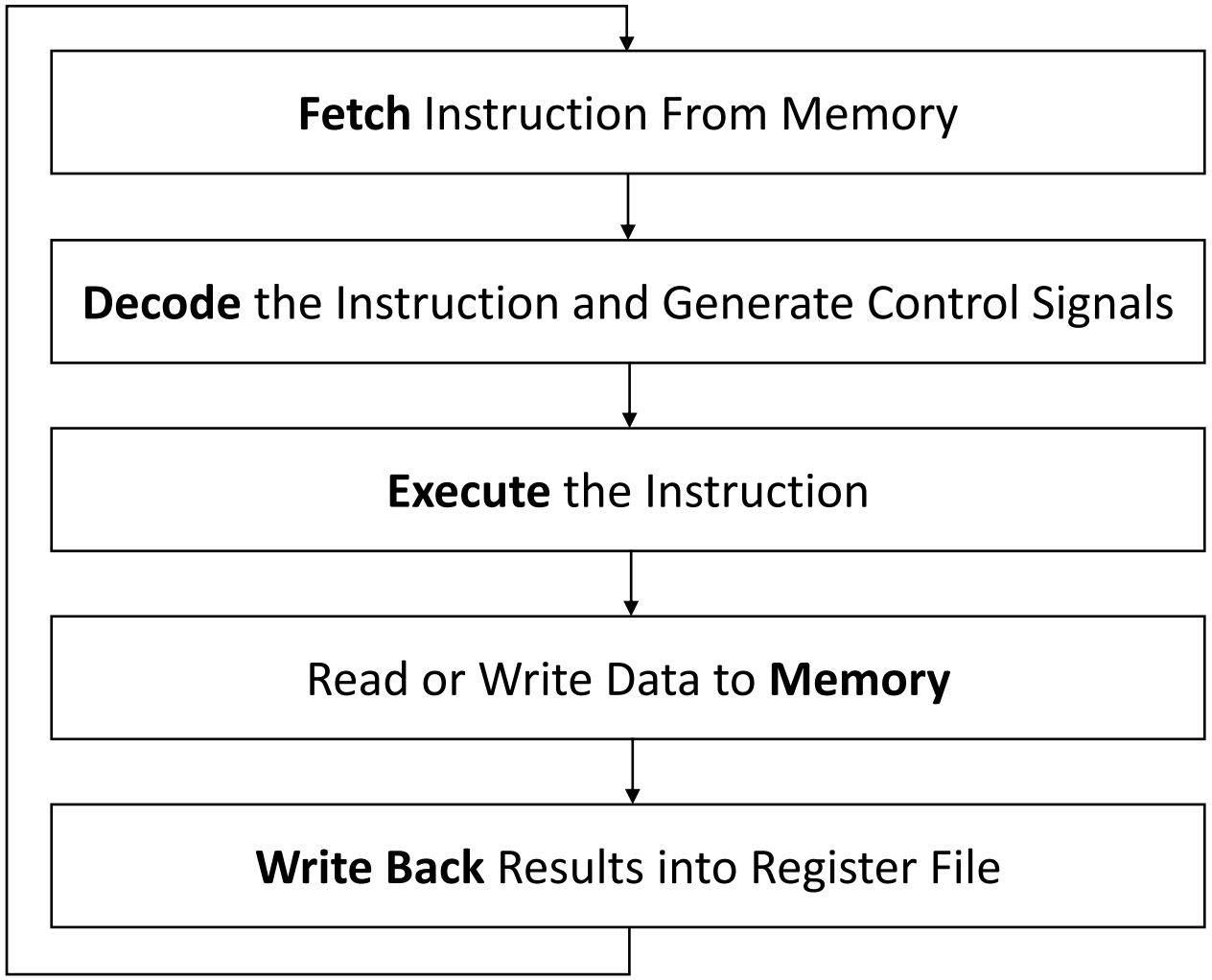
- Control unit orchestrates the execution of each instruction
- Instruction Register (IR): the current instruction being executed on the CPU
- Program Counter (PC): the address of the next instruction to be executed

Function of Control Unit



- Read an instruction from memory into Instruction Register (IR)
  - By loading the memory address pointed by PC
- Decodes the instruction and generate control signals that tell other components what to do

Instruction Processing



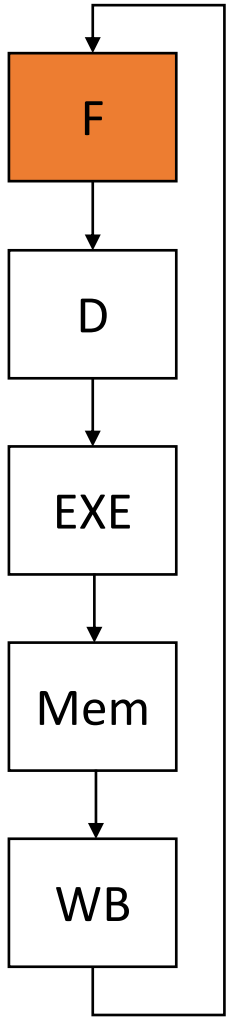
Instruction

Smallest unit of execution in a processor

- Carries two pieces of information
  - Opcode: operation to be performed
  - Operands: data to be used for operation
    - Registers, immediate values, memory addresses
- Instructions are sequences of bits
- Instruction Set Architecture (ISA) is the collection of all instructions that a processor supports

Instruction Processing: Fetch

- Read next instruction (pointed by PC) from memory into IR
- Increment PC (so that it points to the next instruction in memory)
- Question: How much should you increment the PC after fetch stage in a processor that is **byte addressable** and the width of each instruction is **8 Bytes**?



Instruction in memory

PC ⇨ **Add R1, R2, R3** **0x23bdffe0**

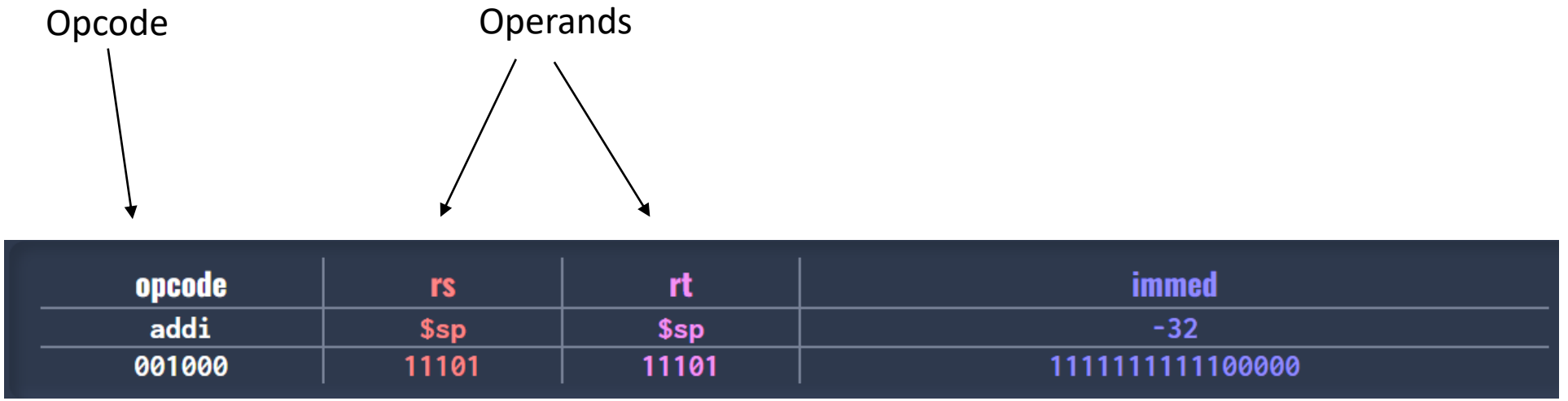
sub R5, 24(10) **0xafb00018**

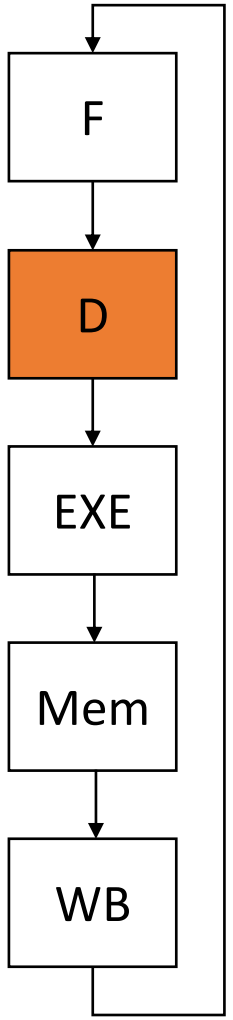
- Q1: What is the value of PC before any after each instruction is fetched?

Machine code in Main Memory	
Address	**Data / Instructions**
0x00	0x23
0x01	0xbd
0x02	0xff
0x03	0xe0
0x04	0xaf
0x05	0xb0
0x06	0x00
0x07	0x18
0x08	....

Instruction Processing: Decode

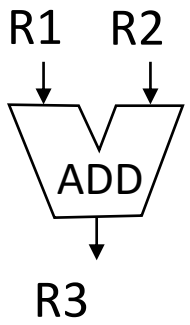
1. Identify the opcode and operands
2. Generate control signal to perform the tasks

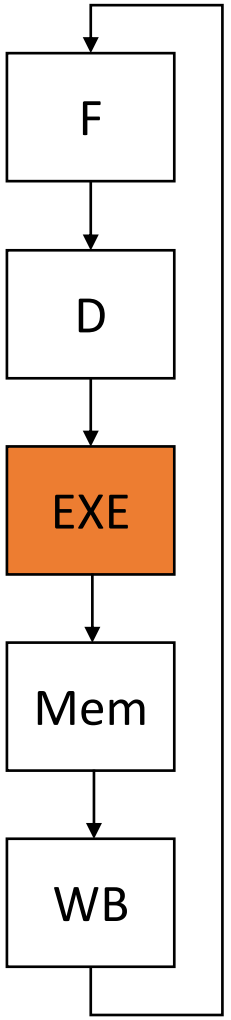




Instruction Processing: Execute

- Perform the operation using the source operands
- E.g., **add R1, R2, R3**



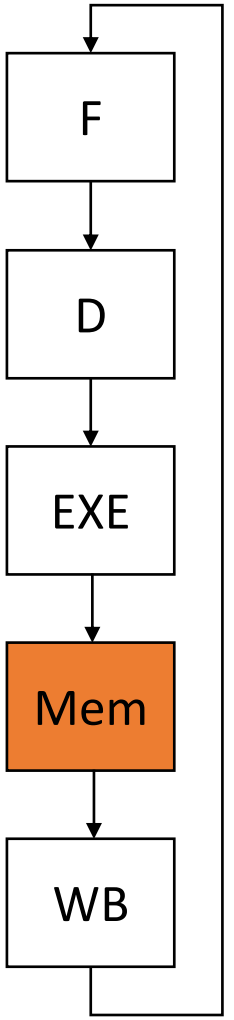


Instruction Processing: Memory Operations

- If the instruction is a load or store, then perform loading from or storing to memory

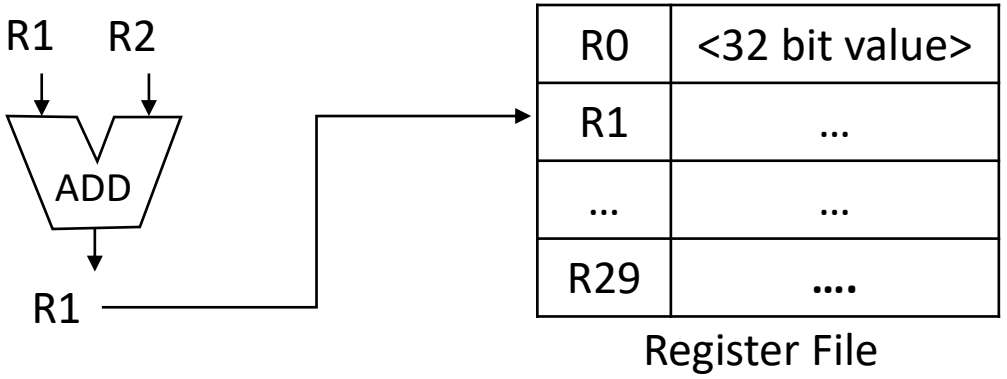
```
sw R1, R1(200)
```

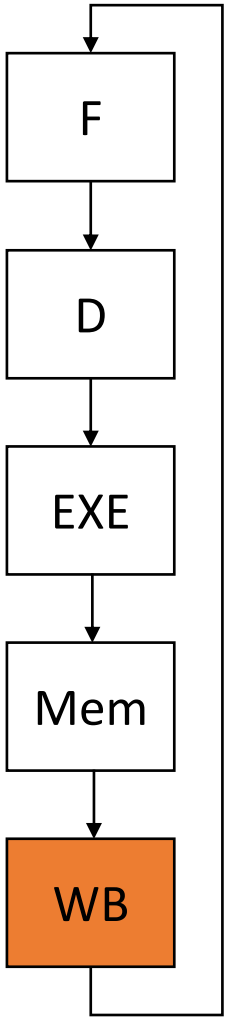
- Load/store also involves ALU operation in EXE stage. What ALU operation is that?



Instruction Processing: Write Back

- Update the register file with newly calculated results
- E.g., **Add R1, R2, R1**
- Here, R1 register needs update after the the Add is performed



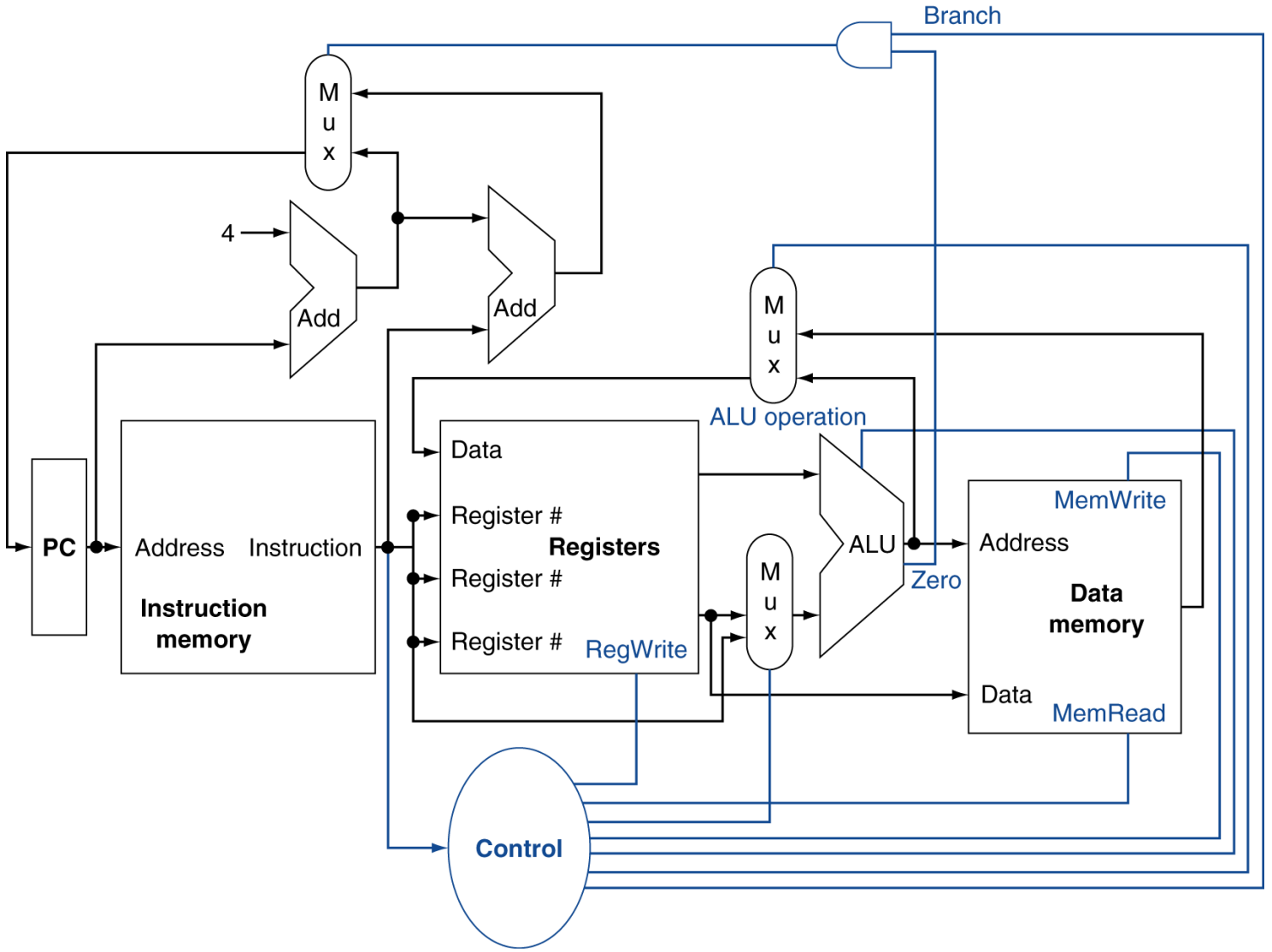


Updating PC for Jump instruction

But we do not always fetch from the next address!

- Control flow instructions (JAL, BEQ, etc.) set the PC value to something other than PC + 4
- E.g., JAL rd, imm_{20}
- Semantics
  - target_addr = PC + imm
  - GPR[rd] ← PC + 4
  - **PC ← target_addr**

Illustration of CPU Datapath for MIPS

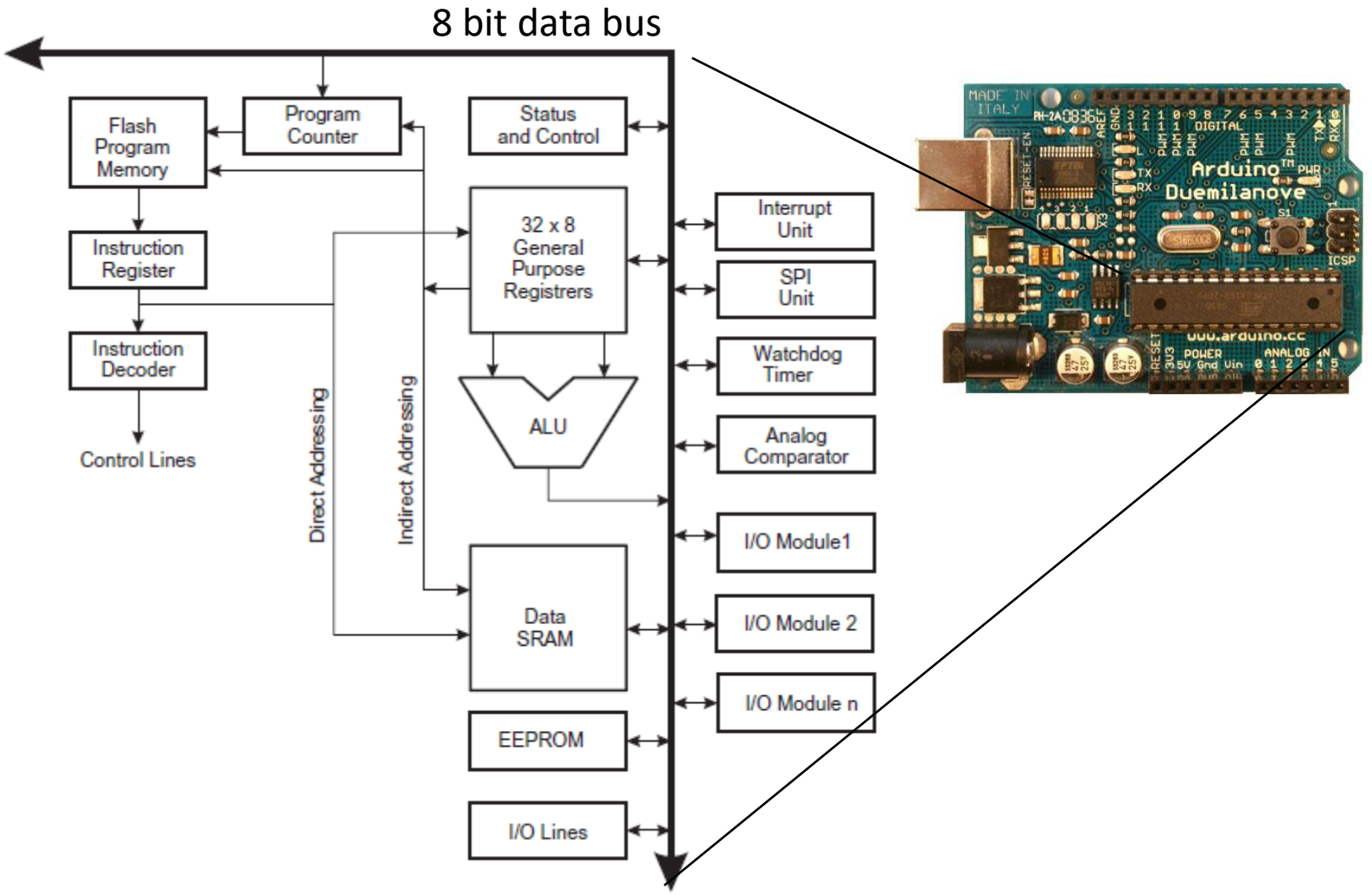


How this architecture operates for:

- Fetch
- Decode
- Execute
- Memory
- Write Back

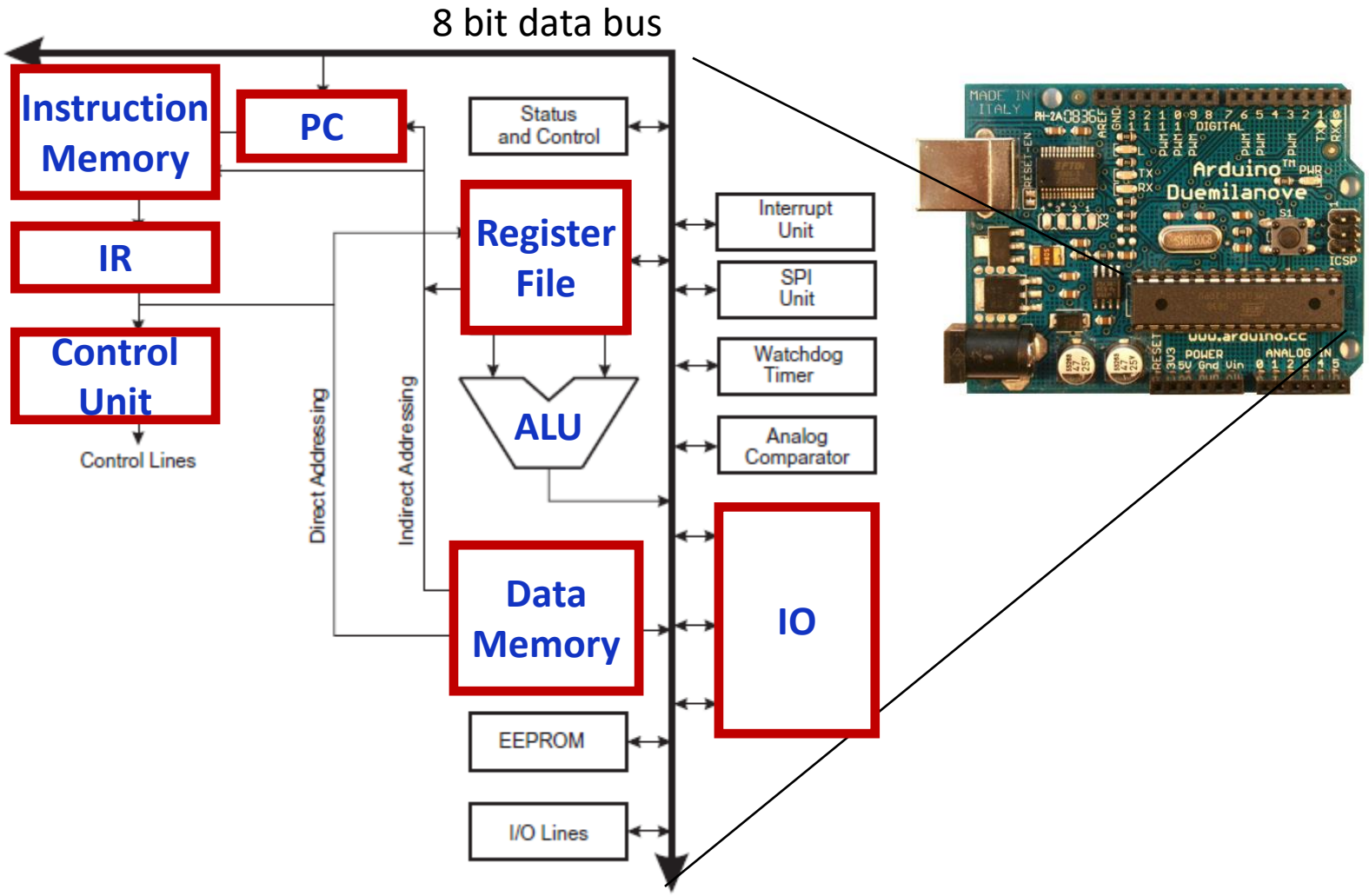
Figure from Chapter 5 of Textbook: Computer Organization and Design, Revised Printing, Third Edition

Case Study: Atmel ATmega168



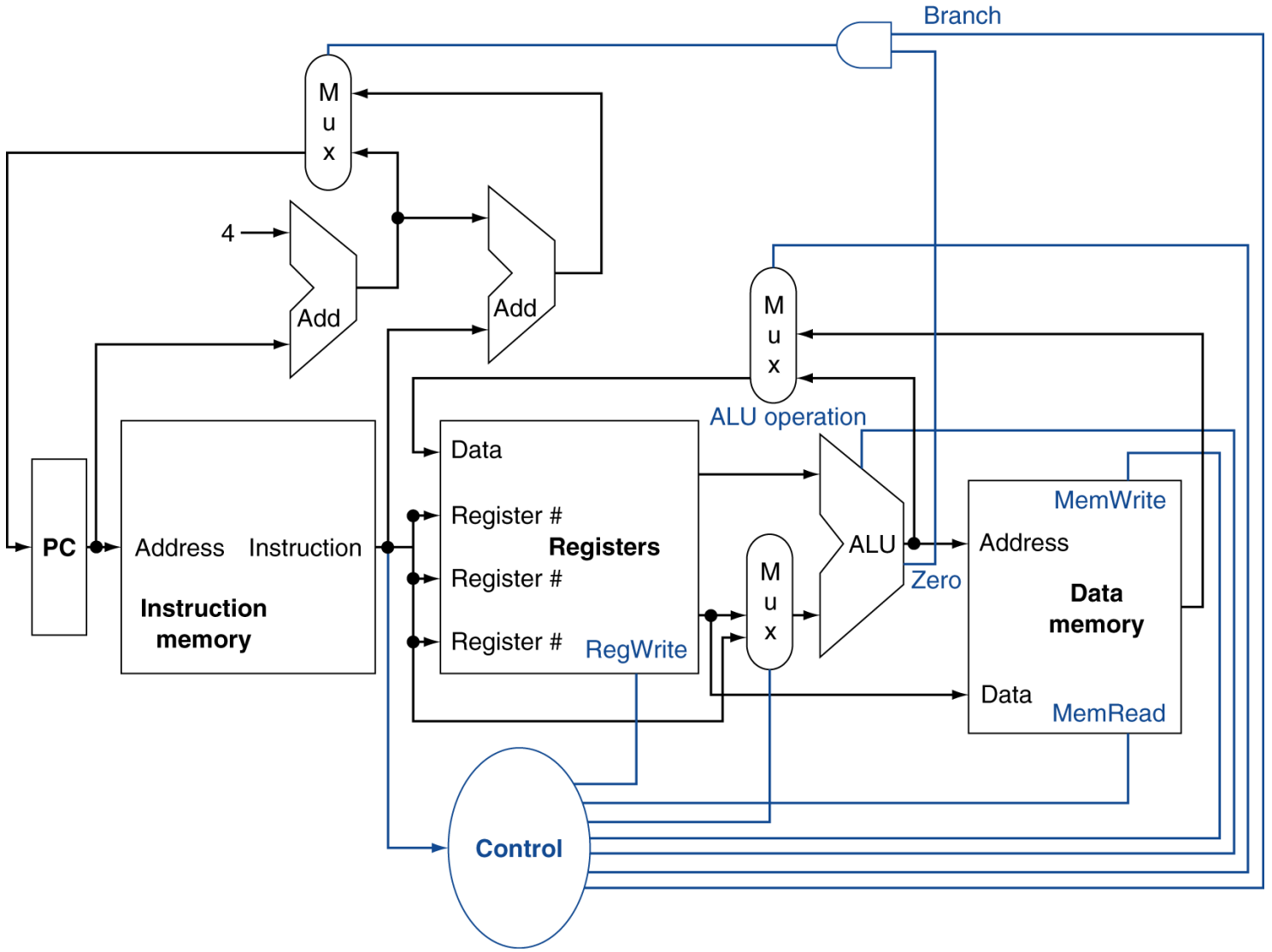
Atmel ATmega168 datasheet

Case Study: Atmel ATmega168



Atmel ATmega168 datasheet

Illustration of CPU Datapath for MIPS

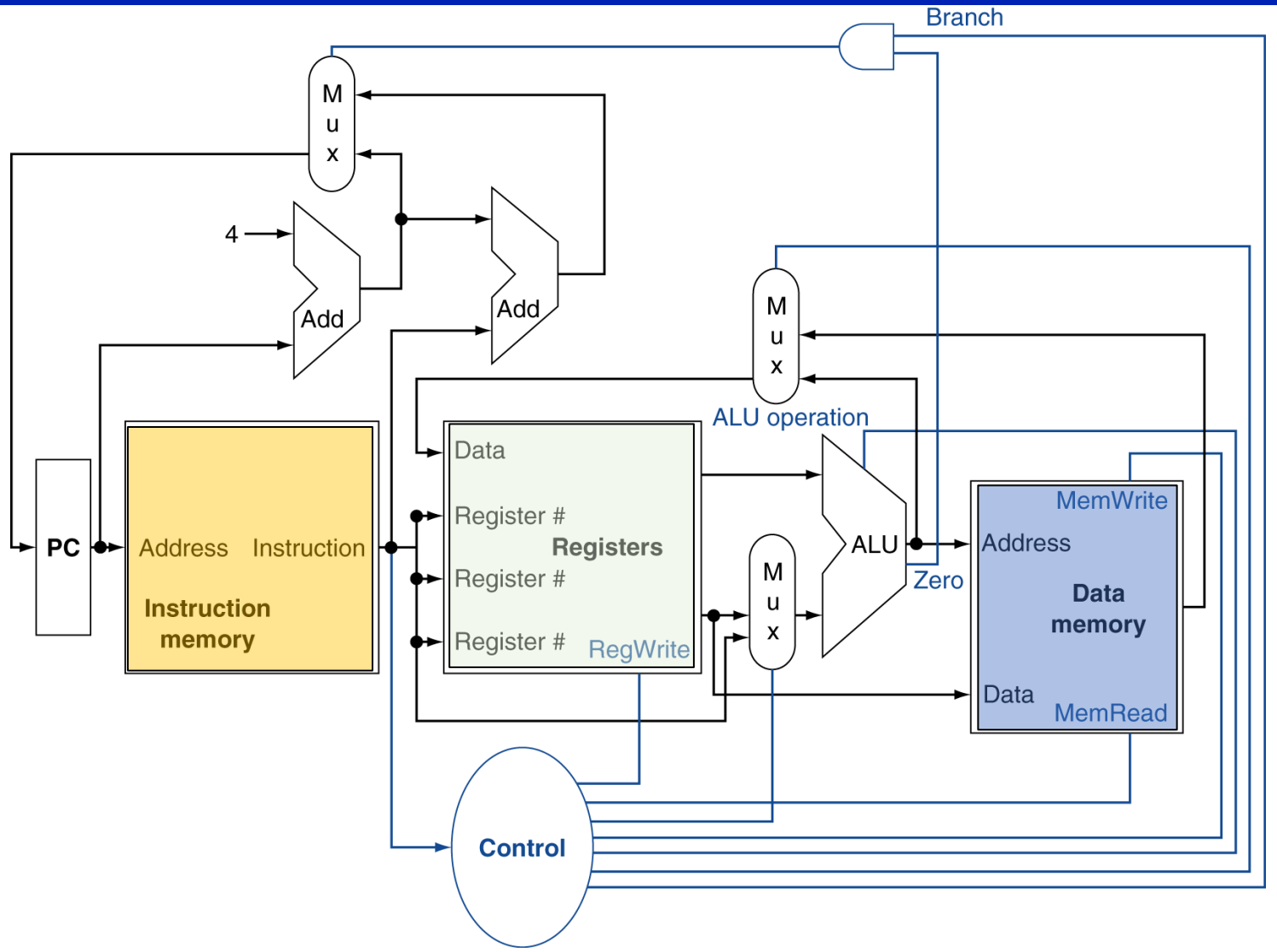


How this architecture operates for:

- Fetch
- Decode
- Execute
- Memory
- Write Back

Figure from Chapter 5 of Textbook: Computer Organization and Design, Revised Printing, Third Edition

Initial State



Register

$t0	0
$t1	40
$t2	60
$s0	200
$s1	
$s2	

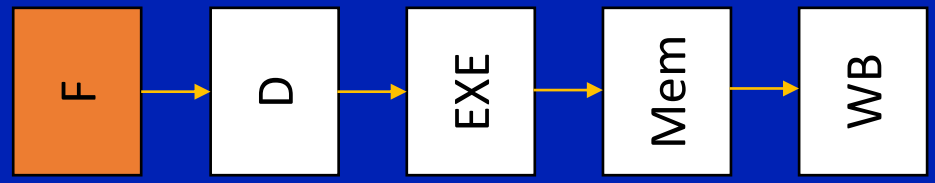
Inst. Memory

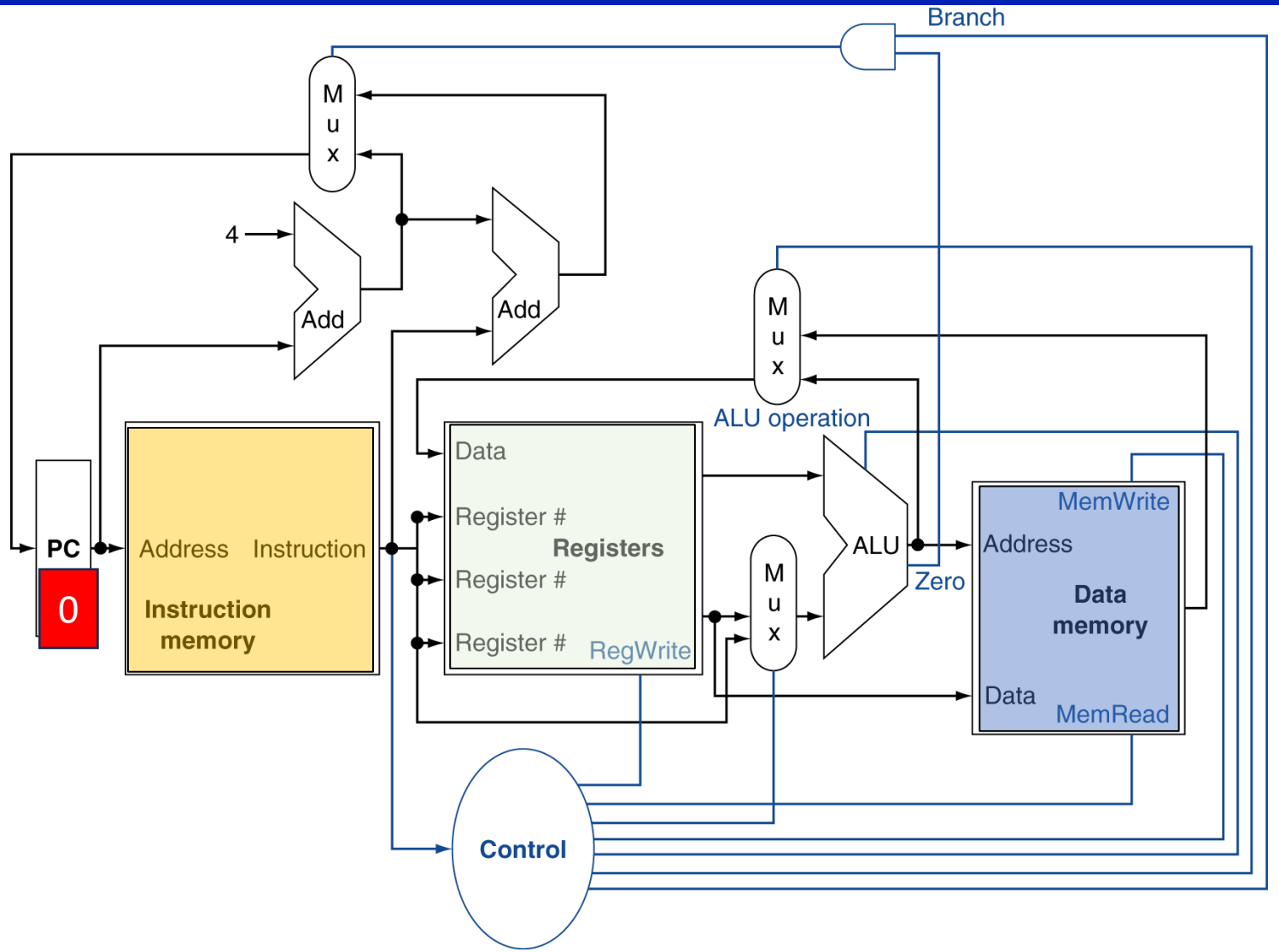
Address	Data
0	add $t0, $t1, $t2
4	sw $s0, 4($t1)
8	beq $s1, $s2, 24

Data Memory

Address	Data
40	0
44	0
48	0

Fetch





Register

$t0	0
$t1	40
$t2	60
$s0	200
$s1	
$s2	

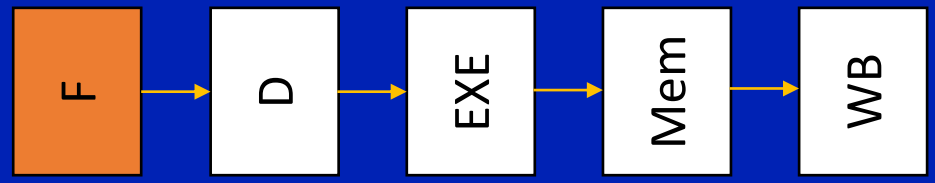
Inst. Memory

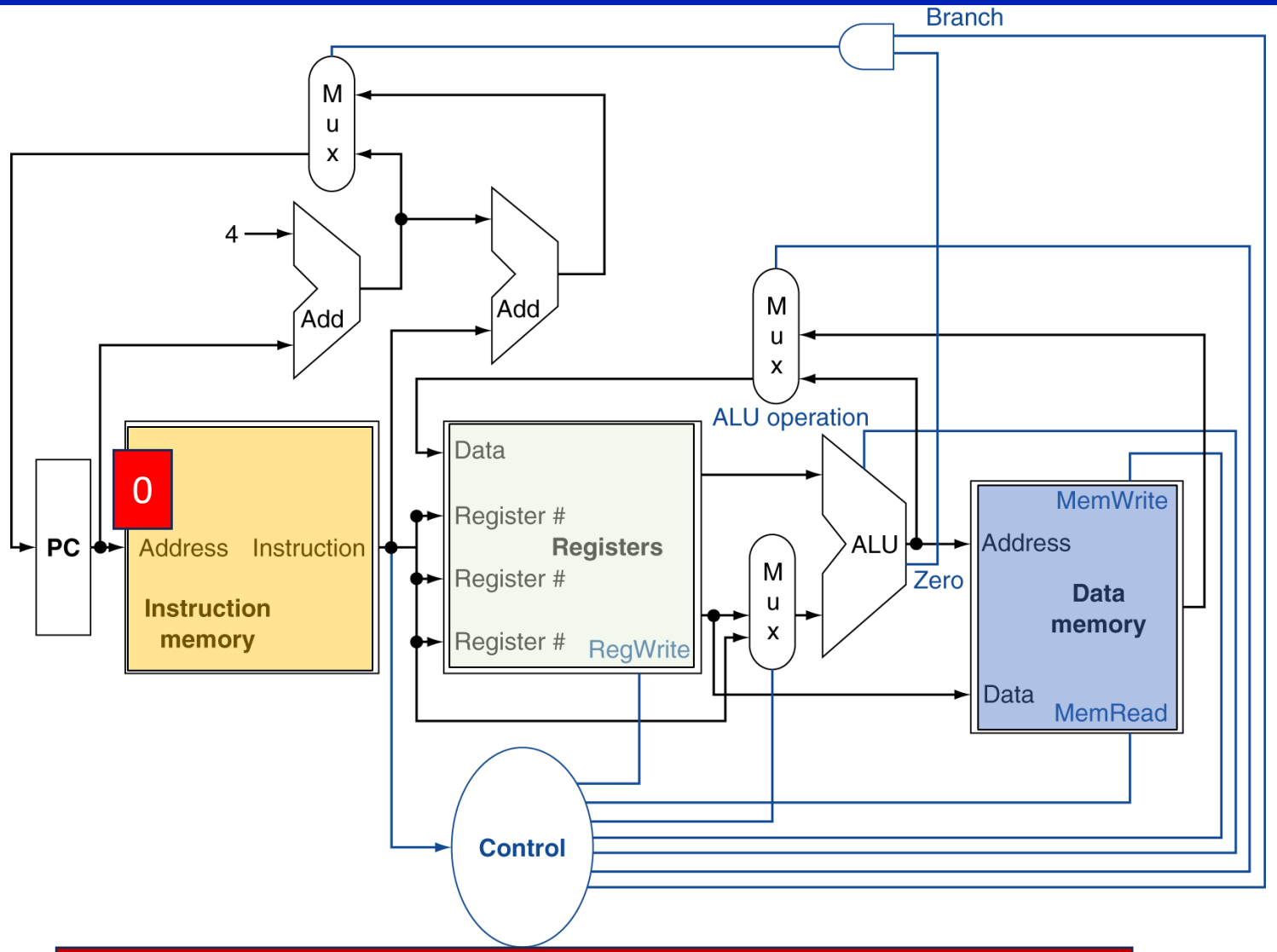
Address	Data
0	add $t0, $t1, $t2
4	sw $s0, 4($t1)
8	beq $s1, $s2, 24

Data Memory

Address	Data
40	0
44	0
48	0

Fetch:





Program counter starts from 0

Register

$t0	0
$t1	40
$t2	60
$s0	200
$s1	
$s2	

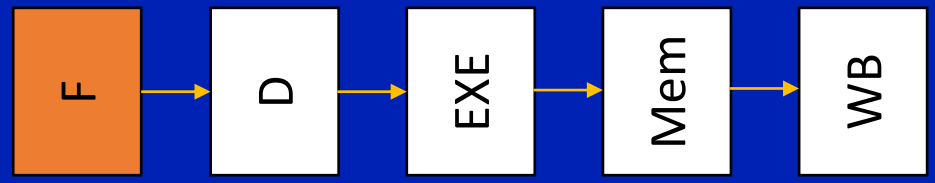
Inst. Memory

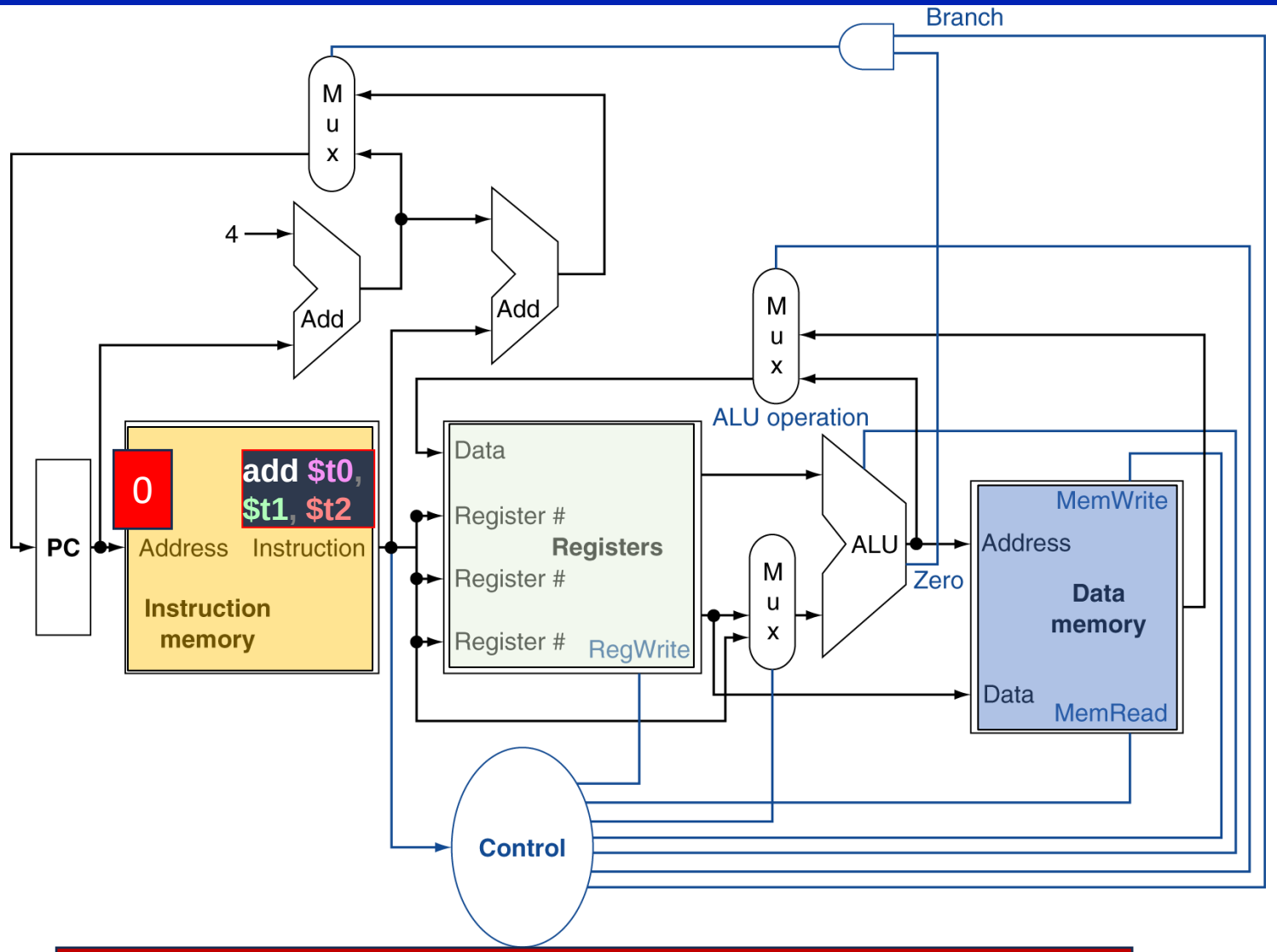
Address	Data
0	add $t0, $t1, $t2
4	sw $s0, 4($t1)
8	beq $s1, $s2, 24

Data Memory

Address	Data
40	0
44	0
48	0

Fetch





Register	
$t0	0
$t1	40
$t2	60
$s0	200
$s1	
$s2	

Inst. Memory

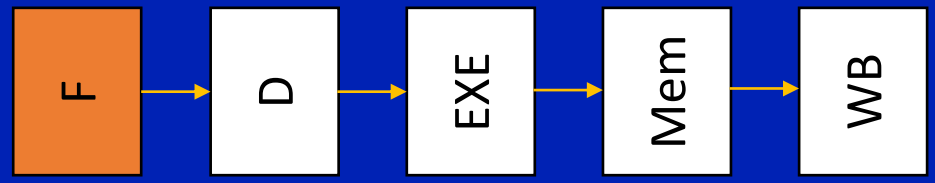
Address	Data
0	add $t0, $t1, $t2
4	sw $s0, 4($t1)
8	beq $s1, $s2, 24

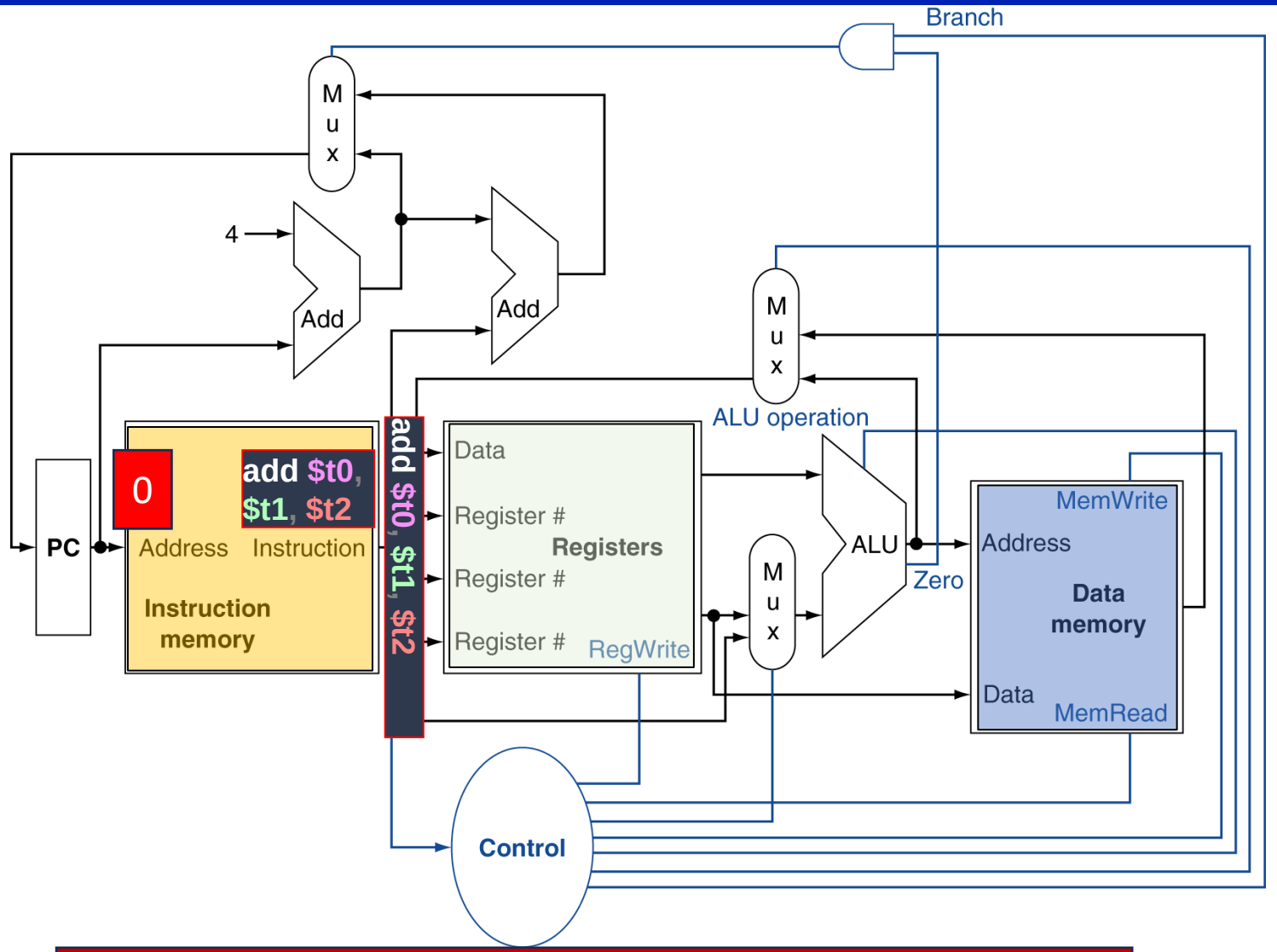
Data Memory

Address	Data
40	0
44	0
48	0

Inst mem gets 0 as input, outputs the content at address 0

Fetch





Content at address 0 is the first instruction

Register

$t0	0
$t1	40
$t2	60
$s0	200
$s1	
$s2	

Inst. Memory

Address	Data
0	add $t0, $t1, $t2
4	sw $s0, 4($t1)
8	beq $s1, $s2, 24

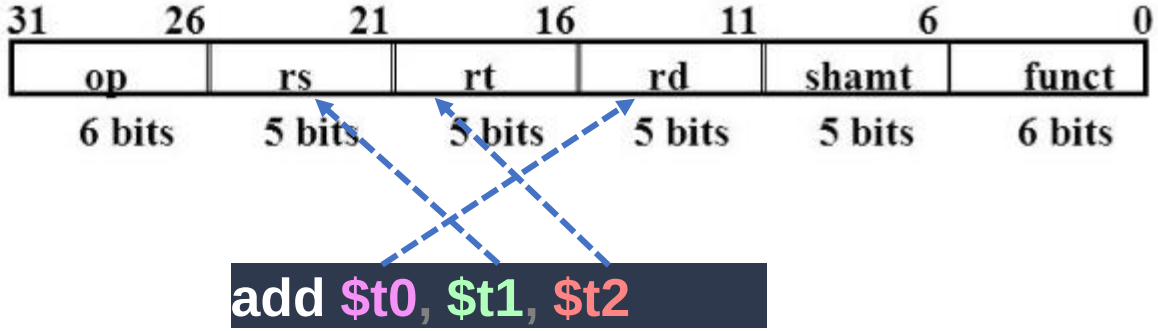
Data Memory

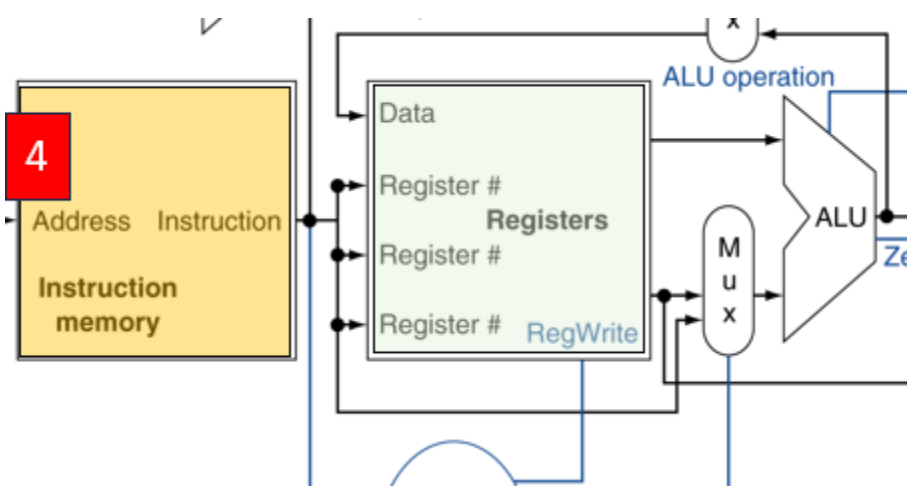
Address	Data
40	0
44	0
48	0

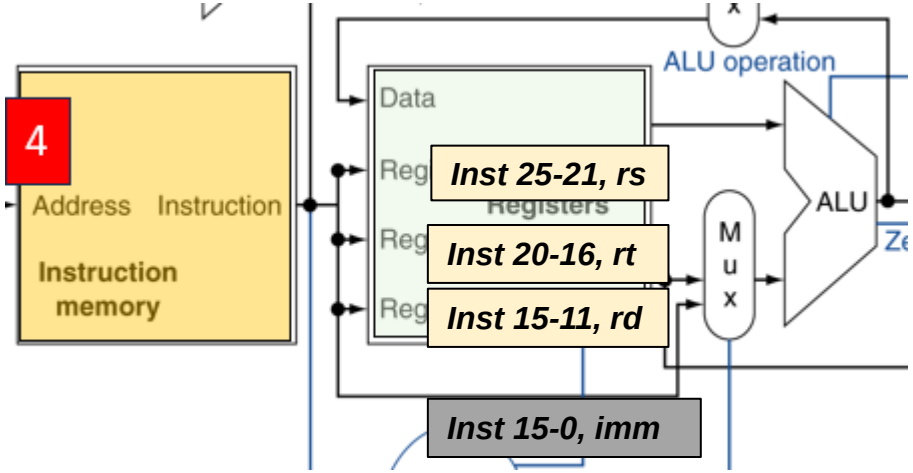
Decode for Add

R-Type:

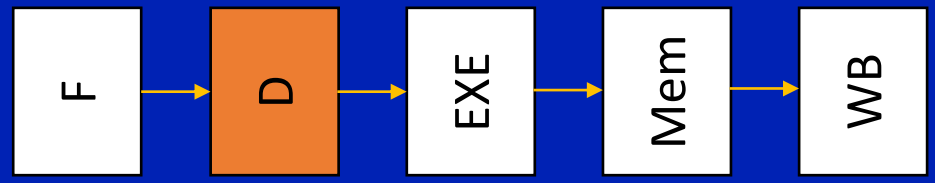
- add rd, rs, rt
- sub rd, rs, rt
- and rd, rs, rt
- or rd, rs, rt
- slt rd, rs, rt

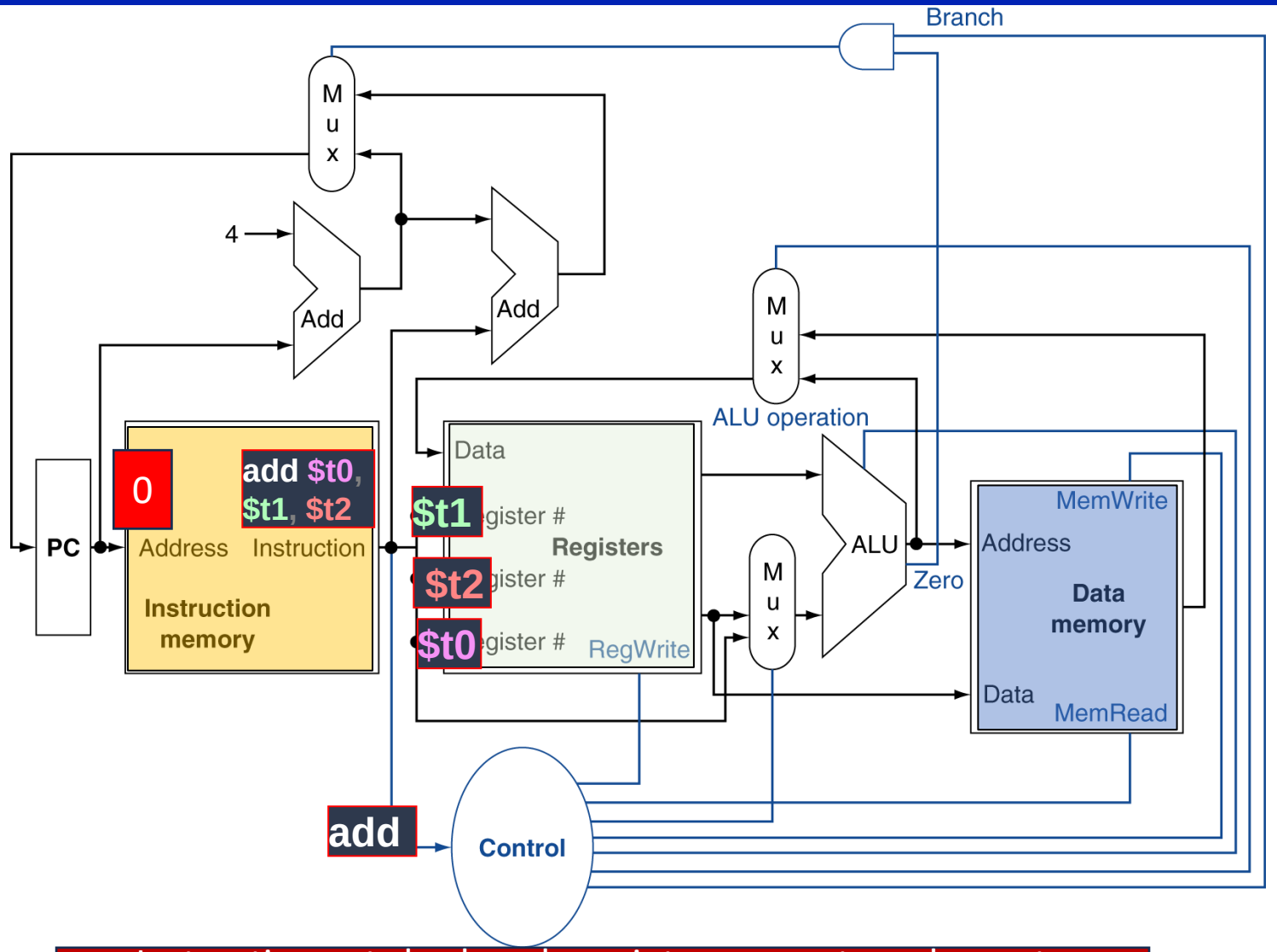






Decode





Register	
$t0	0
$t1	40
$t2	60
$s0	200
$s1	
$s2	

Inst. Memory

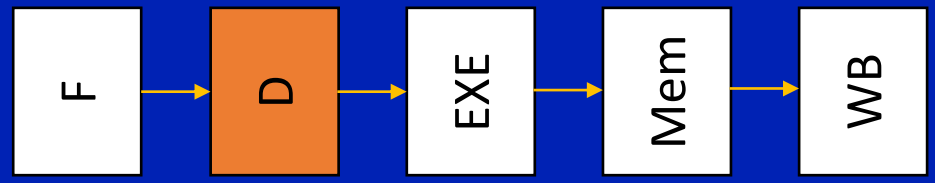
Address	Data
0	add $t0, $t1, $t2
4	sw $s0, 4($t1)
8	beq $s1, $s2, 24

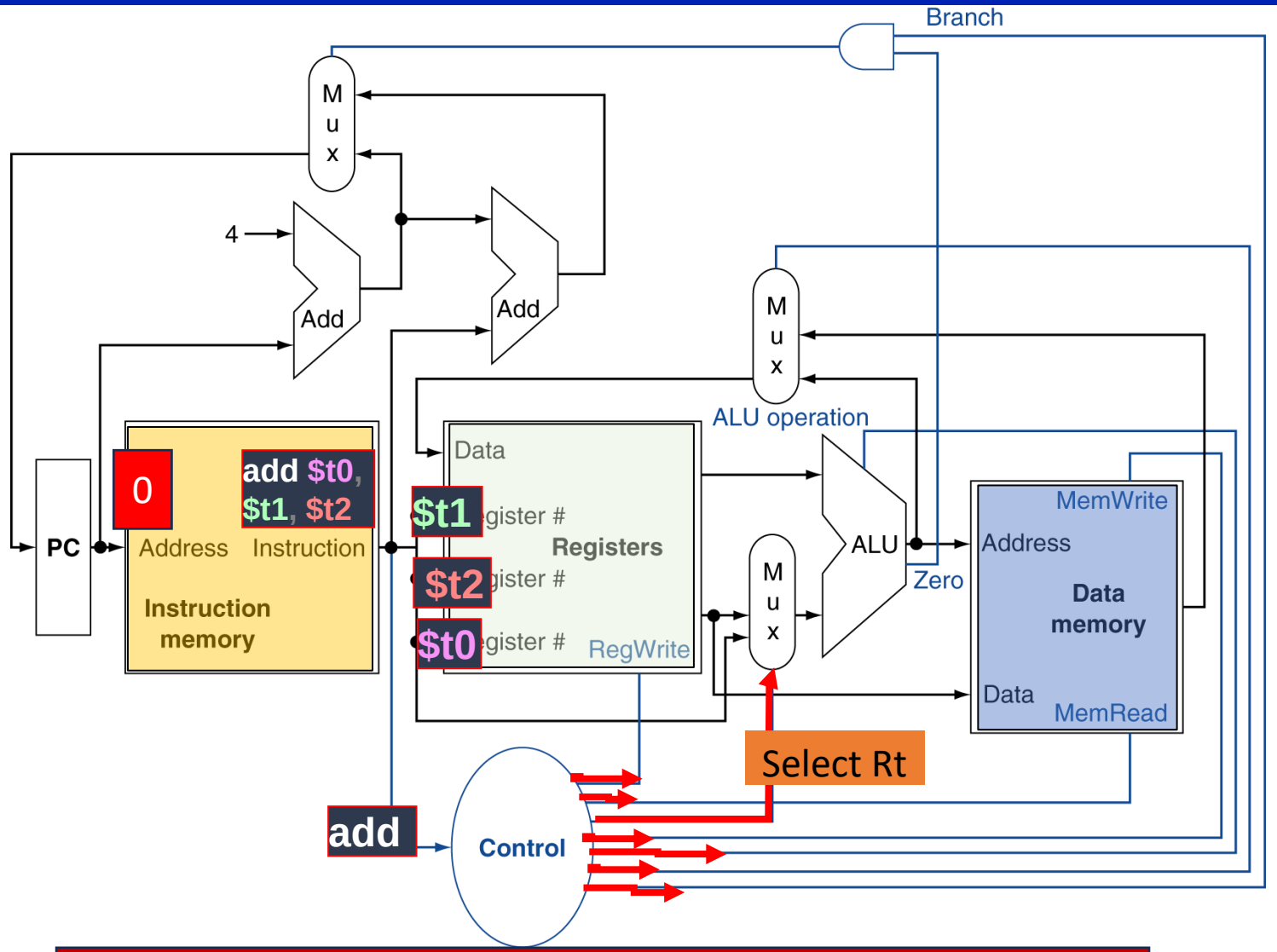
Data Memory

Address	Data
40	0
44	0
48	0

Instruction gets broken down into segments and goes to register and control units

Decode





Control signals are asserted

Register

$t0	0
$t1	40
$t2	60
$s0	200
$s1	
$s2	

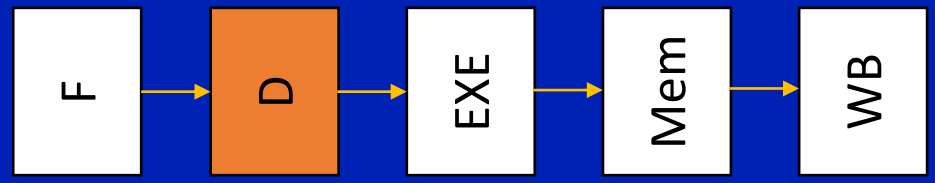
Inst. Memory

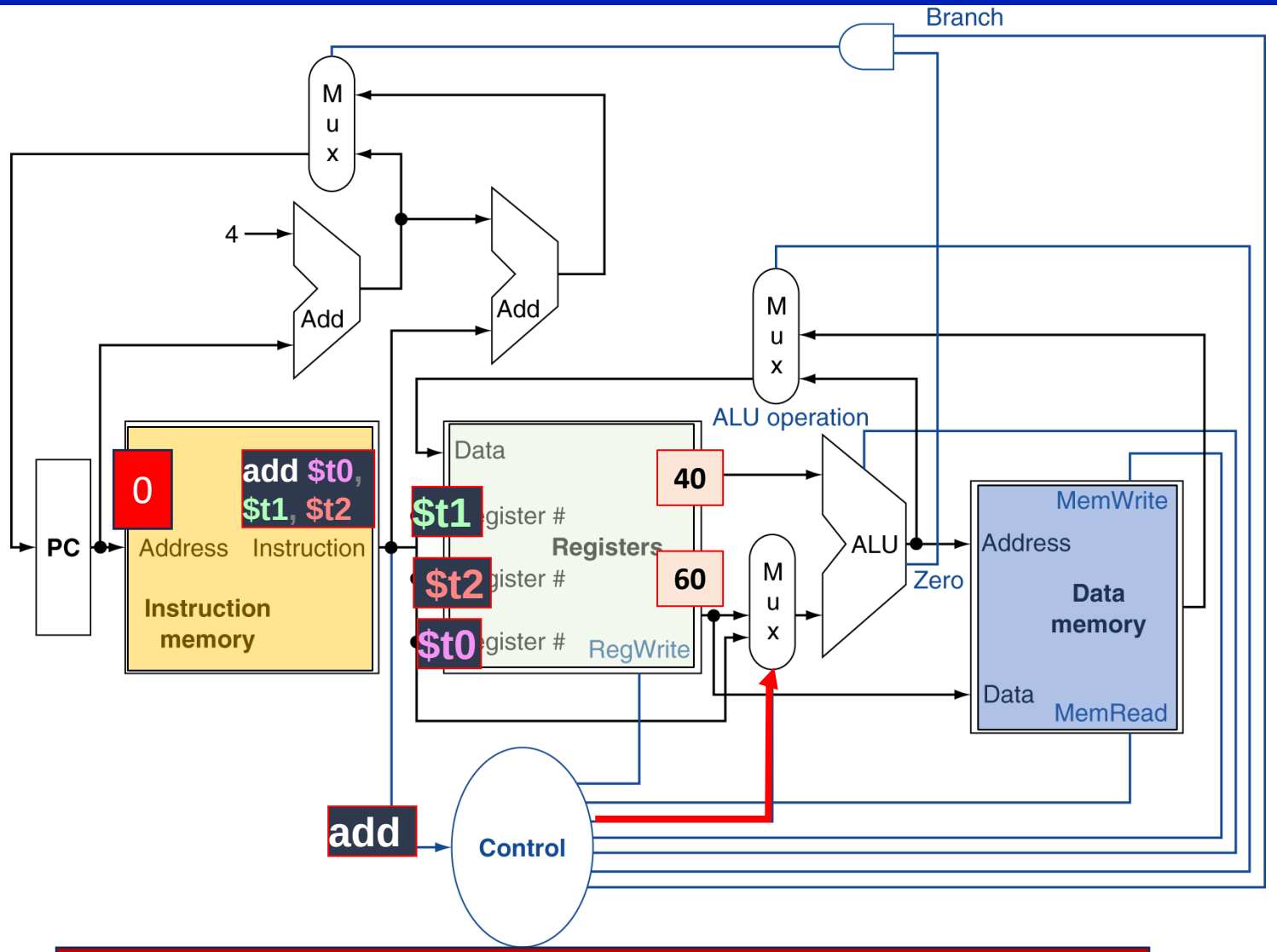
Address	Data
0	add $t0, $t1, $t2
4	sw $s0, 4($t1)
8	beq $s1, $s2, 24

Data Memory

Address	Data
40	0
44	0
48	0

Decode





Register data Rs and Rt comes out

Register		
	$t0	0
	$t1	40
	$t2	60
	$s0	200
	$s1	
	$s2	

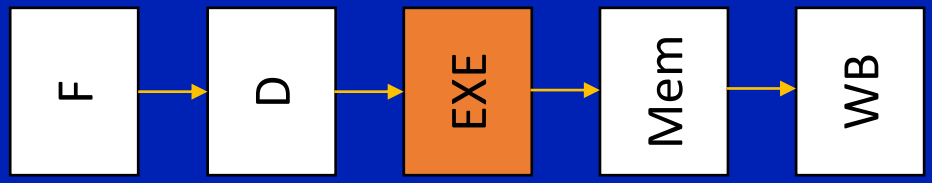
Inst. Memory

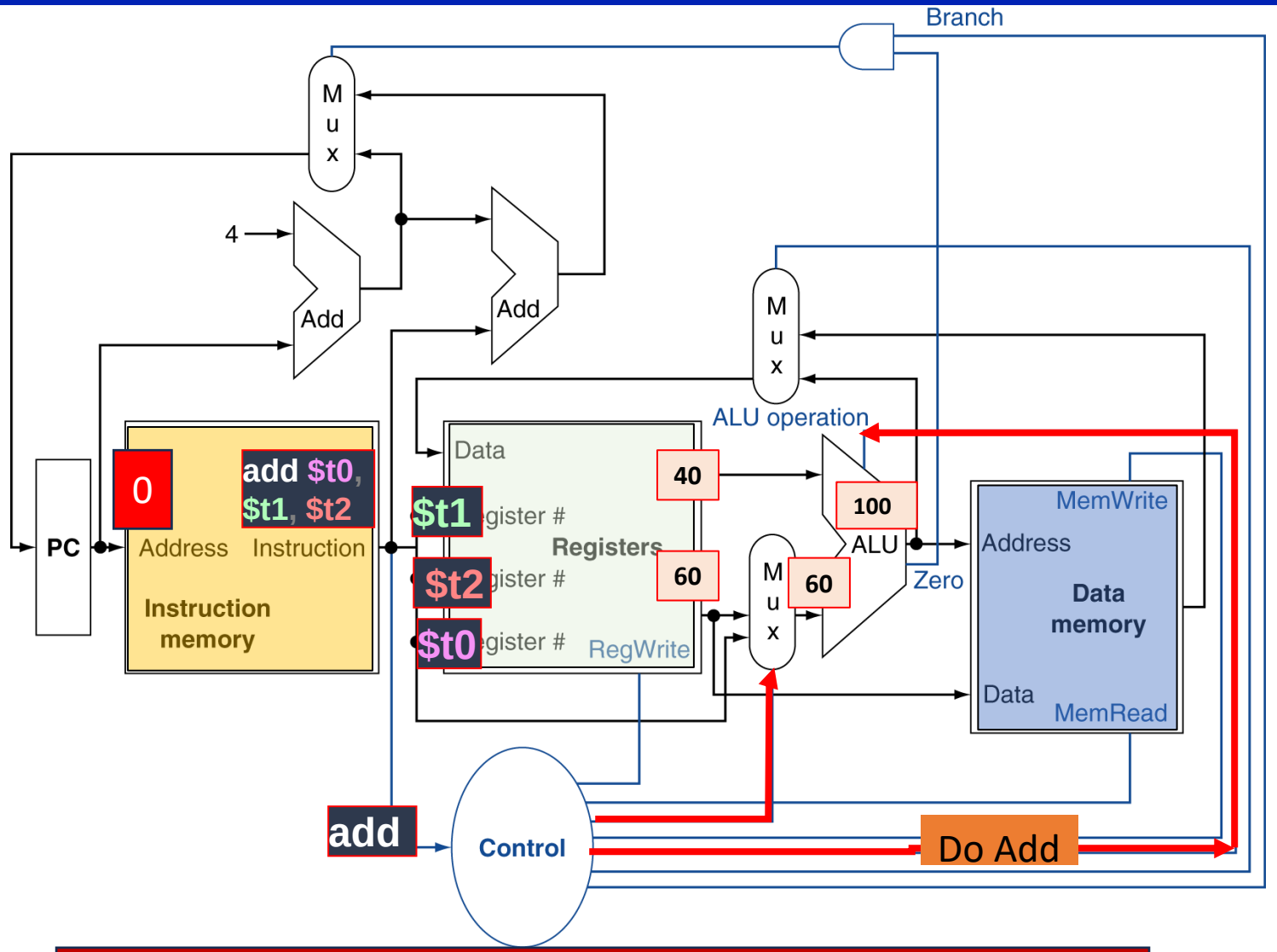
Address	Data
0	add $t0, $t1, $t2
4	sw $s0, 4($t1)
8	beq $s1, $s2, 24

Data Memory

Address	Data
40	0
44	0
48	0

Decode





ALU Performs Add

Register

$t0	0
$t1	40
$t2	60
$s0	200
$s1	
$s2	

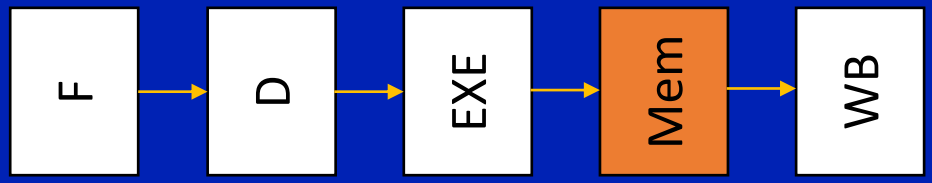
Inst. Memory

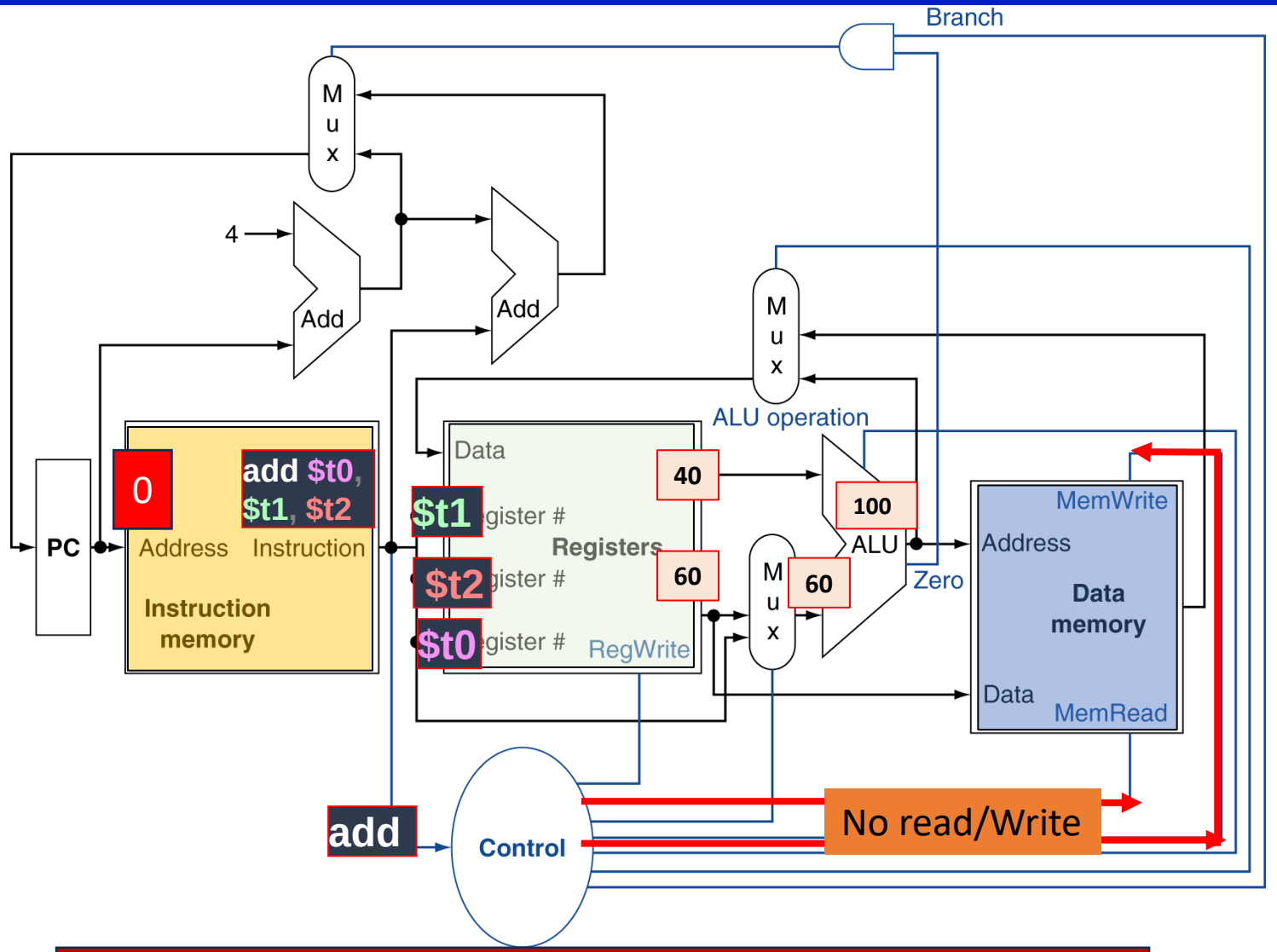
Address	Data
0	add $t0, $t1, $t2
4	sw $s0, 4($t1)
8	beq $s1, $s2, 24

Data Memory

Address	Data
40	0
44	0
48	0

Mem





Mem stage only needed for Load Store

Register

$t0	0
$t1	40
$t2	60
$s0	200
$s1	
$s2	

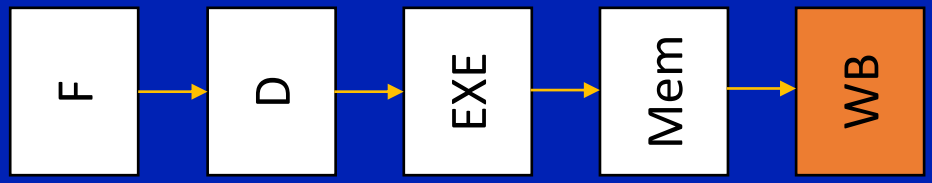
Inst. Memory

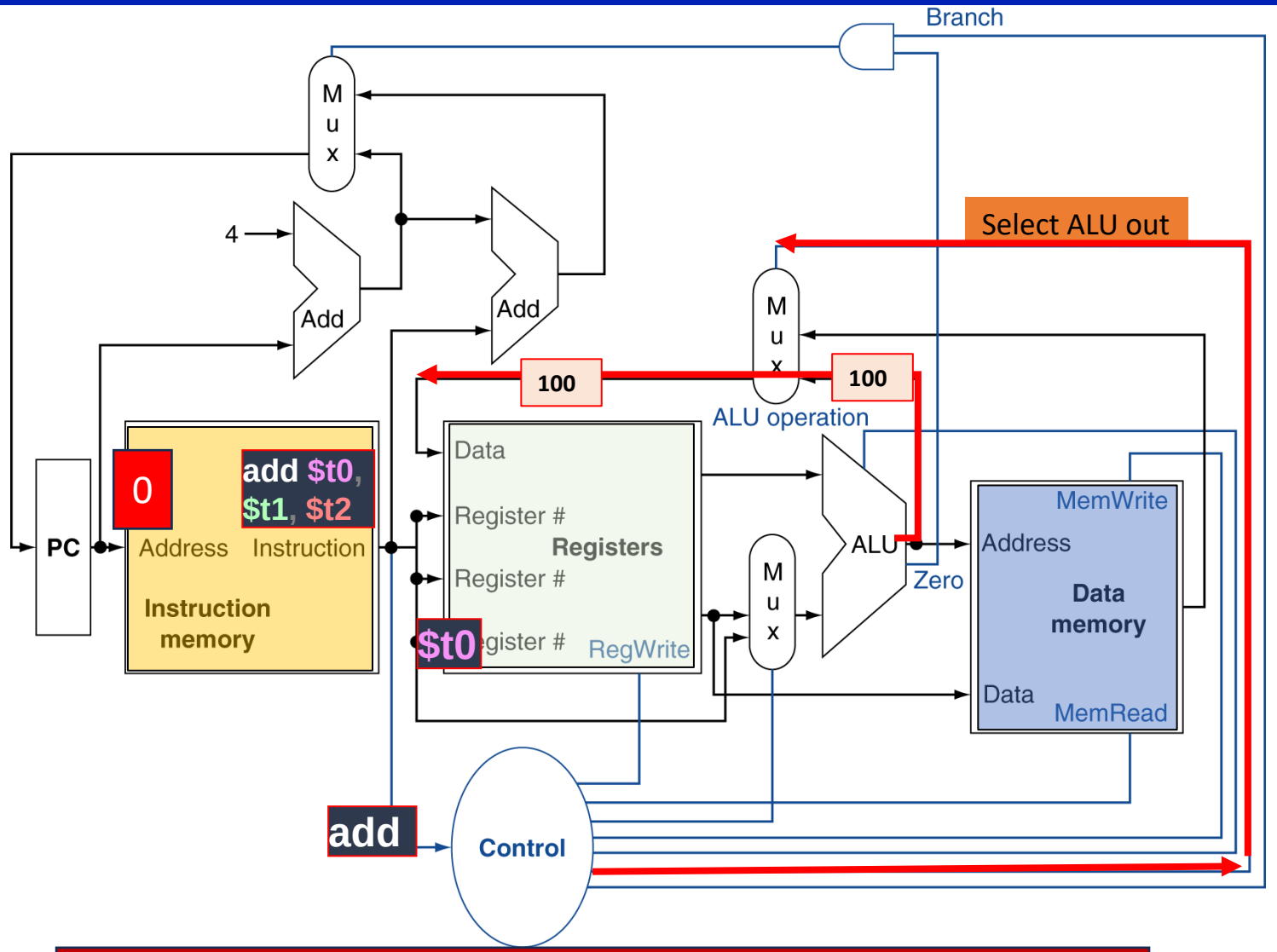
Address	Data
0	add $t0, $t1, $t2
4	sw $s0, 4($t1)
8	beq $s1, $s2, 24

Data Memory

Address	Data
40	0
44	0
48	0

Write Back





Write Back the result to $t0

Register

$t0	100
$t1	40
$t2	60
$s0	200
$s1	
$s2	

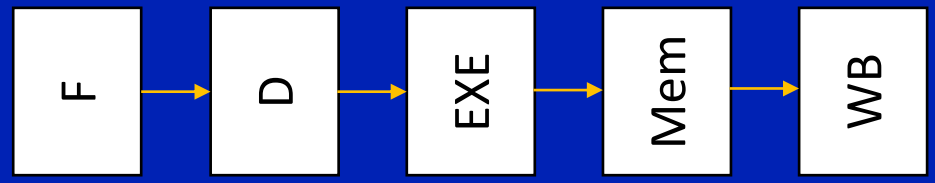
Inst. Memory

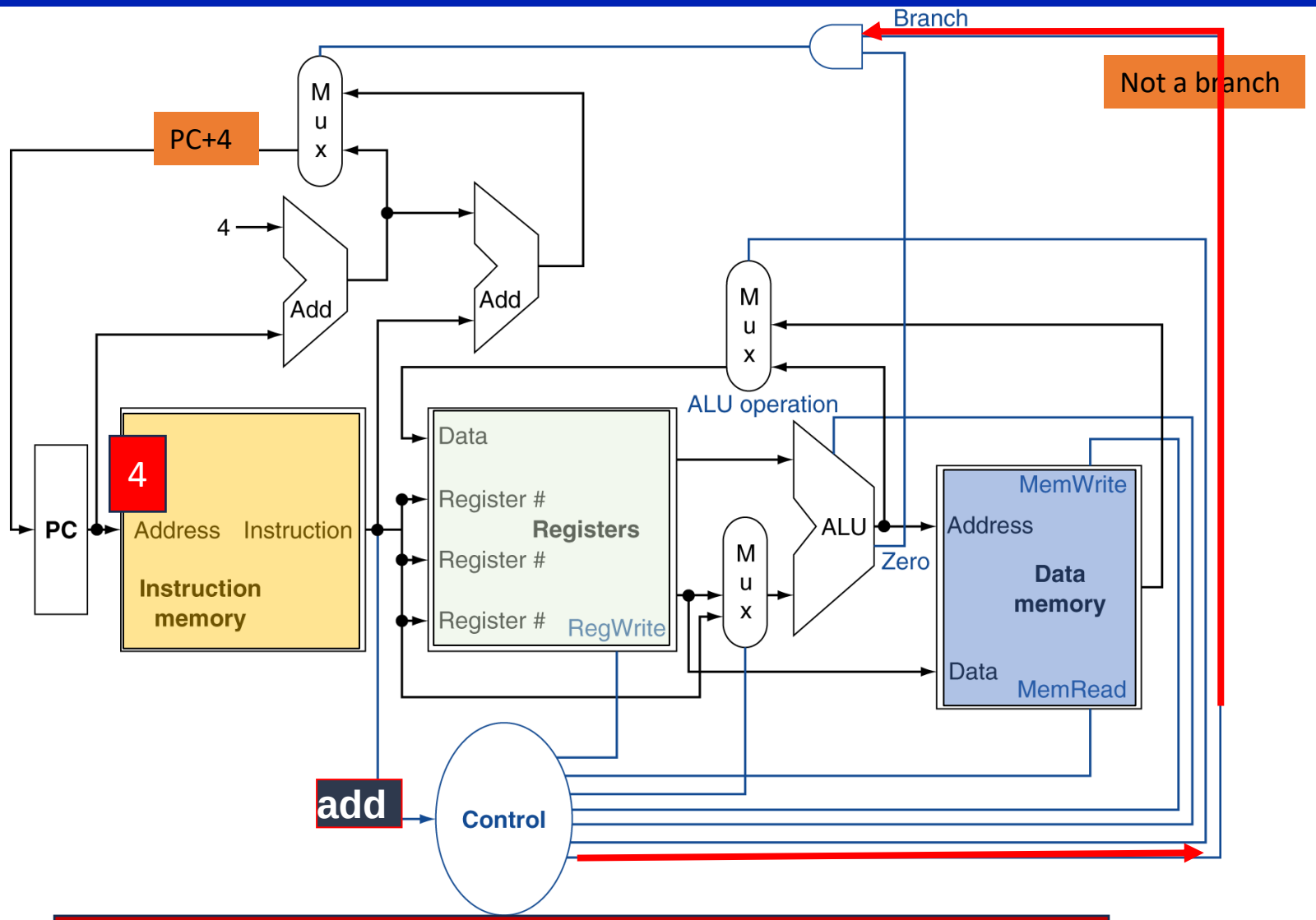
Address	Data
0	add $t0, $t1, $t2
4	sw $s0, 4($t1)
8	beq $s1, $s2, 24

Data Memory

Address	Data

Anytime





Update PC to PC+4 before next fetch

Register	
$t0	100
$t1	40
$t2	60
$s0	200
$s1	
$s2	

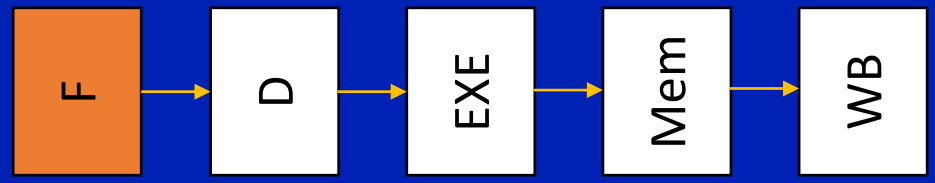
Inst. Memory

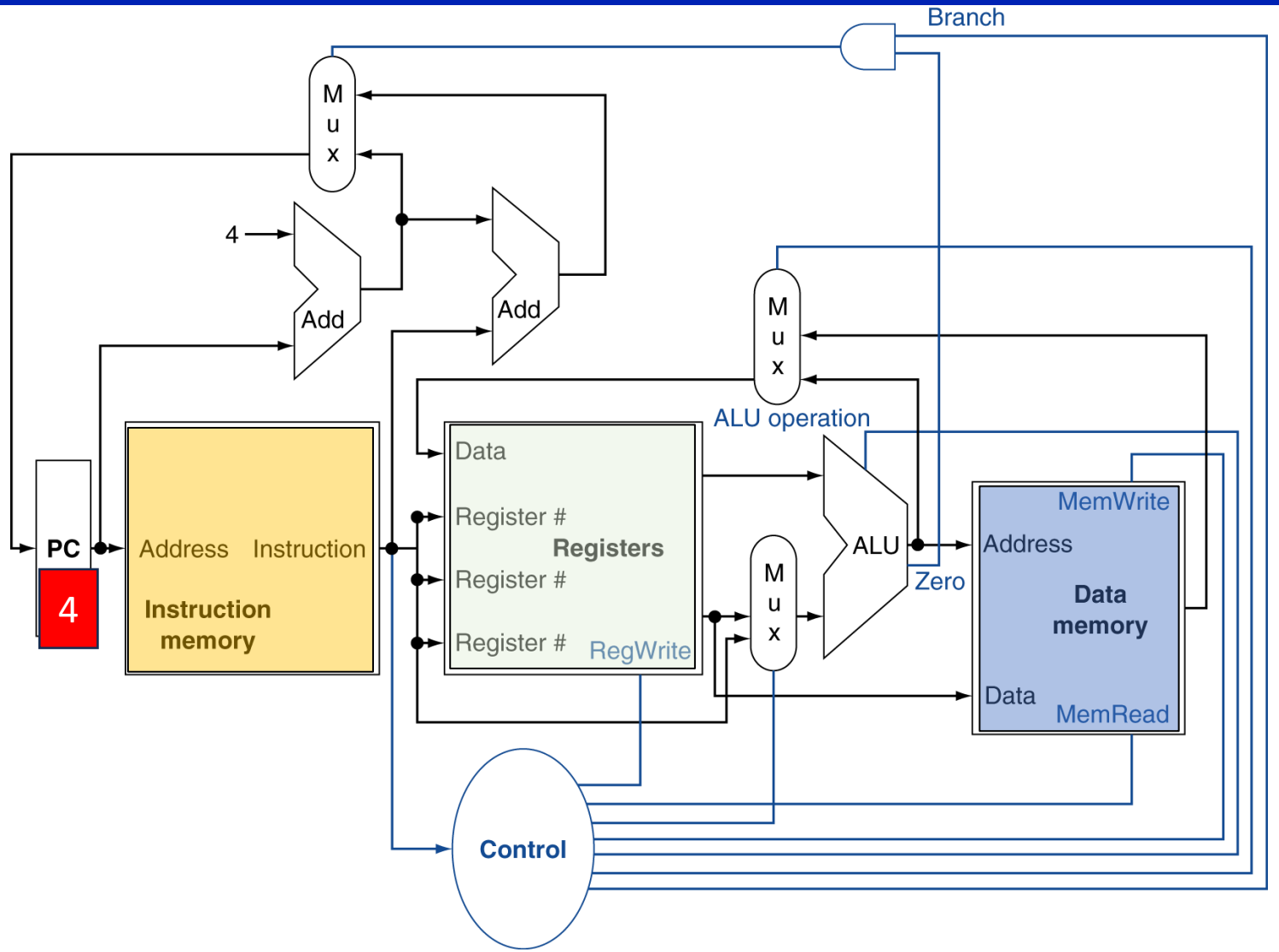
Address	Data
0	add $t0, $t1, $t2
4	sw $s0, 4($t1)
8	beq $s1, $s2, 24

Data Memory

Address	Data

Fetch Next Inst





Register

$t0	100
$t1	40
$t2	60
$s0	200
$s1	
$s2	

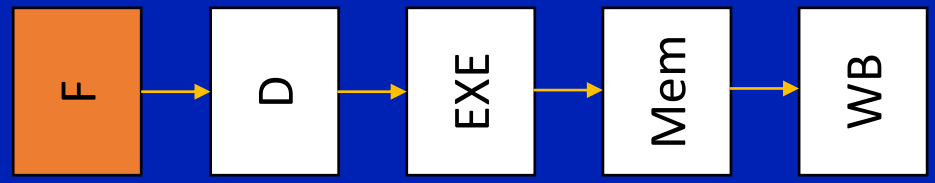
Inst. Memory

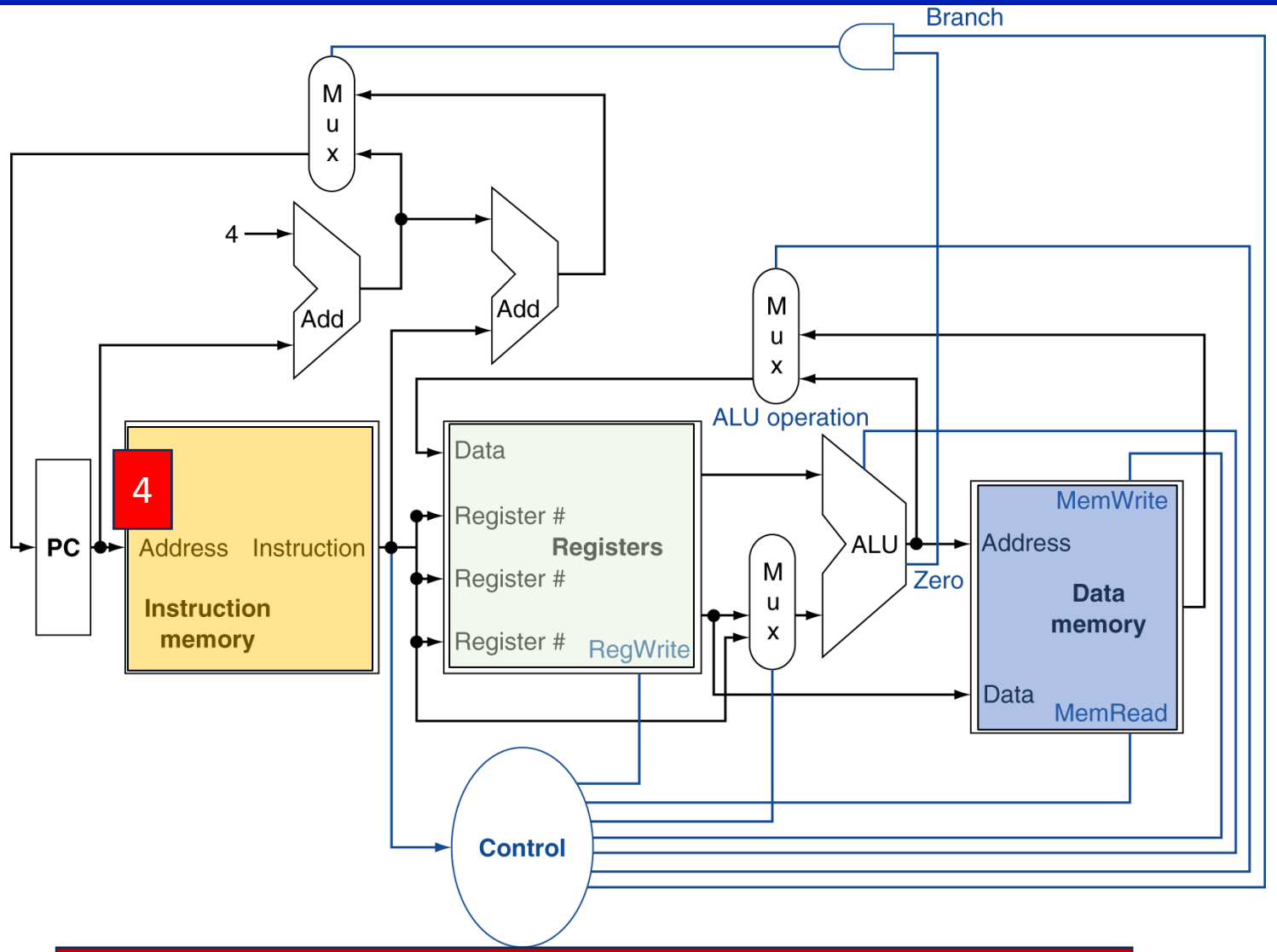
Address	Data
0	add $t0, $t1, $t2
4	sw $s0, 4($t1)
8	beq $s1, $s2, 24

Data Memory

Address	Data

Fetch:





Program counter starts from 0

Register

$t0	100
$t1	40
$t2	60
$s0	200
$s1	
$s2	

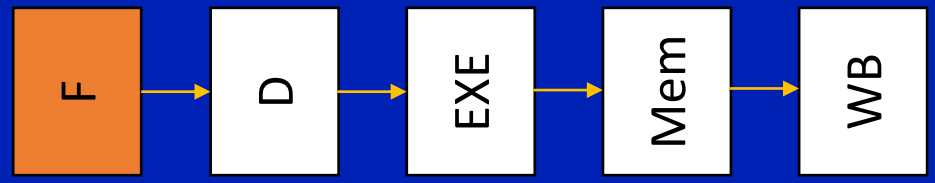
Inst. Memory

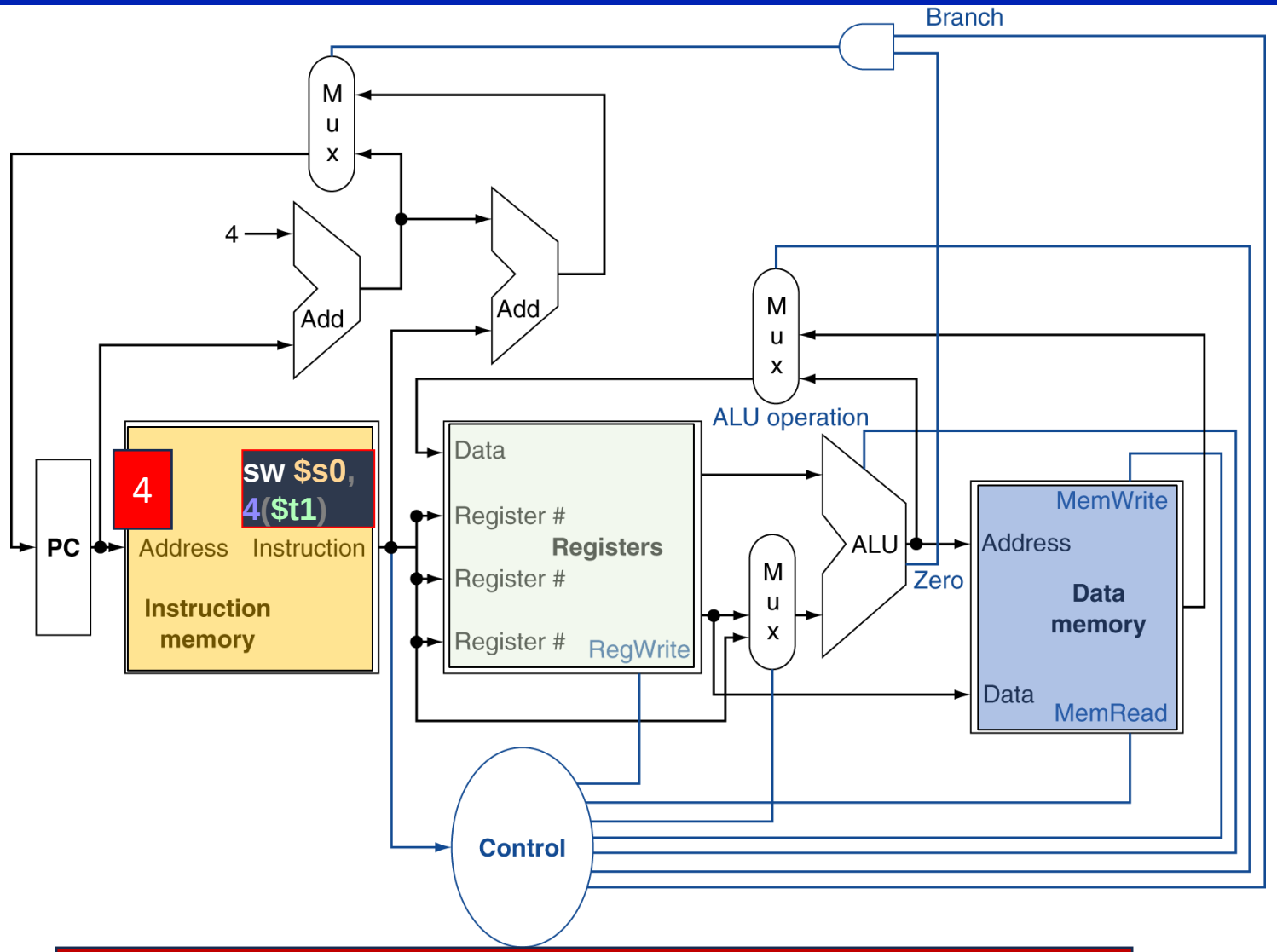
Address	Data
0	add $t0, $t1, $t2
4	sw $s0, 4($t1)
8	beq $s1, $s2, 24

Data Memory

Address	Data
40	0
44	0
48	0

Fetch





Register	
$t0	100
$t1	40
$t2	60
$s0	200
$s1	
$s2	

Inst. Memory

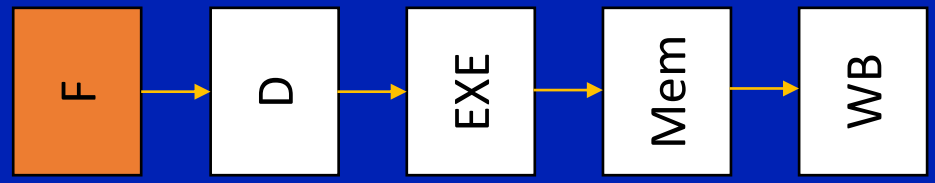
Address	Data
0	add $t0, $t1, $t2
4	sw $s0, 4($t1)
8	beq $s1, $s2, 24

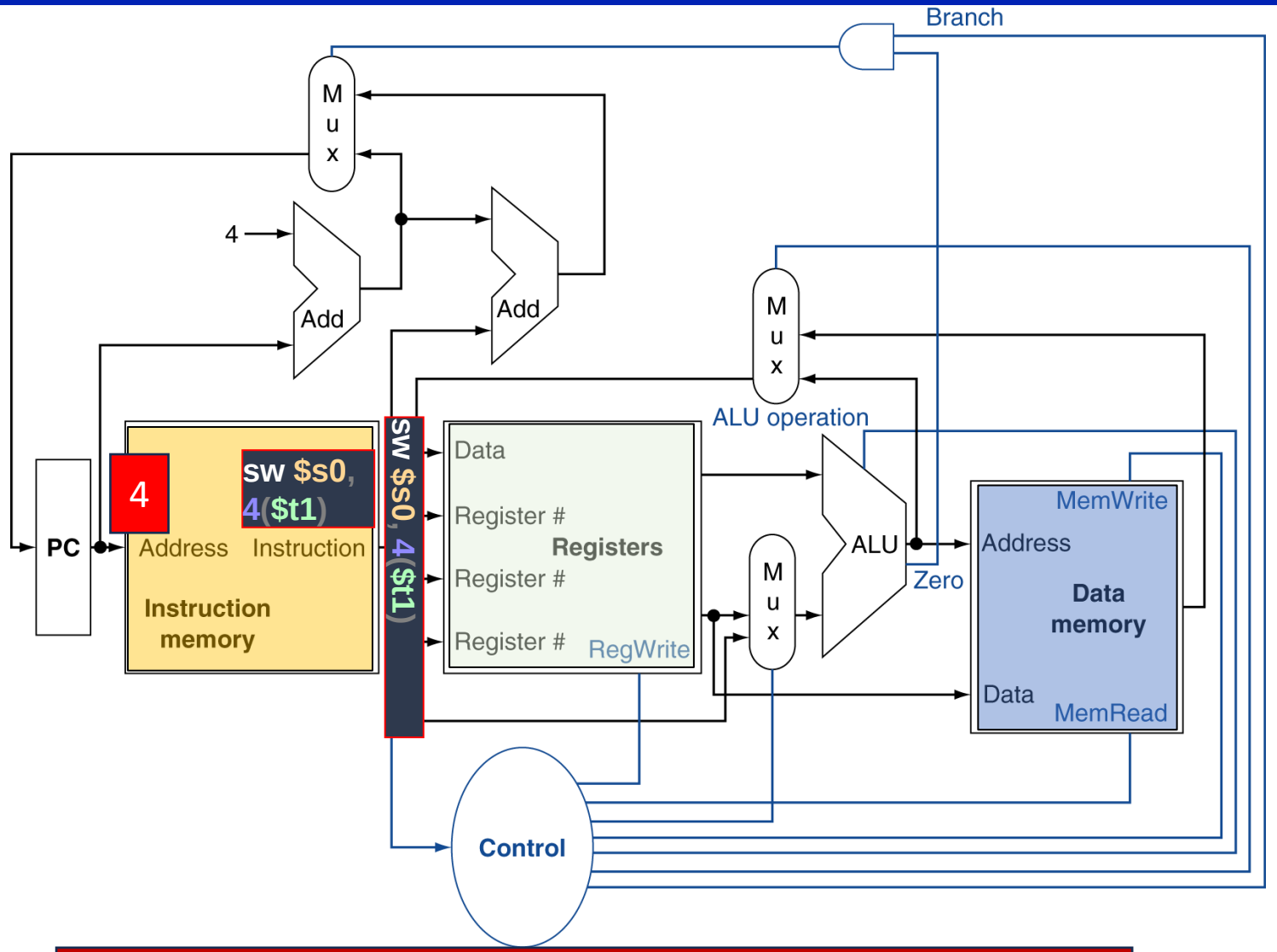
Data Memory

Address	Data
40	0
44	0
48	0

Inst mem gets 4 as input, outputs the content at address 4

Fetch





Content at address 4 is the first instruction

Register

$t0	100
$t1	40
$t2	60
$s0	200
$s1	
$s2	

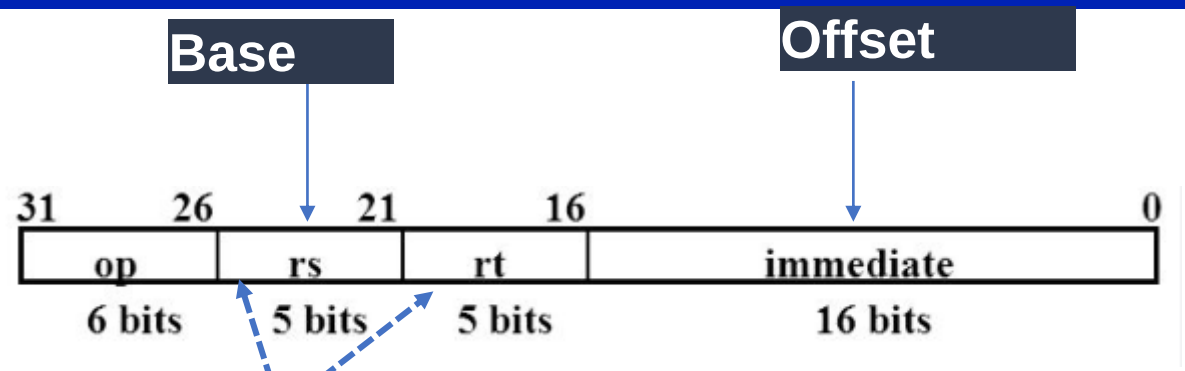
Inst. Memory

Address	Data
0	add $t0, $t1, $t2
4	sw $s0, 4($t1)
8	beq $s1, $s2, 24

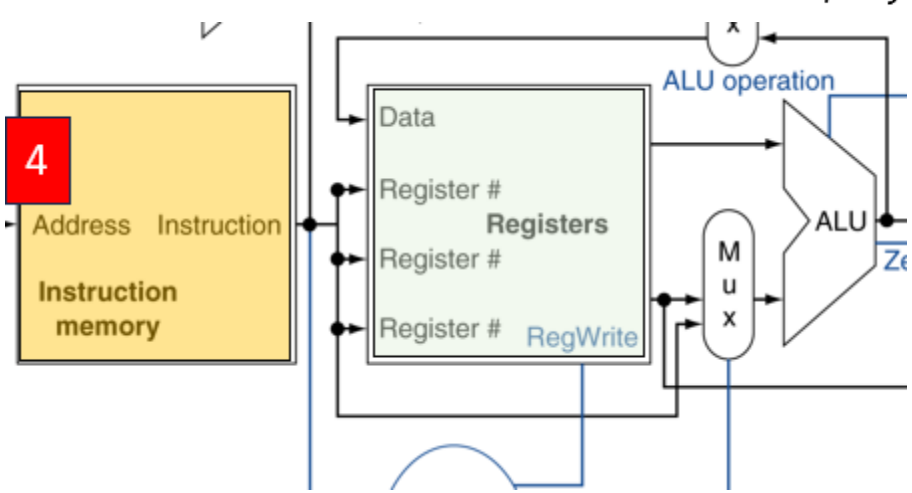
Data Memory

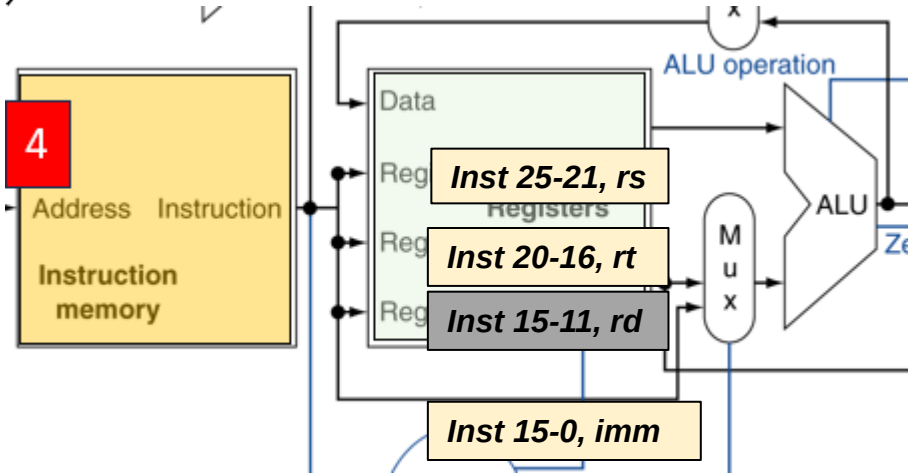
Address	Data
40	0
44	0
48	0

Decode for SW/LW

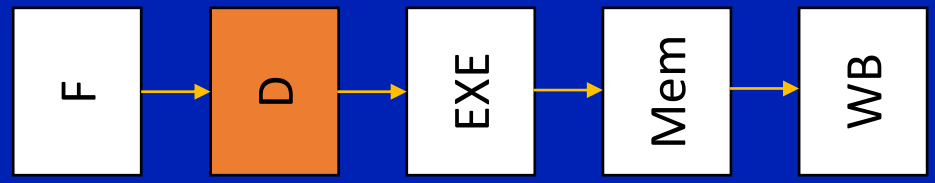


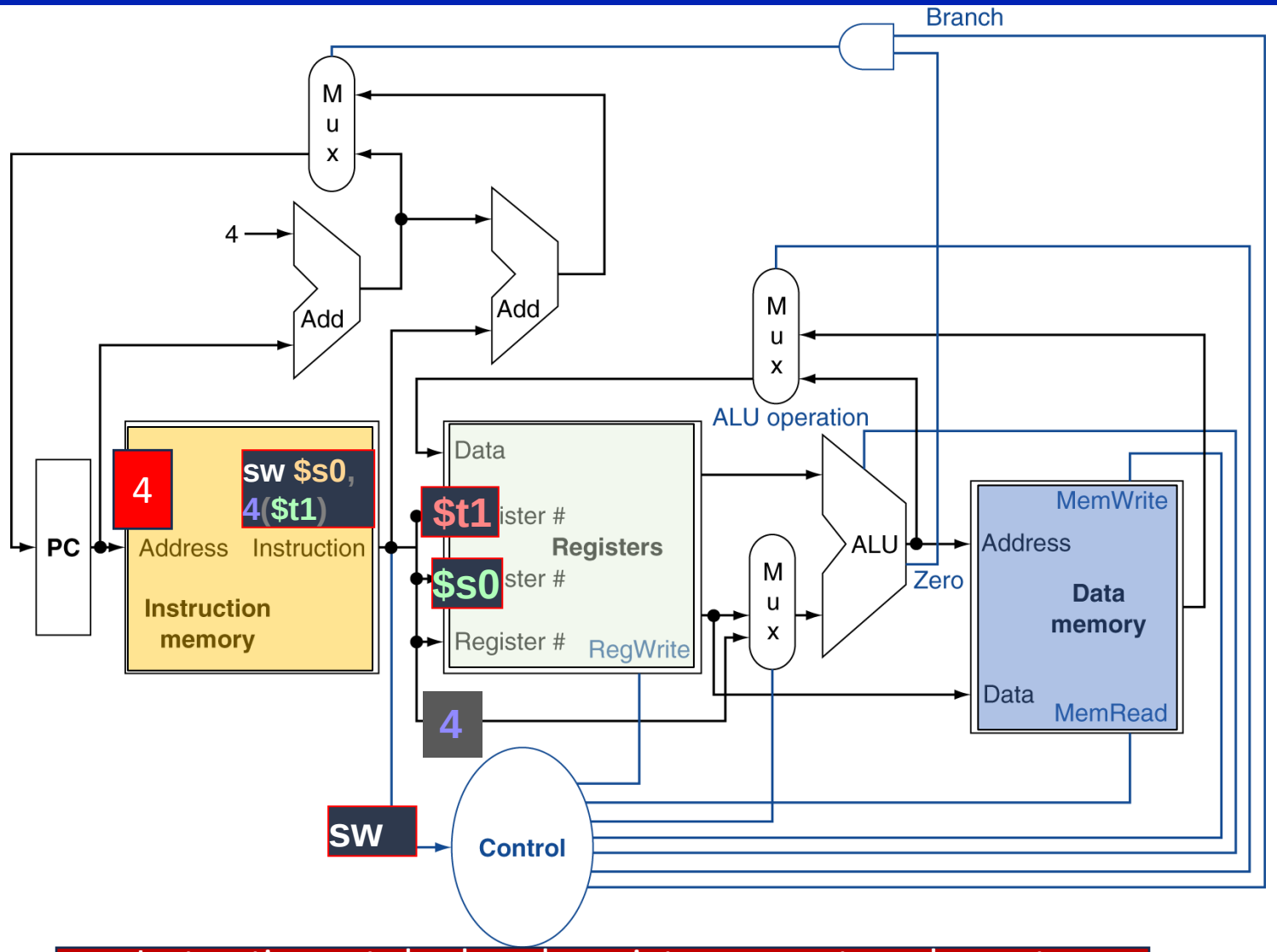

```
lw $rt, immed($rs)
sw $rt, immed($rs)
```





Decode





Register	
$t0	100
$t1	40
$t2	60
$s0	200
$s1	
$s2	

Inst. Memory

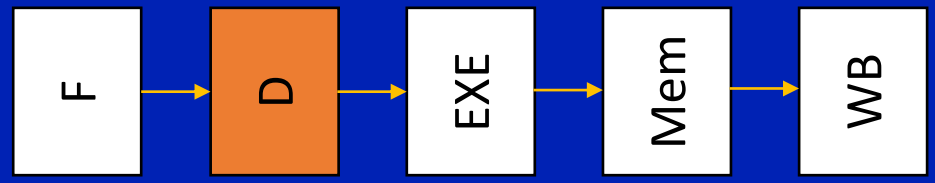
Address	Data
0	add $t0, $t1, $t2
4	sw $s0, 4($t1)
8	beq $s1, $s2, 24

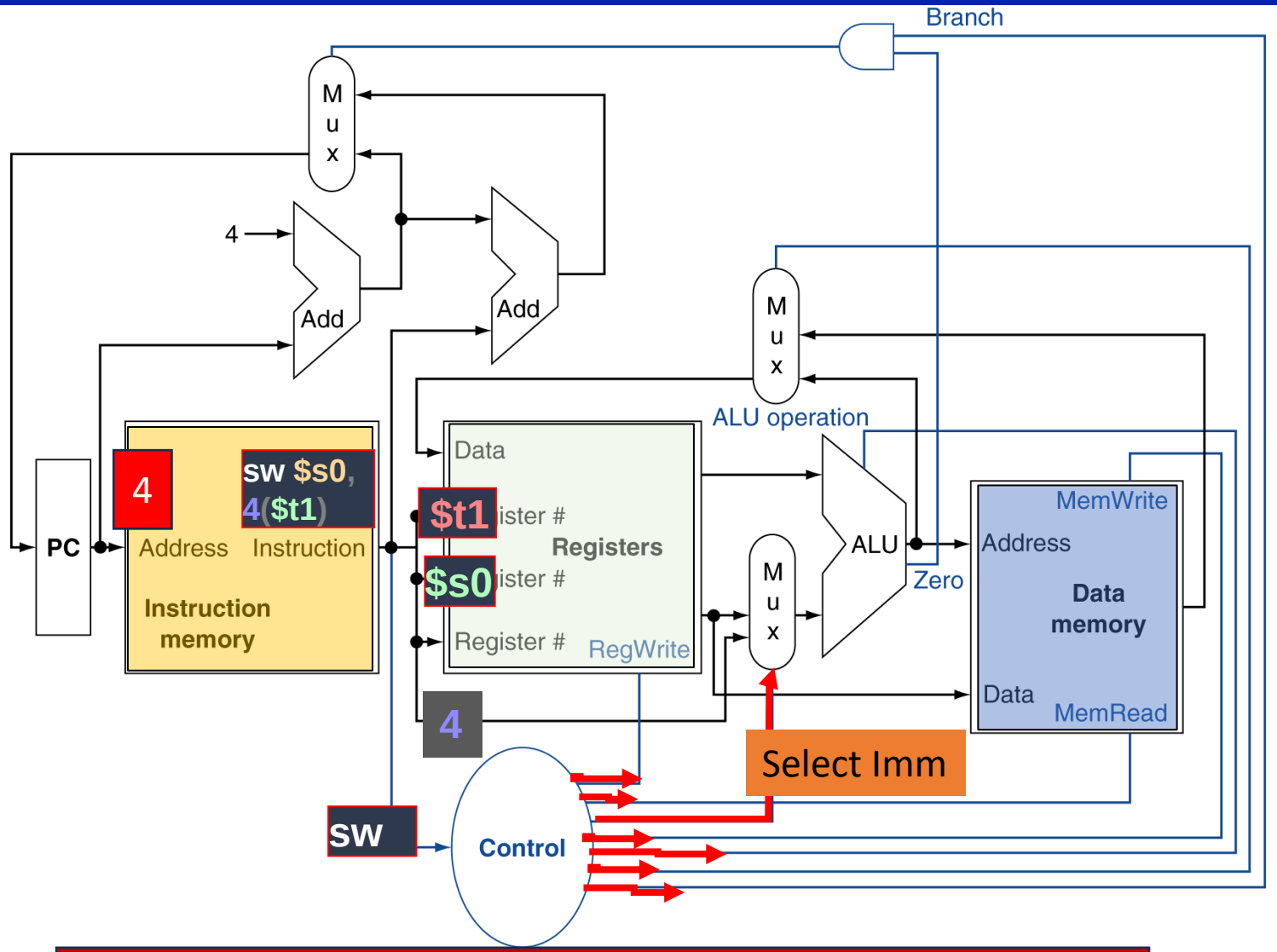
Data Memory

Address	Data
40	0
44	0
48	0

Instruction gets broken down into segments and goes to register and control units

Decode





Control signals are asserted

Register

$t0	100
$t1	40
$t2	60
$s0	200
$s1	
$s2	

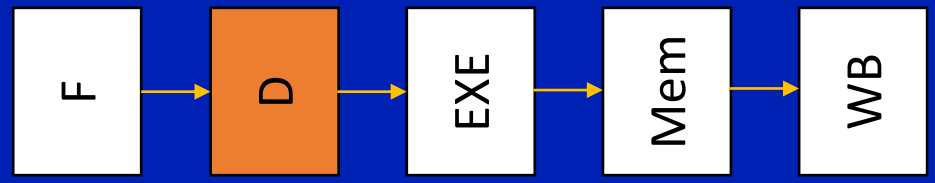
Inst. Memory

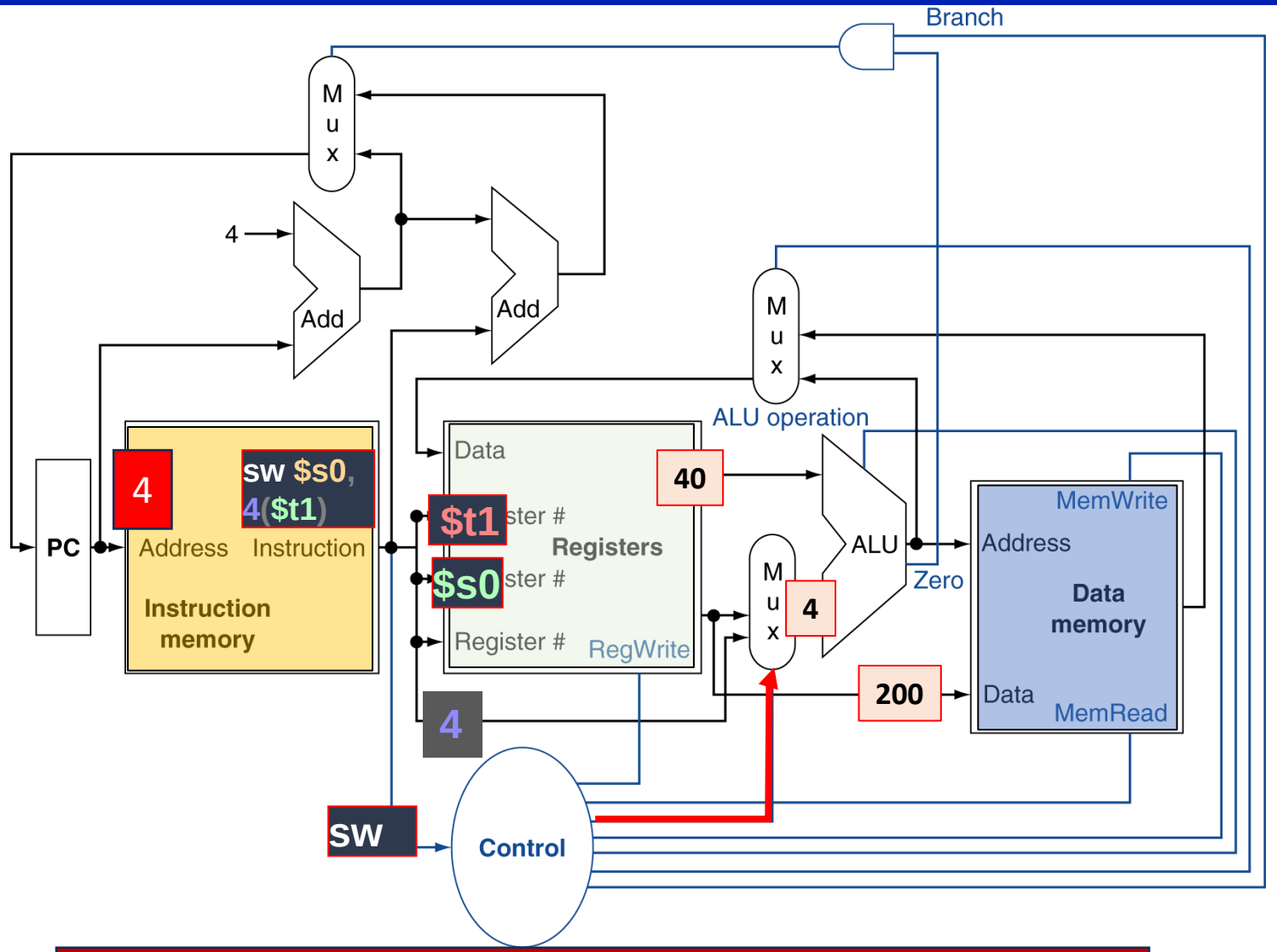
Address	Data
0	add $t0, $t1, $t2
4	sw $s0, 4($t1)
8	beq $s1, $s2, 24

Data Memory

Address	Data
40	0
44	0
48	0

Decode





Register data comes out

Register		
	$t0	100
	$t1	40
	$t2	60
	$s0	200
	$s1	
	$s2	

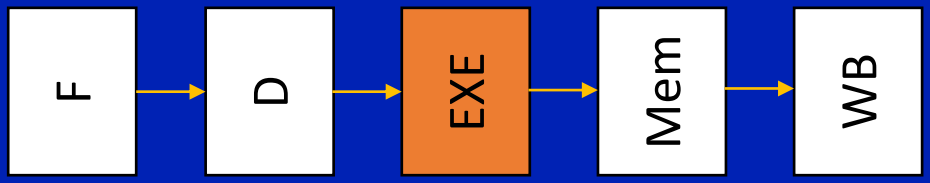
Inst. Memory

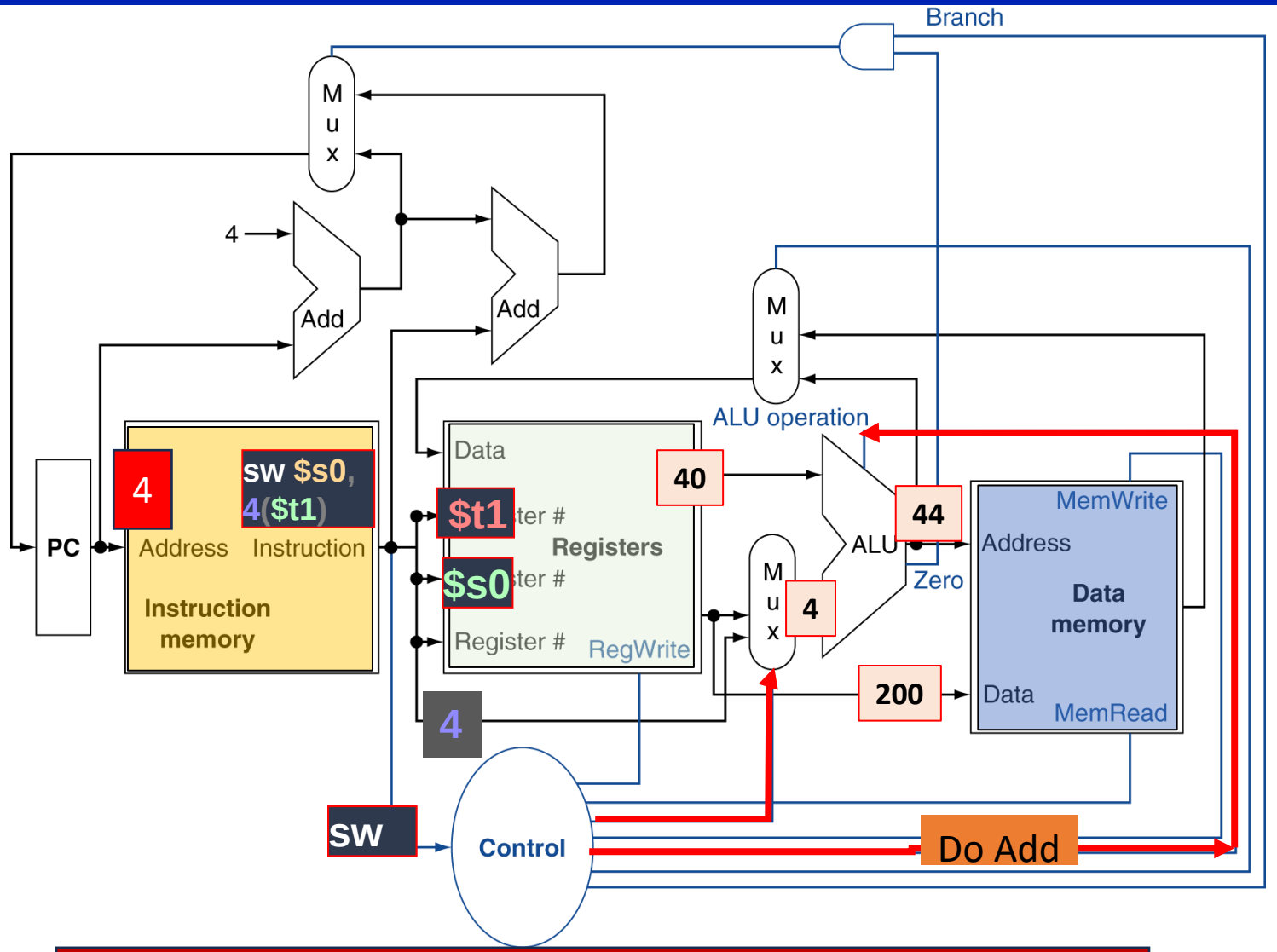
Address	Data
0	add $t0, $t1, $t2
4	sw $s0, 4($t1)
8	beq $s1, $s2, 24

Data Memory

Address	Data
40	0
44	0
48	0

Decode





ALU Performs Add

Register

$t0	100
$t1	40
$t2	60
$s0	200
$s1	
$s2	

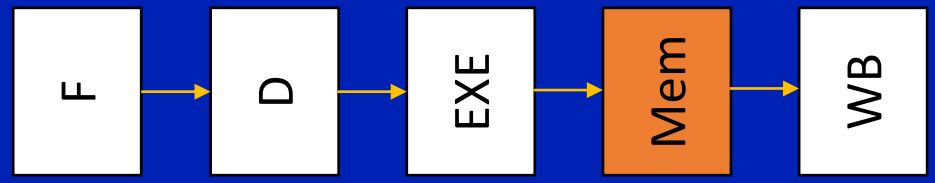
Inst. Memory

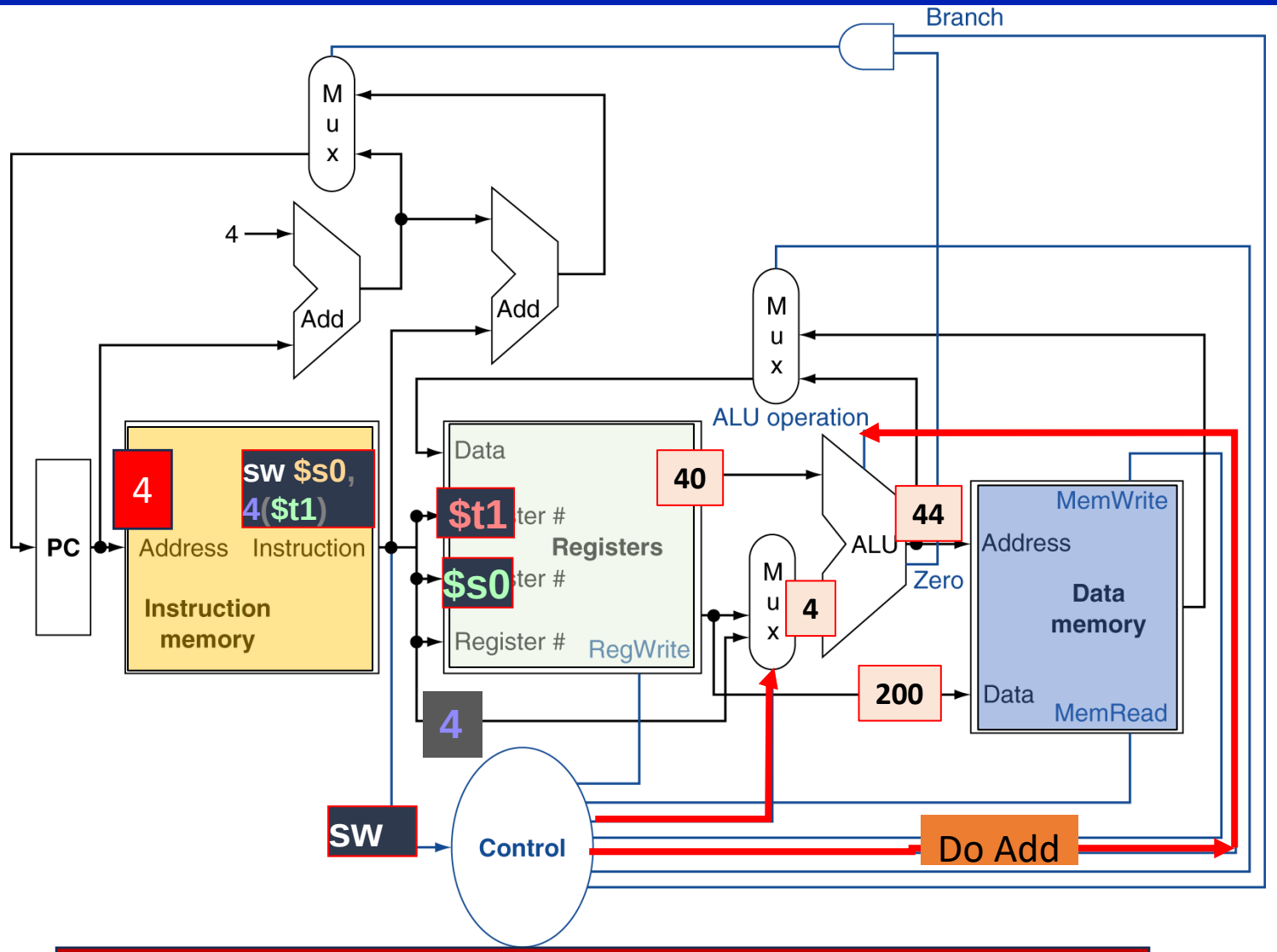
Address	Data
0	add $t0, $t1, $t2
4	sw $s0, 4($t1)
8	beq $s1, $s2, 24

Data Memory

Address	Data
40	0
44	0
48	0

Decode





Store data to address 44

Register

$t0	100
$t1	40
$t2	60
$s0	200
$s1	
$s2	

Inst. Memory

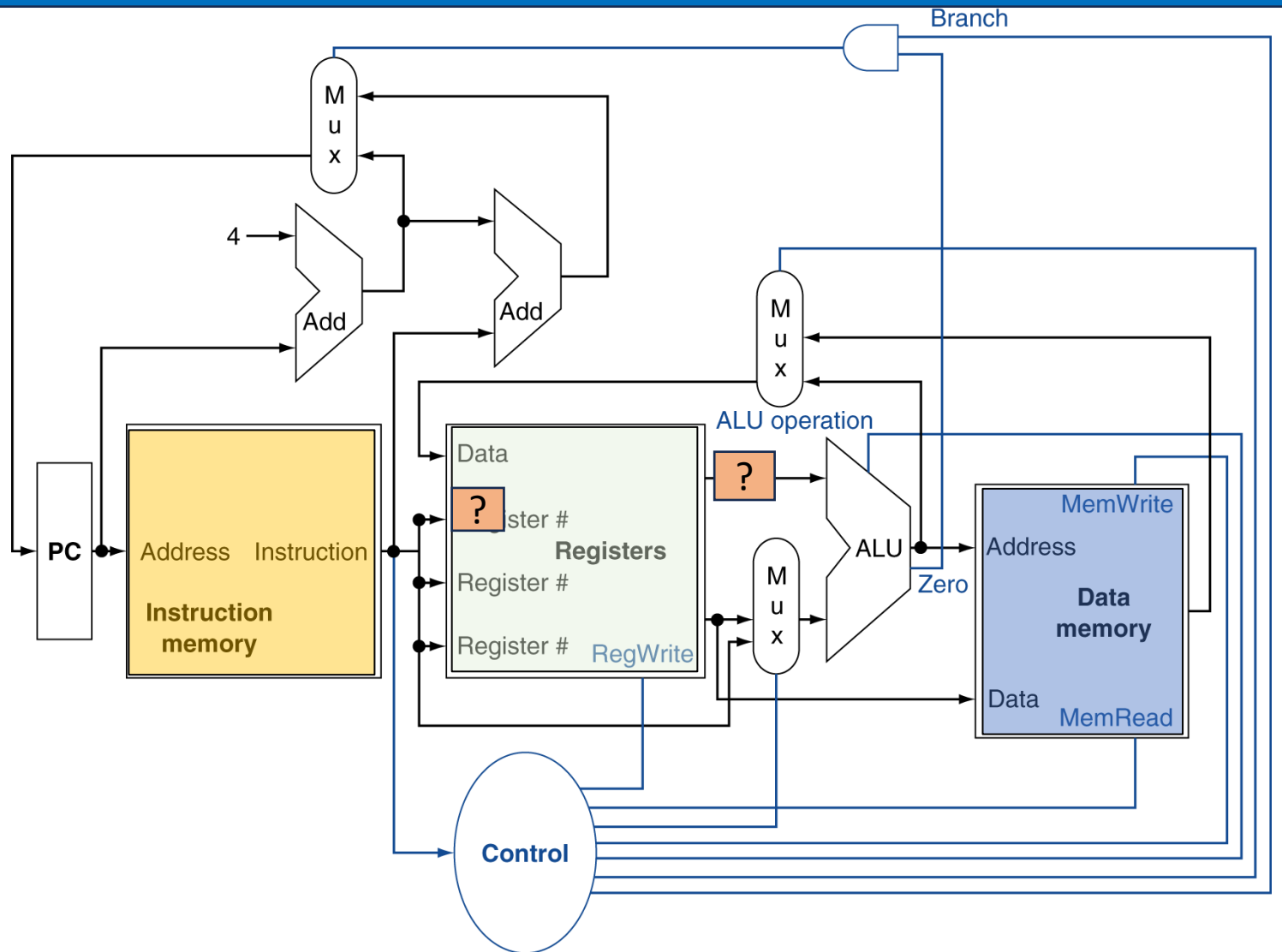
Address	Data
0	add $t0, $t1, $t2
4	sw $s0, 4($t1)
8	beq $s1, $s2, 24

Data Memory

Address	Data
40	0
44	200
48	0

Example Question

Calculate the values in the box after all stages of instruction sw $s0, 4($t1) are done?

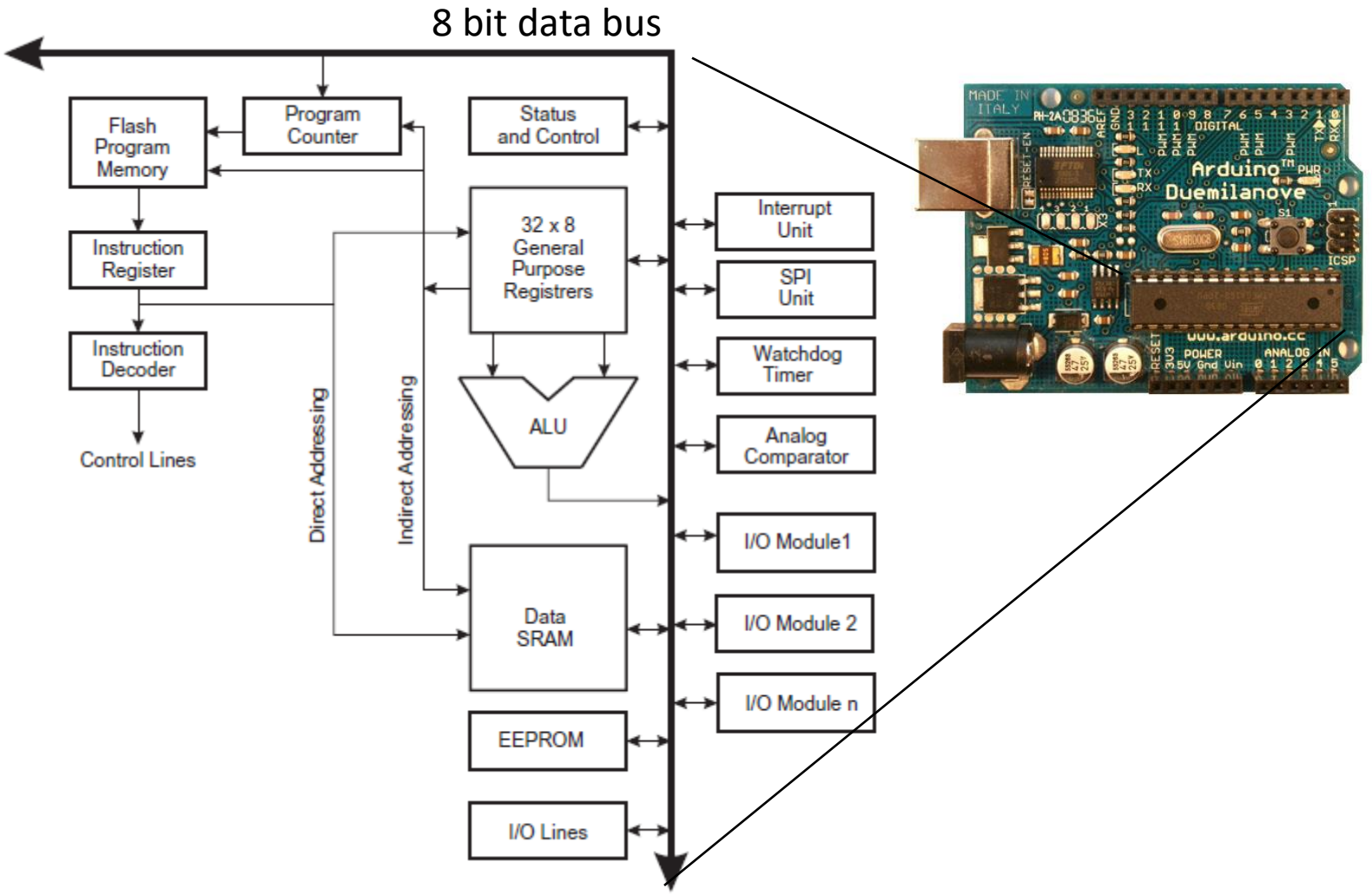


Register	
$t0	100
$t1	40
$t2	60
$s0	200
$s1	
$s2	

Data Memory

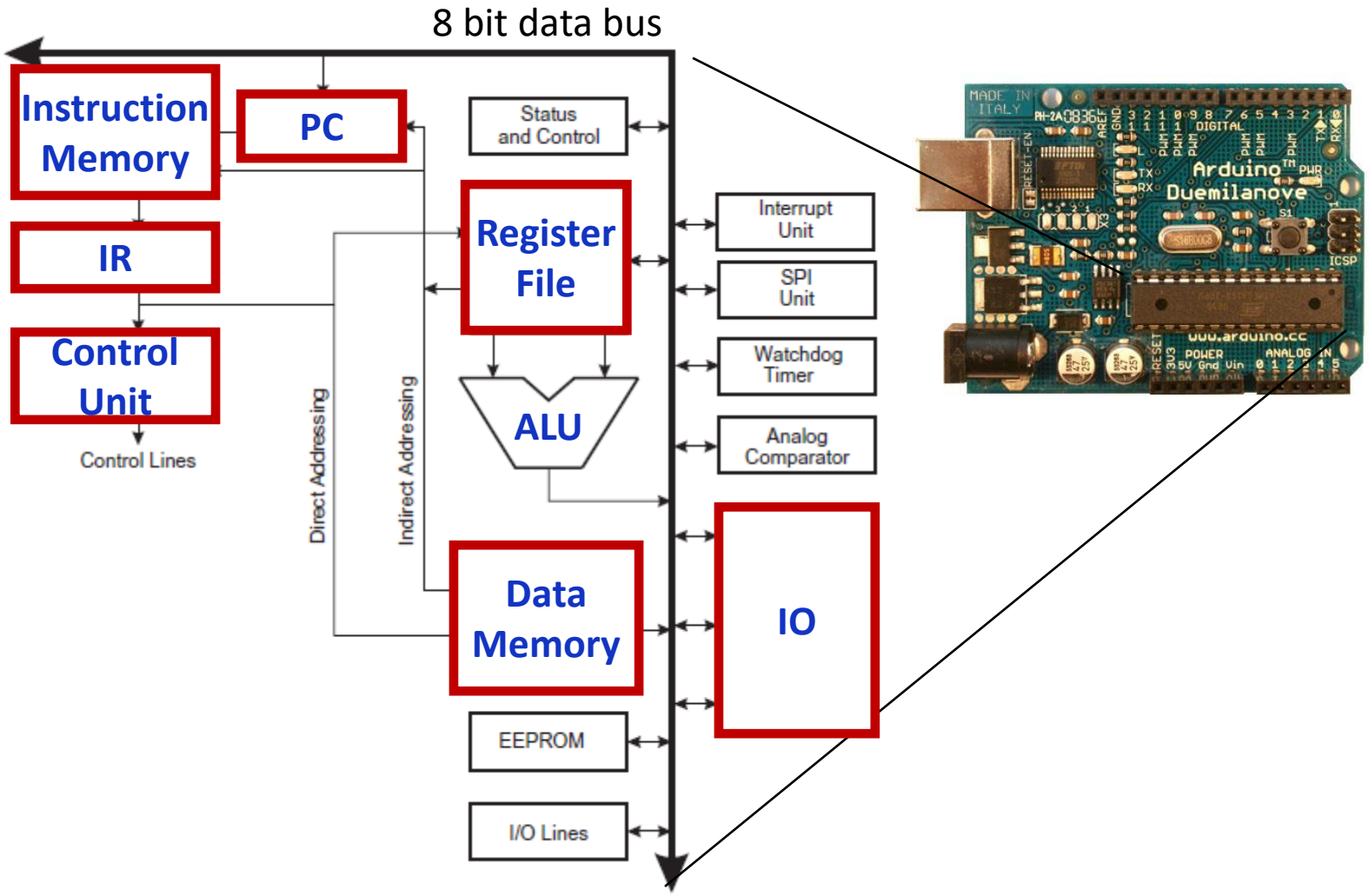
Address	Data
40	0
44	?
48	0

Case Study: Atmel ATmega168



Atmel ATmega168 datasheet

Case Study: Atmel ATmega168



Atmel ATmega168 datasheet